THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms.**THAMIZHSELVI M (20524UCSC062)**, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures:		

	• **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01 Creation of a DDL operations using Books Table

DATE:19.6.2026

Aim

To create a Books table, insert book records, display all records, and retrieve books published from the year 2000 onwards.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects.

DDL COMMANDS:

Create

Alter

Drop

Truncate

Rename

CREATE

Create command is used to create a new table and apply constraints.

Syntax:

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
CREATE TABLE books (
```

```
    id INTEGER PRIMARY KEY,
    title TEXT,
    publish_year INTEGER
);
```

Table Created

ALTER

ADD: Add a new column to the table.

ALTER TABLE books ADD author VARCHAR(100);

Table Altered

MODIFY: Modify the data type or size of a column.

Oracle:

ALTER TABLE books MODIFY title VARCHAR(150);

MySQL:

ALTER TABLE books MODIFY title VARCHAR(150);

SQLite: SQLite does not directly support MODIFY COLUMN; the table generally needs to be recreated.

DROP COLUMN

ALTER TABLE books DROP COLUMN author;

RENAME

RENAME TABLE books TO library_books;

OR

ALTER TABLE books RENAME TO library_books;

Column Rename:

ALTER TABLE books RENAME COLUMN title TO book_title;

TRUNCATE

Truncate command is used to delete all records from a table.

TRUNCATE TABLE books;

DROP

Drop command deletes the table and all its data.

DROP TABLE books**;**

Explanation

Create a table named books.

Insert book details into the table.

Display all book records.

Retrieve books published in or after the year 2000 and sort them by publication year.

Create Table

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    publish_year INTEGER
);
```

Sample Data

```
INSERT INTO books VALUES (1, 'Harry Potter and the Philosopher"s Stone', 1997);
INSERT INTO books VALUES (2, 'The Da Vinci Code', 2003);
INSERT INTO books VALUES (3, 'The Alchemist', 1988);
INSERT INTO books VALUES (4, 'The Hunger Games', 2008);
INSERT INTO books VALUES (5, 'The Kite Runner', 2003);
INSERT INTO books VALUES (6, 'The Fault in Our Stars', 2012);
```

SQL Query

Query 1: Display all book records

SELECT * FROM books;

Resultant Query

ID	TITLE	PUBLISH_YEAR
1	Harry potter and the pilosphere	1997
2	The da vinci code	2003
3	The alchemist	1988
4	The hunger games	2008
5	The kite runner	2003

Query 2: Display books published from the year 2000 onwards

SELECT *

FROM books

WHERE publish_year >= 2000

ORDER BY publish_year;

Resultant Query

ID	TITLE	PUBLISH_YEAR
2	The da vinci code	2003
5	The kite runner	2003
4	The hunger games	2008
6	The fault in our stars	2012

RESULT

Thus, the Books table was created successfully, records were inserted, all book details were displayed, and the books published from the year 2000 onwards were retrieved successfully.

EX. NO:02 Creation of DML Operations using Employee Table

DATE:25.6.2026

Aim

To create an Employee table, insert employee records, display all records, update employee details, delete an employee record, and retrieve employees based on their salary.

Software Used

Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML Commands

- INSERT
- UPDATE
- DELETE .
- SELECT

Create Table

```
CREATE TABLE employee (
   id INTEGER PRIMARY KEY,
   name TEXT,
   department TEXT,
   salary INTEGER
);
```

Output:

Table Created

INSERT RECORDS

INSERT INTO employee VALUES (1, 'Kavin', 'IT', 30000);

INSERT INTO employee VALUES (2, 'Meena', 'HR', 25000);

INSERT INTO employee VALUES (3, 'Arun', 'IT', 40000);

INSERT INTO employee VALUES (4, 'Divya', 'Sales', 28000);

Output:

Records Inserted

Query 1: Display All Employee Records

SELECT * FROM employee;

Result:

ID	NAME	DEPARTMENT	SALARY
1	Kavin	IT	30000
2	Meena	HR	25000
3	Arun	IT	40000
4	Divya	SALES	28000

Query 2: Update the Salary of Kavin

UPDATE employee

SET salary = 35000

WHERE id = 1;

Output:

1 row updated.

Updated Record:

ID	NAME	DEPARTMENT	SALARY
1	KAVIN	IT	35000

Query 3: Delete Rahul's Record

```
DELETE FROM employee

WHERE id = 5;
```

Output:

1 row deleted.

Remaining Records:

ID	NAME	DEPARTMENT	SALARY
1	Kavin	IT	35000
2	Meena	HR	25000
3	Arun	IT	40000
4	Priya	Sales	28000

Query 4: Display Employees Who Earn 30000 or Above

```
SELECT *

FROM employee

WHERE salary >= 30000

ORDER BY salary;
```

Result:

ID	NAME	DEPARTMENT	SALARY
1	Kavin	IT	35000
3	Arun	IT	40000
4	Priya	IT	45000

Result

Thus, the Employee table was created successfully. The records were inserted and displayed, the salary of an employee was updated, an employee record was deleted, and employees who earn 30000 or above were retrieved successfully.

EX. NO:03 Creation of DQL Operations using Products Table

DATE: 6.7.2026

Aim

To create a Products table, insert product records, display all records, update product details, delete a product record, and retrieve products based on their price.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data from database tables.

DQL COMMANDS:

- SELECT
- WHERE
- ORDER BY
- DISTINCT
- GROUP BY
- HAVING

SELECT

SELECT command is used to retrieve data from a table.

Syntax:

SELECT column1, column2

FROM table_name;

Table Structure

CREATE TABLE products (

id INTEGER PRIMARY KEY,

product_name TEXT,

```
    category TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000);
INSERT INTO products VALUES (2, 'Smartphone', 'Electronics', 25000);
INSERT INTO products VALUES (3, 'Headphones', 'Accessories', 3000);
INSERT INTO products VALUES (4, 'Keyboard', 'Accessories', 1500);
INSERT INTO products VALUES (5, 'Smart Watch', 'Electronics', 8000);
```

Explanation

- Create a table named products.
- Insert product details into the table.
- Display all product records.
- Retrieve products whose price is greater than or equal to ₹10,000 and sort them by price.

SQL Query

Query 1: Display all product records

```
SELECT * FROM products;
```

Resultant Query

ID	PRODUCT_NAME	CATEGORY	PRICE
1	Laptop	Electronics	55000
2	Smartphone	Electronics	25000
3	Headphone	Accessories	3000
4	Keyboard	Accessories	1500
5	Smart watch	Electronics	8000

Query 2: Display products with price ₹10,000 and above

```
SELECT *

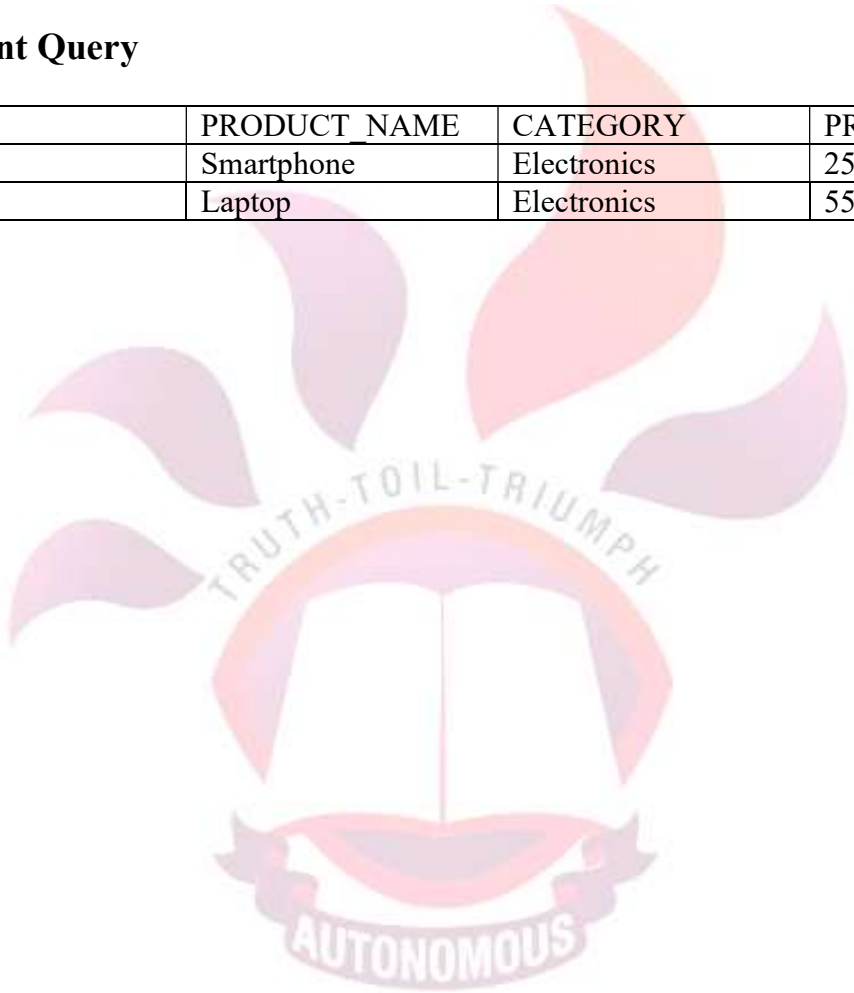
FROM products

WHERE price >= 10000

ORDER BY price;
```

Resultant Query

ID	PRODUCT_NAME	CATEGORY	PRICE
2	Smartphone	Electronics	25000
1	Laptop	Electronics	55000



RESULT

Thus, the Product table was created successfully, records were inserted, all product details were displayed, and the products with price ₹10,000 and above were retrieved successfully using DQL operations.

EX NO:04 Creation of Transaction Operations using Orders Tables

DATE:14.7.2026

Aim

To create an Orders table, insert order records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / sqlite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

COMMIT;

Query

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax

ROLLBACK;

Query

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

- Create a table named **orders**.
- Insert order details into the table.
- Use COMMIT to permanently save the inserted records.
- Update an order amount and create a SAVEPOINT.
- Make another change after the savepoint.

- Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.
- Use ROLLBACK to undo uncommitted changes.

Create Table

```
CREATE TABLE orders (
   order_id INTEGER PRIMARY KEY,
   customer_name TEXT,
   amount INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO orders VALUES (101, 'Kavin', 2500);
INSERT INTO orders VALUES (102, 'Meena', 3000);
INSERT INTO orders VALUES (103, 'Arun', 2000);
INSERT INTO orders VALUES (104, 'Divya', 3500);
INSERT INTO orders VALUES (105, 'Priya', 4000);
```

Records Inserted

SQL Query

Query 1: Display all order records

```
SQL> SELECT * FROM orders;
```

Resultant Query

ORDER_ID	CUSTOMER_NAME	AMOUNT
101	Kavin	2500
102	Meena	3000
103	Arun	2000
104	Divya	3500
105	Priya	4000

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed

The inserted order records are now permanently saved.

Query 3: Update Kavin's order amount

SQL> UPDATE orders

SET amount = 3000

WHERE order_id = 101;

1 row updated.

Updated Record

ORDER_ID	CUSTOMER_NAME	AMOUNT
101	Kavin	3000

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5: Update Meena's order amount

SQL> UPDATE orders

SET amount = 3500

WHERE order_id = 102;

1 row updated.

Updated Record

ORDER_ID	CUSTOMER_NAME	AMOUNT
102	Kavin	3500

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The changes made after SAVEPOINT sp1 are undone.

Resultant Query

ORDER_ID	CUSTOMER_NAME	AMOUNT
101	Kavin	3000
102	Meena	3000

Query 7: Rollback an uncommitted change

SQL> UPDATE orders

SET amount = 1500

WHERE order_id = 103;

1 row updated.

SQL> ROLLBACK;

Transaction rolled back.

The amount of Arun is restored to its previous value.

Resultant Query

ORDER_ID	CUSTOMER_NAME	AMOUNT
103	Arun	2000

RESULT

Thus, the Orders table was created successfully, order records were inserted, and the transaction operations **COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT** were performed successfully.

EX. NO:05 Creation of join Operations Using Students and course Table

DATE:20.7.2026

AIM

To create Students and Courses tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

SOFTWARE USED

Oracle / MySQL / SQLite3

DESCRIPTION

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

SELECT columns

FROM table1

INNER JOIN table2

ON table1.column = table2.column;

LEFT JOIN

Left Join returns all records from the left table and matching records from the right table.

Syntax:

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

RIGHT JOIN

Right Join returns all records from the right table and matching records from the left table.

Syntax:

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

FULL JOIN

Full Join returns all records from both tables, including matching and non-matching records.

Syntax:

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

SELF JOIN

Self Join is used to join a table with itself.

Explanation

- Create Students and Courses tables.
- Insert student and course details.
- Display matching student and course records using Inner Join.
- Display all students and their matching courses using Left Join.
- Display all courses and their matching students using Right Join.

- Display matching and non-matching records using Full Join.
- Use Self Join to display employees and their managers.

Create Tables

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    department TEXT
);


CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_name TEXT,
    marks INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO students VALUES (1, 'Rahul', 'CSE');
INSERT INTO students VALUES (2, 'Priya', 'ECE');
INSERT INTO students VALUES (3, 'Vijay', 'IT');
INSERT INTO students VALUES (4, 'Anitha', 'EEE');
```

Insert Courses Records

```
INSERT INTO courses VALUES (101, 1, 'Python', 85);
INSERT INTO courses VALUES (102, 2, 'Java', 78);
INSERT INTO courses VALUES (103, 1, 'SQL', 90);
```

INSERT INTO courses VALUES (104, 3, 'Web Design', 88);

Records Inserted

SQL Query

Query 1: Inner Join

SELECT students.student_name, courses.course_name, courses.marks

FROM students

INNER JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

STUDENT_NAME	COURSE_NAME	MARKS
Rahul	Python	85
Priya	Java	78
Vijay	SOL	90
Anitha	Web design	88

Query 2: Left Join

SELECT students.student_name, courses.course_name, courses.marks

FROM students

LEFT JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

STUDENT_NAME	CUSTOMER_NAME	MARKS
Rahul	Python	85
Priya	Java	78
Vijay	SQL	90
Anitha	Web design	88

Query 3: Right Join

SELECT students.student_name, courses.course_name, courses.marks

FROM students

RIGHT JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

STUDENT_NAME	CUSTOMER_NAME	MARKS
Rahul	Python	85
Priya	Java	78
Vijay	SQL	90
Anitha	Web design	88

Query 4: Full Join

SELECT students.student_name, courses.course_name, courses.marks

FROM students

FULL JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

STUDENT_NAME	CUSTOMER_NAME	MARKS
Rahul	Python	85
Priya	Java	78
Vijay	SQL	90
Anitha	Web design	88

Query 5: Self Join

Create Teacher Table

```
CREATE TABLE teachers (
    teacher_id INTEGER PRIMARY KEY,
    teacher_name TEXT,
```

```
    senior_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO teachers VALUES (1, 'Ramesh', NULL);
INSERT INTO teachers VALUES (2, 'Suresh', 1);
INSERT INTO teachers VALUES (3, 'Divya', 1);
INSERT INTO teachers VALUES (4, 'Meena', 2);
INSERT INTO teachers VALUES (5, 'Arun', 2);
```

Records Inserted

SQL Query

```
SELECT A.teacher_name AS Teacher,
       B.teacher_name AS Senior
FROM teachers A
LEFT JOIN teachers B
ON A.senior_id = B.teacher_id;
```

Resultant Query

TEACHER	SENIOR
Ramesh	Null
Suresh	Ramesh
Divya	Ramesh
Meena	Suresh
Arun	Suresh

RESULT

Thus, the Students and Courses tables were created successfully, and the required records were retrieved successfully using Inner Join, Left Join, Right Join, Full Join, and Self Join.

EX. NO:06 Creation of a SQL functions using Items Table

DATE: 31.7.2026

Aim

To create an Items table, insert item records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

SOFTWARE USED

Oracle / MySQL / SQLite3

DESCRIPTION

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a group of rows and return a single result.

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS

String functions are used to perform operations on text values.

- UPPER()

- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name**;**

DATE FUNCTIONS

Date functions are used to work with date values.

CURRENT_DATE

Date comparison

Syntax

SELECT CURRENT_DATE;

CREATE TABLE

CREATE TABLE items (

item_id INTEGER PRIMARY KEY,

item_name TEXT,

category TEXT,

price INTEGER,

purchase_date DATE

);

Table Created

SAMPLE DATA

INSERT INTO items VALUES (1, 'Pen', 'Stationery', 20, '2026-01-10');

INSERT INTO items VALUES (2, 'Notebook', 'Stationery', 80, '2026-02-15');

INSERT INTO items VALUES (3, 'Bag', 'Accessories', 1200, '2026-03-20');

INSERT INTO items VALUES (4, 'Bottle', 'Utility', 500, '2026-04-12');

INSERT INTO items VALUES (5, 'Clock', 'Home', 900, '2026-05-18');

Records Inserted

SQL QUERY

Query 1: Display all item records

SELECT * FROM items;

Resultant Query

ITEM_ID	ITEM_NAME	CATEGORY	PRICE	PUECHASE_DATE
1	Pen	Stationery	20	2026-01-10
2	Notebook	Stationery	80	2026-02-15
3	Bag	Accessories	1200	2026-03-15
4	Bottle	Utility	500	2026-04-12
5	Clock	Home	900	2026-05-18

AGGREGATE FUNCTIONS

Query 2: Count total items

SELECT COUNT(*)

AS Total_Items

FROM items;

Result

Total_Items
5

Query 3: Calculate total price

SELECT SUM(price) AS Total_Price

FROM items;

Result

Total_Price
2700

Query 4: Calculate average price

SELECT AVG(price) AS Average_Price

FROM items;

Result

Average_Price
540

Query 5: Find highest and lowest price

SELECT MAX(price) AS Highest_Price,

MIN(price) AS Lowest_Price

FROM items;

Result

HIGHEST_PRICE	LOWEST_PRICE
1200	20

STRING FUNCTIONS

Query 6: Display item names in uppercase

SELECT item_name, UPPER(item_name) AS Upper_Name

FROM items;

Result

ITEM_NAME	UPPER_NAME
Pen	PEN
Notebook	NOTEBOOK
Bag	BAG
Bottle	BOTTLE
Clock	CLOCK

Query 7: Display item names in lowercase

SELECT item_name, LOWER(item_name) AS Lower_Name

FROM items;

Result

ITEM_NAME	LOWER_NAME
Pen	pen
Notebook	notebook
Bag	bag
Bottle	bottle
Clock	clock

Query 8: Display length of item names

SELECT item_name, LENGTH(item_name) AS Name_Length

FROM items;

RESULT

ITEM_NAME	NAME_LENGTH
Pen	3
Notebook	8
Bag	3
Bottle	6
Clock	5

DATE FUNCTIONS

Query 9: Display current date

SELECT CURRENT_DATE AS Current_Date;

Result

Current_Date
Current system date

Query 10: Display items purchased after March 2026

SELECT item_name, purchase_date

FROM items

WHERE purchase_date > '2026-03-31';

Result

ITEM_NAME	PURCHASE_DATE
Bottle	2026-04-12
Clock	2026-05-18

RESULT

Thus, the Items table was created successfully, item records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

Ex no: 07 Subqueries and Nested Queries using Patients Table

DATE: 2.8.2026

Aim

To create a Patients table, insert patient records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / SQLite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name > (SELECT column_name

        FROM table_name

        WHERE condition);
```

MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators**.**

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name IN (SELECT column_name

                      FROM table_name

                      WHERE condition);
```

NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name = (SELECT column_name

                     FROM table_name

                     WHERE condition);
```

Explanation

Create a table named patients.

Insert patient details into the table

Display all patient records.

Find patients whose age is greater than the average age.

Find patients who belong to Cardiology or General department

Find the patient with the highest age.

Retrieve patients based on the result of another query.

Create Table

```
CREATE TABLE patients (
   patient_id INTEGER PRIMARY KEY,
   patient_name TEXT,
   department TEXT,
   age INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO patients VALUES (1, 'Ravi', 'Cardiology', 45);
INSERT INTO patients VALUES (2, 'Sneha', 'General', 30);
INSERT INTO patients VALUES (3, 'Vijay', 'Neurology', 55);
INSERT INTO patients VALUES (4, 'Latha', 'Cardiology', 40);
```

Records Inserted

SQL Query

Query 1: Display all patient records

```
SELECT * FROM patients;
```

Resultant Query

PATIENT_ID	PATIENT_NAME	DEPARTMENT	AGE
1	Ravi	Cardiology	45
2	Sneha	General	30
3	Vijay	Neurology	55
4	Latha	Cardiology	40

Query 2: Display patients whose age is greater than the average age

```
SELECT patient_name, age
FROM patients
WHERE age > (SELECT AVG(age)
        FROM patients);
```

Resultant Query

PATIENT_NAME	AGE
Ravi	45
Vijay	55

Query 3: Display patients who belong to Cardiology or General department

```
SELECT patient_name, department
FROM patients
WHERE department IN (SELECT department
            FROM patients
            WHERE department IN ('Cardiology', 'General'));
```

Resultant Query

PATIENT_NAME	DEPARTMENT
Ravi	Cardiology
Sneha	General
Latha	Cardiology

Query 4: Display the patient with the highest age

```
SELECT patient_name, age
```

```
FROM patients
WHERE age = (SELECT MAX(age)
        FROM patients);
```

Resultant Query

PATIENT_NAME	AGE
Vijay	55

Query 5: Display patients whose age is greater than Sneha's age

```
SELECT patient_name, age
FROM patients
WHERE age > (SELECT age
        FROM patients
        WHERE patient_name = 'Sneha');
```

Resultant Query

PATIENT_NAME	AGE
Ravi	45
Vijay	55
Latha	40

RESULT

Thus, the Patients table was created successfully, patient records were inserted, and the required patient details were retrieved successfully using Subqueries and Nested Queries.

EX. NO: 08 Creation of Views, Indexes and Sequences

DATE: 10.8.2026

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

1. VIEW

Create Table

```
CREATE TABLE student_details (
   student_id NUMBER PRIMARY KEY,
   student_name VARCHAR2(50),
   course VARCHAR2(30),
   marks NUMBER
);
```

Insert Records

```
INSERT INTO student_details VALUES (201, 'Ajay', 'BCA', 78);
INSERT INTO student_details VALUES (202, 'Nisha', 'BBA', 85);
INSERT INTO student_details VALUES (203, 'Ravi', 'BCA', 72);
INSERT INTO student_details VALUES (204, 'Keerthi', 'BSc', 90);
INSERT INTO student_details VALUES (205, 'Vijay', 'BCA', 88);
COMMIT;
```

Create View

CREATE VIEW bca_students AS

SELECT student_id, student_name, marks

FROM student_details

WHERE course = 'BCA';

Display View

SELECT * FROM bca_students;

Result

Student_id	Student_name	Marks
201	Ajay	78
203	Ravi	72
205	Vijay	88

Explanation:

The bca_students view displays only students belonging to the BCA course.

2. INDEX

Create Index

CREATE INDEX idx_student_name

ON student_details(student_name);

Search Using Index

SELECT *

FROM student_details

WHERE student_name = 'Nisha';

Result

Student_id	Student_name	course	marks
202	Nisha	BBA	85

3. SEQUENCE

Create Sequence

CREATE SEQUENCE student_seq

START WITH 206

INCREMENT BY 1;

Insert Record Using Sequence

INSERT INTO student_details

VALUES (student_seq.NEXTVAL, 'Sanjay', 'BCom', 76);

COMMIT;

Display Sequence Value

SELECT student_seq.CURRVAL AS current_sequence

FROM dual;

Result

current_sequence
206

Insert Another Record

INSERT INTO student_details

VALUES (student_seq.NEXTVAL, 'Harini', 'BSc', 82);

COMMIT;

Display All Records

SELECT * FROM student_details

ORDER BY student_id;

Result

Student_id	Student_name	course	Marks
201	Ajay	BCA	78
202	Nisha	BBA	85
203	Ravi	BCA	72
204	Keerthi	BSC	90
205	Vijay	BCA	88
206	Sanjay	BCOM	76
207	Harini	BSC	82

Important Commands

DROP VIEW bca_students;

DROP INDEX idx_student_name;

DROP SEQUENCE student_seq;

Result

Thus, the View, Index, and Sequence were successfully created and operated using SQL commands.

EX. NO:09 CREATION OF STORED PROCEDURES AND FUNCTIONS

DATE:18.8.2026

AIM

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

1. STORED PROCEDURE

Create Table

```
CREATE TABLE hotel_rooms (
    room_id NUMBER PRIMARY KEY,
    guest_name VARCHAR2(50),
    room_type VARCHAR2(30),
    rent NUMBER
);
```

Insert Records

```
INSERT INTO hotel_rooms VALUES (401, 'Rohan', 'Single', 1500);
INSERT INTO hotel_rooms VALUES (402, 'Tanya', 'Double', 2500);
INSERT INTO hotel_rooms VALUES (403, 'Vimal', 'Suite', 4500);
INSERT INTO hotel_rooms VALUES (404, 'Neha', 'Single', 1500);
INSERT INTO hotel_rooms VALUES (405, 'Suresh', 'Double', 2500);
COMMIT;
```

Display Records

```
SELECT * FROM hotel_rooms;
```

Resultant Query

Room_id	Guest_name	Room_type	Rent
401	Rohan	Single	1500
402	Tanya	Double	2500
403	Vimal	Suite	4500
404	Neha	Single	1500
405	Suresh	Double	2500

Create Stored Procedure

```
CREATE OR REPLACE PROCEDURE display_room (
    p_room_id IN NUMBER
)
IS
    v_guest hotel_rooms.guest_name%TYPE;
    v_rent hotel_rooms.rent%TYPE;
BEGIN
    SELECT guest_name, rent
    INTO v_guest, v_rent
    FROM hotel_rooms
    WHERE room_id = p_room_id;

    DBMS_OUTPUT.PUT_LINE('Guest Name: ' || v_guest);
    DBMS_OUTPUT.PUT_LINE('Room Rent: ' || v_rent);
END;
/
```

Execute Procedure

```
SET SERVEROUTPUT ON;

EXEC display_room(403);
```

Output

Guest Name: Vimal

Room Rent: 4500

2. FUNCTION

Create Function

```
CREATE OR REPLACE FUNCTION get_room_charge (
   p_rent IN NUMBER
)
RETURN NUMBER
IS
BEGIN
   RETURN p_rent + 500;
END;
/
```

Execute Function

```
SELECT room_id, guest_name, rent,
      get_room_charge(rent) AS total_charge
FROM hotel_rooms;
```

Resultant Query

Room_id	Guest_name	Rent	Total_charge
401	Rohan	1500	2000
402	Tanya	2500	3000
403	Vimal	4500	5000
404	Neha	1500	2000
405	Suresh	2500	3000

Important Commands

```
SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_ROOM';

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_ROOM_CHARGE';
```

Drop Procedure

```
DROP PROCEDURE display_room;
```

Drop Function

```
DROP FUNCTION get_room_charge;
```

Result

Thus, the Stored Procedure and Function were successfully created and executed using PL/SQL. The Stored Procedure was used to display guest and room details, and the Function was used to calculate the total room charge.

EX. NO:10 Creation of Triggers method using Bank account table

DATE:7.9.2026

AIM

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT or UPDATE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

1. CREATE TABLE

```
CREATE TABLE bank_account (
   account_id NUMBER PRIMARY KEY,
   customer_name VARCHAR2(50),
   balance NUMBER,
   account_type VARCHAR2(20)
);
```

Table Created

Insert Records

```
INSERT INTO bank_account VALUES (501, 'Aravind', 25000, 'Savings');
INSERT INTO bank_account VALUES (502, 'Swetha', 18000, 'Savings');
INSERT INTO bank_account VALUES (503, 'Karthi', 32000, 'Current');
INSERT INTO bank_account VALUES (504, 'Pavithra', 15000, 'Savings');
INSERT INTO bank_account VALUES (505, 'Naveen', 40000, 'Current');

COMMIT;
```

Records Inserted

Display Bank Records

SELECT * FROM bank_account;

Resultant Query

Account_id	Customer_name	Balance	Account_type
501	Aravind	25000	Savings
502	Swetha	18000	Savings
503	Karthi	32000	Current
504	Pavithra	15000	Savings
505	Naveen	40000	Current

2. BEFORE INSERT TRIGGER

```
CREATE OR REPLACE TRIGGER check_account_balance
BEFORE INSERT ON bank_account
FOR EACH ROW
BEGIN
   IF :NEW.balance < 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO bank_account
VALUES (506, 'Megha', -5000, 'Savings');
```

Output

ORA-20001: Balance cannot be negative

3. BEFORE UPDATE TRIGGER

```
CREATE OR REPLACE TRIGGER check_updated_balance
```

```
BEFORE UPDATE OF balance ON bank_account
FOR EACH ROW
BEGIN
   IF :NEW.balance < 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Updated balance cannot be negative');
   END IF;
END;
/
```

Execute Trigger

```
UPDATE bank_account
SET balance = -1000
WHERE account_id = 503;
```

Output

ORA-20002: Updated balance cannot be negative

4. AFTER INSERT TRIGGER

Create Audit Table

```
CREATE TABLE bank_audit (
   audit_id NUMBER,
   account_id NUMBER,
   action_type VARCHAR2(20)
);
```

Create Sequence

```
CREATE SEQUENCE bank_audit_seq
START WITH 1
INCREMENT BY 1;
```

Create Trigger

```
CREATE OR REPLACE TRIGGER bank_insert_audit
AFTER INSERT ON bank_account
FOR EACH ROW
BEGIN
   INSERT INTO bank_audit
   VALUES (bank_audit_seq.NEXTVAL, :NEW.account_id, 'INSERT');
END;
/
```

Insert New Account

```
INSERT INTO bank_account
VALUES (506, 'Megha', 22000, 'Savings');

COMMIT;
```

Display Audit Records

```
SELECT * FROM bank_audit;
```

Resultant Query

Audit_id	Account_id	Action_type
1	506	INSERT

5. DISPLAY FINAL RECORDS

```
SELECT * FROM bank_account
ORDER BY account_id;
```

Account_id	Customer_name	Balance	Account_type
501	Aravind	25000	Savings
502	Swetha	18000	Savings
503	Karthi	32000	Current
504	Pavithra	15000	Savings
505	Naveen	40000	Current
506	Megha	22000	Savings

Important Commands

SELECT trigger_name, status

FROM user_triggers;

DROP TRIGGER check_account_balance;

DROP TRIGGER check_updated_balance;

DROP TRIGGER bank_insert_audit;

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers validated account balance values and recorded newly inserted bank account details in the audit table.

EX. NO:11 Creation of cursor implementation using Vehicle service table

DATE: 19.9.2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time. In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```
CREATE TABLE vehicle_service (
   service_id NUMBER PRIMARY KEY,
   owner_name VARCHAR2(50),
   vehicle_type VARCHAR2(30),
   service_cost NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO vehicle_service VALUES (701, 'Yuvan', 'Car', 4500);
INSERT INTO vehicle_service VALUES (702, 'Shalini', 'Bike', 1800);
INSERT INTO vehicle_service VALUES (703, 'Pranav', 'Car', 5200);
INSERT INTO vehicle_service VALUES (704, 'Dhanya', 'Scooter', 1500);
INSERT INTO vehicle_service VALUES (705, 'Roshan', 'Car', 6000);
```

COMMIT;

Records Inserted

Display Records

SELECT * FROM vehicle_service;

Resultant Query

Service_id	Owner_name	Vehicle_type	Service_cost
701	Yuvan	Car	4500
702	Shalini	Bike	1800
703	Pranav	Car	5200
704	Dhanya	Scooter	1500
705	Roshan	Car	6000

2. EXPLICIT CURSOR

```
SET SERVEROUTPUT ON;


DECLARE
   CURSOR vehicle_cursor IS
      SELECT owner_name, vehicle_type, service_cost
      FROM vehicle_service;


   v_owner vehicle_service.owner_name%TYPE;
   v_vehicle vehicle_service.vehicle_type%TYPE;
   v_cost vehicle_service.service_cost%TYPE;
BEGIN
   OPEN vehicle_cursor;


   LOOP
```

```
        FETCH vehicle_cursor INTO v_owner, v_vehicle, v_cost;

        EXIT WHEN vehicle_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Owner: ' || v_owner ||
            ' | Vehicle: ' || v_vehicle ||
            ' | Cost: ₹' || v_cost
        );
    END LOOP;

    CLOSE vehicle_cursor;
END;
/
```

Output

Owner: Yuvan | Vehicle: Car | Cost: ₹4500

Owner: Shalini | Vehicle: Bike | Cost: ₹1800

Owner: Pranav | Vehicle: Car | Cost: ₹5200

Owner: Dhanya | Vehicle: Scooter | Cost: ₹1500

Owner: Roshan | Vehicle: Car | Cost: ₹6000

3. CURSOR WITH CONDITION

The following cursor displays vehicles whose service cost is greater than ₹3000.

```
SET SERVEROUTPUT ON;

DECLARE
```

```
    CURSOR vehicle_cursor IS
        SELECT service_id, owner_name, vehicle_type, service_cost
        FROM vehicle_service
        WHERE service_cost > 3000;

    v_id vehicle_service.service_id%TYPE;
    v_owner vehicle_service.owner_name%TYPE;
    v_vehicle vehicle_service.vehicle_type%TYPE;
    v_cost vehicle_service.service_cost%TYPE;
BEGIN
    OPEN vehicle_cursor;

    LOOP
        FETCH vehicle_cursor INTO v_id, v_owner, v_vehicle, v_cost;

        EXIT WHEN vehicle_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || v_id ||
            ' | Owner: ' || v_owner ||
            ' | Vehicle: ' || v_vehicle ||
            ' | Cost: ₹' || v_cost
        );
    END LOOP;
```

```
    CLOSE vehicle_cursor;
END;
/
```

Resultant Query

Service_id	Owner_name	Vehicle_type	Service_cost
701	Yuvan	Car	4500
703	Pranav	Car	5200
705	Roshan	Car	6000

4. CURSOR USING %ROWCOUNT

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR vehicle_cursor IS
        SELECT owner_name, vehicle_type, service_cost
        FROM vehicle_service;

    v_owner vehicle_service.owner_name%TYPE;
    v_vehicle vehicle_service.vehicle_type%TYPE;
    v_cost vehicle_service.service_cost%TYPE;
BEGIN
    OPEN vehicle_cursor;

    LOOP
        FETCH vehicle_cursor INTO v_owner, v_vehicle, v_cost;

        EXIT WHEN vehicle_cursor%NOTFOUND;
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Owner: ' || v_owner ||
            ' | Vehicle: ' || v_vehicle ||
            ' | Cost: ₹' || v_cost
        );
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(
        'Total Vehicles Processed: ' || vehicle_cursor%ROWCOUNT
    );

    CLOSE vehicle_cursor;
END;
/
```

Output

Owner: Yuvan | Vehicle: Car | Cost: ₹4500

Owner: Shalini | Vehicle: Bike | Cost: ₹1800

Owner: Pranav | Vehicle: Car | Cost: ₹5200

Owner: Dhanya | Vehicle: Scooter | Cost: ₹1500

Owner: Roshan | Vehicle: Car | Cost: ₹6000

Total Vehicles Processed: 5

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process vehicle service records one by one, apply conditions, and count the number of processed records.

EX. NO:12 Creation of PL/SQL Exception Handling using Flowers table

DATE: 28.9.2026

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program. Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type

Description

Predefined Exception

Built-in exceptions provided by Oracle

User-Defined Exception

Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE flowers (
    flower_id NUMBER PRIMARY KEY,
    flower_name VARCHAR2(50),
    price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO flowers VALUES (601, 'Rose', 150);
INSERT INTO flowers VALUES (602, 'Lily', 200);
INSERT INTO flowers VALUES (603, 'Jasmine', 180);
INSERT INTO flowers VALUES (604, 'Lotus', 250);
INSERT INTO flowers VALUES (605, 'Tulip', 300);

COMMIT;
```

Records Inserted

Display Flower Records

```
SELECT * FROM flowers;
```

Resultant Query

Flower_id	Flower _name	Price
601	Rose	150
602	Lily	200
603	Jasmine	180
604	Lotus	250
605	Tulip	300

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name flowers.flower_name%TYPE;
   v_price flowers.price%TYPE;
BEGIN
   SELECT flower_name, price
   INTO v_name, v_price
   FROM flowers
   WHERE flower_id = 699;


   DBMS_OUTPUT.PUT_LINE('Flower Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Price: ₹' || v_price);


EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Flower not found.');
END;
/
```

Output

Exception: Flower not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name flowers.flower_name%TYPE;
BEGIN
   SELECT flower_name
   INTO v_name
   FROM flowers
   WHERE price > 150;


   DBMS_OUTPUT.PUT_LINE('Flower Name: ' || v_name);


EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: More than one flower found.'
      );
```

```
END;
/
```

Output

Exception: More than one flower found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_price NUMBER := 150;
   v_divisor NUMBER := 0;
   v_result NUMBER;
BEGIN
   v_result := v_price / v_divisor;

   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);

EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
```

```
        'Exception: Cannot divide by zero.'
    );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, the discount amount should not be greater than the flower price.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_price flowers.price%TYPE;
   v_discount NUMBER := 500;
   invalid_discount EXCEPTION;

BEGIN
   SELECT price
   INTO v_price
   FROM flowers
```

```
    WHERE flower_id = 601;

    IF v_discount > v_price THEN
        RAISE invalid_discount;
    END IF;

    UPDATE flowers
    SET price = price - v_discount
    WHERE flower_id = 601;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Discount Applied Successfully.');

EXCEPTION
    WHEN invalid_discount THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Discount exceeds flower price.'
        );
END;
/
```

Output

Exception: Discount exceeds flower price.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program applies a valid discount when the discount amount is less than the flower price.

```
SET SERVEROUTPUT ON;


DECLARE
   v_price flowers.price%TYPE;
   v_discount NUMBER := 50;


BEGIN
   SELECT price
   INTO v_price
   FROM flowers
   WHERE flower_id = 601;


   IF v_discount > v_price THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Discount exceeds flower price'
      );
   END IF;


   UPDATE flowers
   SET price = price - v_discount
   WHERE flower_id = 601;
```

```
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Discount Applied Successfully.');
    DBMS_OUTPUT.PUT_LINE(
        'Discount Amount: ₹' || v_discount
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Discount Applied Successfully.

Discount Amount: ₹50

Display Updated Flower

```
SELECT * FROM flowers
WHERE flower_id = 601;
```

Resultant Query

Flower_id	Flower_name	Price
601	Rose	100

Important Predefined Exceptions

Exception

Description

NO_DATA_FOUND

No record is found

TOO_MANY_ROWS

More than one record is returned

ZERO_DIVIDE

Division by zero occurs

DUP_VAL_ON_INDEX

Duplicate value is inserted into a unique/primary key column

VALUE_ERROR

Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented using the Flowers table. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 Creation of Database Normalization using Employee project table

DATE: 1.10.2026

AIM

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

SOFTWARE USED

Oracle SQL / SQL Command Line

DESCRIPTION

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Employee Project table.

The table contains repeated information. For example, the manager name Ramesh is repeated for employees working on the same project.

Unnormalized Table

Employee_ID	Employee name	Project ID	Project name	Manager ID	Manager name
101	Anitha	P01	Website	M01	Ramesh
102	Bharath	P02	Mobile app	M02	Suresh
103	Divya	P01	Website	M01	Ramesh
104	Karthik	P03	Database	M03	Priya

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above table contains atomic values, so it can be considered as the 1NF table.

1NF Table

Employee ID	Employee name	Project ID	Project name	Manager ID	Manager name
101	Anitha	P01	Website	M01	Ramesh
102	Bharath	P02	Mobile app	M02	Suresh
103	Divya	P01	Website	M01	Ramesh
104	Anitha	P03	Database	M03	Priya

Primary Key: employee_id, project_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

1.It is already in 1NF.

2.There is no partial dependency.

3.Every non-key attribute must depend on the complete primary key.

In the 1NF table, some attributes depend only on part of the composite key.

For example:

- employee_name depends only on employee_id.
- project_name depends only on project_id.
- manager_id depends on project_id.

Therefore, the table is divided into separate tables.

Employee Table

Employee ID	Employee name
101	Anitha
102	Bharath
103	Divya
104	karthik

Project Table

Project ID	Project name	Manager ID
P01	Website	M01
P02	Mobile app	M02
P03	Database	M03

Assignment Table

Employee ID	Project ID
101	P01
102	P02
103	P01
104	P03

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes must depend only on the primary key.

In the Project table:

project_id → manager_id

and

manager_id → manager_name

Therefore, manager_name has a transitive dependency on project_id.

To remove this dependency, the manager information is separated into another table.

Project Table

Project ID	Project name	Manager ID
P01	Website	M01
P02	Mobile app	M02
P03	Database	M03

Manager Table

Manager ID	Manager name
M01	Ramesh
M02	Suresh
M03	Priya

Assignment Table

Employee ID	Project ID
101	P01
102	P02
103	P01
104	P03

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency, the determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider the following Manager_Project table.

Assume that each manager is responsible for only one project.

Functional dependency:

manager_id → project_id

Here, manager_id determines project_id.

If manager_id is not a candidate key in the given relationship, the relation violates BCNF.

Therefore, the relation can be decomposed into separate tables.

Manager Table

After decomposition, the determinant is treated as a key in its corresponding relation.

Therefore, the relation is decomposed to satisfy BCNF.

SQL COMMANDS

Create Employee Table

```
CREATE TABLE Employee (
   employee_id NUMBER PRIMARY KEY,
   employee_name VARCHAR2(30)
);
```

Insert Records

```
INSERT INTO Employee VALUES (101, 'Anitha');
INSERT INTO Employee VALUES (102, 'Bharath');
INSERT INTO Employee VALUES (103, 'Divya');
INSERT INTO Employee VALUES (104, 'Karthik');
```

Create Manager Table

```
CREATE TABLE Manager (
   manager_id VARCHAR2(5) PRIMARY KEY,
   manager_name VARCHAR2(30)
);
```

Insert Records

```
INSERT INTO Manager VALUES ('M01', 'Ramesh');
INSERT INTO Manager VALUES ('M02', 'Suresh');
INSERT INTO Manager VALUES ('M03', 'Priya');
```

Create Project Table

```
CREATE TABLE Project (
    project_id VARCHAR2(5) PRIMARY KEY,
    project_name VARCHAR2(30),
    manager_id VARCHAR2(5),
    FOREIGN KEY (manager_id) REFERENCES Manager(manager_id)
);
```

Insert Records

```
INSERT INTO Project VALUES ('P01', 'Website', 'M01');
INSERT INTO Project VALUES ('P02', 'Mobile App', 'M02');
INSERT INTO Project VALUES ('P03', 'Database', 'M03');
```

Create Assignment Table

```
CREATE TABLE Assignment (
    employee_id NUMBER,
    project_id VARCHAR2(5),
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

Insert Records

```
INSERT INTO Assignment VALUES (101, 'P01');
INSERT INTO Assignment VALUES (102, 'P02');
INSERT INTO Assignment VALUES (103, 'P01');
INSERT INTO Assignment VALUES (104, 'P03');
```

Display Records

SELECT * FROM Employee;

SELECT * FROM Manager;

SELECT * FROM Project;

SELECT * FROM Assignment;

RESULT

Thus, the Database Normalization process was studied and implemented up to 1NF, 2NF, 3NF, and BCNF using Employee and Project tables. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

EX.NO:14 MINI PROJECT

DATE:3.10.2026

CREDIT CARD TRANSACTION AND FRAUD FLAGGING

AIM

To develop a Credit Card Transaction and Fraud Flagging system using SQL and PL/SQL to analyze transaction details such as customer, amount, merchant, location, and fraud status, and to identify potentially fraudulent transactions based on predefined rules.

SOFTWARE USED

Oracle Database

Oracle SQL Developer

SQL

PL/SQL

GitHub – for storing and managing the project source code

DESCRIPTION

The Credit Card Transaction and Fraud Flagging system is designed to analyze credit card transactions and identify suspicious activities. The system stores details such as transaction ID, customer ID, transaction date, transaction amount, merchant, location, and fraud flag.

SQL queries are used to display transaction details, calculate total and average spending, analyze fraud and non-fraud transactions, identify high-value transactions, and find transactions occurring outside a selected location.

PL/SQL functions and procedures are used to apply fraud-detection rules, retrieve a customer's transaction history, and generate fraud summaries based on merchant and transaction details.

Note: The fraud flag used in this project is based on predefined educational rules and is not a real-world fraud detection model.

TABLE CREATION

```
CREATE TABLE CARD_TRANSACTIONS (
   TRANSACTION_ID NUMBER PRIMARY KEY,
   CUSTOMER_ID NUMBER,
   TRANSACTION_DATE DATE,
   AMOUNT NUMBER(12,2),
   MERCHANT VARCHAR2(50),
   LOCATION VARCHAR2(50),
   FRAUD_FLAG VARCHAR2(10)
);
```

DATASET INSERTION

```
INSERT INTO CARD_TRANSACTIONS VALUES
(1001, 101, TO_DATE('01-10-2026','DD-MM-YYYY'), 25000,  'Amazon',      'Chennai',
'NO');

INSERT INTO CARD_TRANSACTIONS VALUES
(1002, 102, TO_DATE('01-10-2026','DD-MM-YYYY'), 75000,  'Flipkart',    'Bangalore',
'YES');

INSERT INTO CARD_TRANSACTIONS VALUES
(1003, 101, TO_DATE('02-10-2026','DD-MM-YYYY'), 15000,  'Amazon',      'Chennai',
'NO');

INSERT INTO CARD_TRANSACTIONS VALUES
(1004, 103, TO_DATE('02-10-2026','DD-MM-YYYY'), 550000, 'Apple Store', 'Mumbai',
'YES');
```

```
INSERT INTO CARD_TRANSACTIONS VALUES

(1005, 104, TO_DATE('03-10-2026','DD-MM-YYYY'), 30000,  'Myntra',      'Chennai',    'NO');


INSERT INTO CARD_TRANSACTIONS VALUES

(1006, 102, TO_DATE('03-10-2026','DD-MM-YYYY'), 650000, 'Amazon',      'Delhi',
'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1007, 105, TO_DATE('04-10-2026','DD-MM-YYYY'), 45000,  'Flipkart',    'Chennai',    'NO');


INSERT INTO CARD_TRANSACTIONS VALUES

(1008, 103, TO_DATE('04-10-2026','DD-MM-YYYY'), 80000,  'Amazon',      'Bangalore',
'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1009, 104, TO_DATE('05-10-2026','DD-MM-YYYY'), 20000,  'Myntra',      'Chennai',    'NO');


INSERT INTO CARD_TRANSACTIONS VALUES

(1010, 105, TO_DATE('05-10-2026','DD-MM-YYYY'), 600000, 'Apple Store', 'Hyderabad',
'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1011, 101, TO_DATE('05-10-2026','DD-MM-YYYY'), 35000,  'Flipkart',    'Chennai',    'NO');


INSERT INTO CARD_TRANSACTIONS VALUES
```

(1012, 102, TO_DATE('05-10-2026','DD-MM-YYYY'), 90000, 'Amazon', 'Chennai', 'YES');

INSERT INTO CARD_TRANSACTIONS VALUES

(1013, 106, TO_DATE('06-10-2026','DD-MM-YYYY'), 18000, 'Myntra', 'Chennai', 'NO');

INSERT INTO CARD_TRANSACTIONS VALUES

(1014, 107, TO_DATE('06-10-2026','DD-MM-YYYY'), 72000, 'Flipkart', 'Mumbai', 'YES');

INSERT INTO CARD_TRANSACTIONS VALUES

(1015, 106, TO_DATE('07-10-2026','DD-MM-YYYY'), 42000, 'Amazon', 'Chennai', 'NO');

INSERT INTO CARD_TRANSACTIONS VALUES

(1016, 108, TO_DATE('07-10-2026','DD-MM-YYYY'), 580000, 'Apple Store', 'Delhi', 'YES');

INSERT INTO CARD_TRANSACTIONS VALUES

(1017, 109, TO_DATE('08-10-2026','DD-MM-YYYY'), 28000, 'Myntra', 'Chennai', 'NO');

INSERT INTO CARD_TRANSACTIONS VALUES

(1018, 107, TO_DATE('08-10-2026','DD-MM-YYYY'), 95000, 'Amazon', 'Bangalore', 'YES');

INSERT INTO CARD_TRANSACTIONS VALUES

(1019, 110, TO_DATE('09-10-2026','DD-MM-YYYY'), 32000, 'Flipkart', 'Chennai', 'NO');

```
INSERT INTO CARD_TRANSACTIONS VALUES

(1020, 108, TO_DATE('09-10-2026','DD-MM-YYYY'), 620000, 'Apple Store', 'Hyderabad',
'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1021, 109, TO_DATE('10-10-2026','DD-MM-YYYY'), 22000,  'Amazon',      'Chennai',
'NO');


INSERT INTO CARD_TRANSACTIONS VALUES

(1022, 110, TO_DATE('10-10-2026','DD-MM-YYYY'), 68000,  'Flipkart',    'Pune',        'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1023, 111, TO_DATE('11-10-2026','DD-MM-YYYY'), 40000,  'Myntra',      'Chennai',    'NO');


INSERT INTO CARD_TRANSACTIONS VALUES

(1024, 111, TO_DATE('11-10-2026','DD-MM-YYYY'), 85000,  'Amazon',      'Mumbai',
'YES');


INSERT INTO CARD_TRANSACTIONS VALUES

(1025, 112, TO_DATE('12-10-2026','DD-MM-YYYY'), 55000,  'Flipkart',    'Chennai',    'YES');

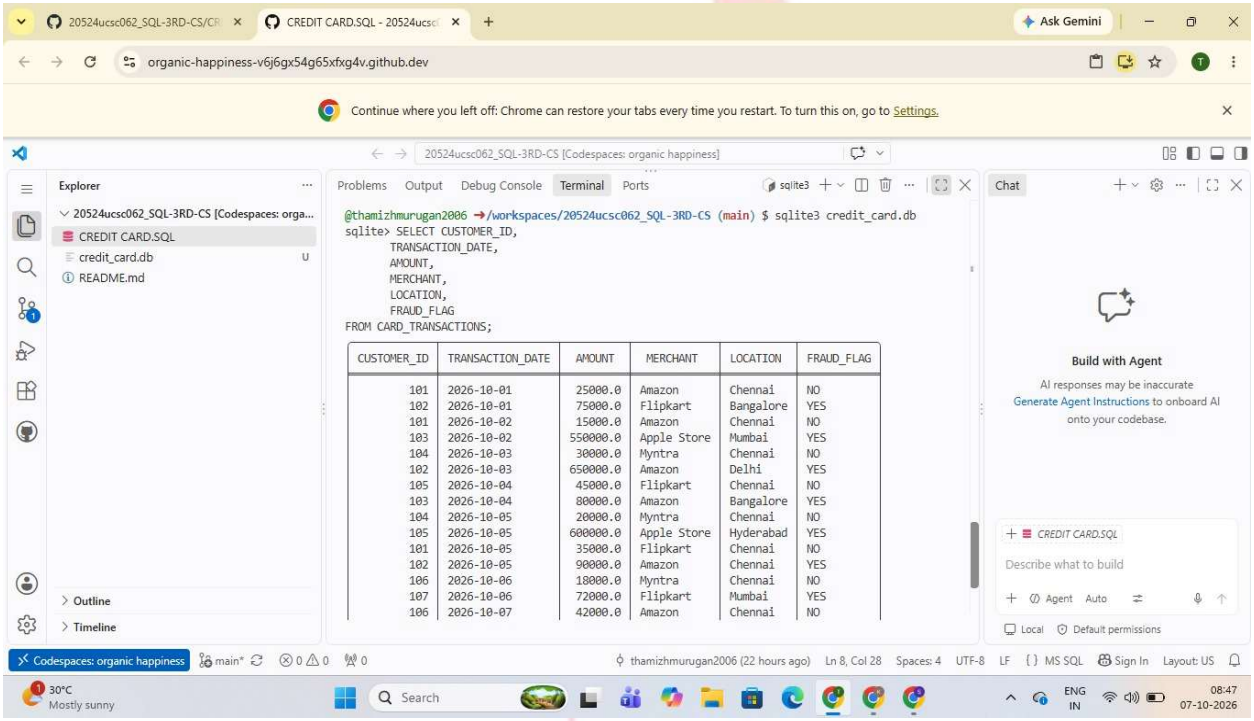

COMMIT;
```

SQL Query 1: Display all transactions

```
SELECT CUSTOMER_ID,
    TRANSACTION_DATE,
```

```
    AMOUNT,
    MERCHANT,
    LOCATION,
    FRAUD_FLAG
FROM CARD_TRANSACTIONS;
```

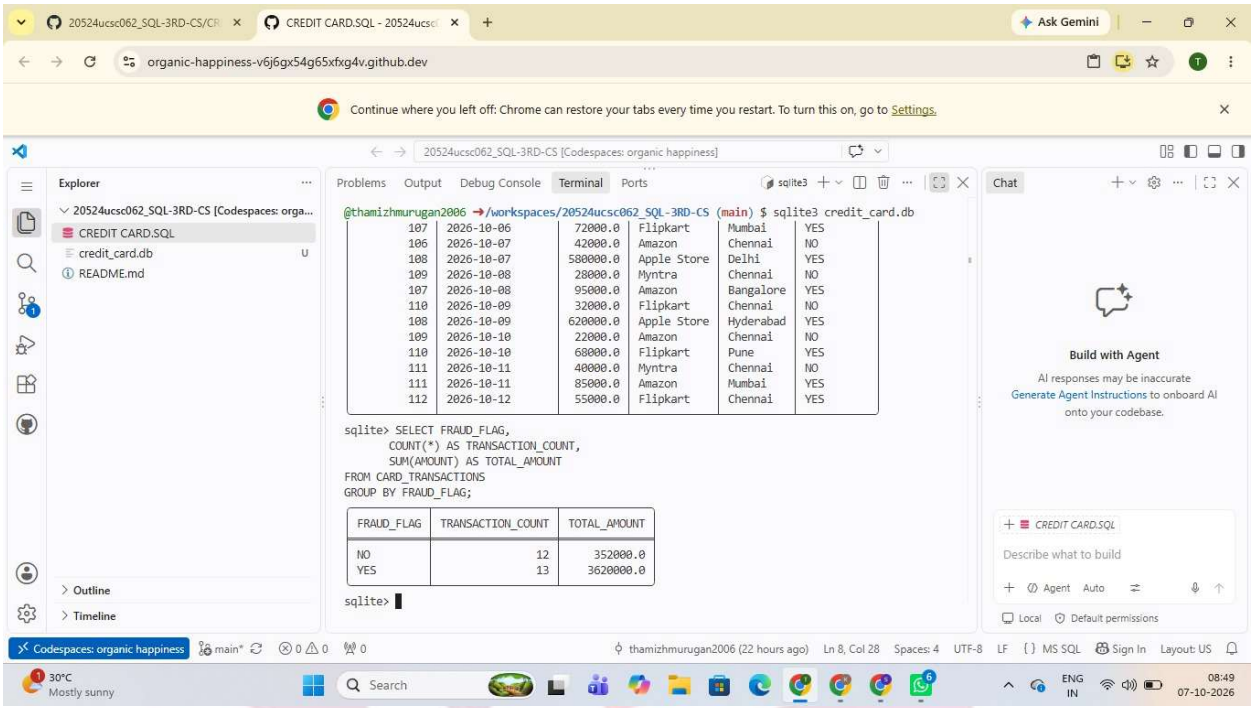
Output:



SQL QUERY 2: Total Transaction Amount – Fraud / Non-Fraud

```
SELECT FRAUD_FLAG,
    COUNT(*) AS TRANSACTION_COUNT,
    SUM(AMOUNT) AS TOTAL_AMOUNT
FROM CARD_TRANSACTIONS
GROUP BY FRAUD_FLAG;
```

OUTPUT:



SQL QUERY 3: Customers with Multiple Transactions Above 500000

```
SELECT CUSTOMER_ID,
    COUNT(*) AS HIGH_VALUE_TRANSACTIONS
FROM CARD_TRANSACTIONS
WHERE AMOUNT > 500000
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;
```

OUTPUT:

```
sqlite> SELECT CUSTOMER_ID,
       COUNT(*) AS HIGH_VALUE_TRANSACTIONS
FROM CARD_TRANSACTIONS
WHERE AMOUNT > 500000
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;
```

CUSTOMER_ID	HIGH_VALUE_TRANSA...
108	2

```
sqlite>
```

SQL QUERY 4:

```
SELECT CUSTOMER_ID,
    COUNT(*) AS TRANSACTION_COUNT
FROM CARD_TRANSACTIONS
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;
```

OUTPUT:

```
sqlite> SELECT CUSTOMER_ID,
       COUNT(*) AS TRANSACTION_COUNT
FROM CARD_TRANSACTIONS
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1;
```

CUSTOMER_ID	TRANSACTION_COUNT
101	3
102	3
103	2
104	2
105	2
106	2
107	2
108	2
109	2
110	2
111	2

SQL QUERY 5: Average and Total Spending for Each Merchant

```
SELECT MERCHANT,
```

```
    AVG(AMOUNT) AS AVERAGE_SPENDING,
    SUM(AMOUNT) AS TOTAL_SPENDING
FROM CARD_TRANSACTIONS
GROUP BY MERCHANT;
```

OUTPUT:

```
sqlite> SELECT MERCHANT,
       AVG(AMOUNT) AS AVERAGE_SPENDING,
       SUM(AMOUNT) AS TOTAL_SPENDING
FROM CARD_TRANSACTIONS
GROUP BY MERCHANT;
```

MERCHANT	AVERAGE_SPENDING	TOTAL_SPENDING
Amazon	122666.66666666667	1104000.0
Apple Store	587500.0	2350000.0
Flipkart	54571.428571428573	382000.0
Myntra	27200.0	136000.0

SQL QUERY 6: Transactions Outside Chennai

```
SELECT TRANSACTION_ID,
    CUSTOMER_ID,
    AMOUNT,
    MERCHANT,
    LOCATION,
    FRAUD_FLAG
FROM CARD_TRANSACTIONS
WHERE LOCATION <> 'Chennai';
```

OUTPUT:

```
sqlite> SELECT TRANSACTION_ID,
        CUSTOMER_ID,
        AMOUNT,
        MERCHANT,
        LOCATION,
        FRAUD_FLAG
FROM CARD_TRANSACTIONS
WHERE LOCATION <> 'Chennai';
```

TRANSACTION_ID	CUSTOMER_ID	AMOUNT	MERCHANT	LOCATION	FRAUD_FLAG
1002	102	75000.0	Flipkart	Bangalore	YES
1004	103	550000.0	Apple Store	Mumbai	YES
1006	102	650000.0	Amazon	Delhi	YES
1008	103	80000.0	Amazon	Bangalore	YES
1010	105	600000.0	Apple Store	Hyderabad	YES
1014	107	72000.0	Flipkart	Mumbai	YES
1016	108	580000.0	Apple Store	Delhi	YES
1018	107	95000.0	Amazon	Bangalore	YES
1020	108	620000.0	Apple Store	Hyderabad	YES
1022	110	68000.0	Flipkart	Pune	YES
1024	111	85000.0	Amazon	Mumbai	YES

PL/SQL PROCEDURE 1: Function – Fraud Flagging

```
CREATE OR REPLACE FUNCTION CHECK_FRAUD (
   P_AMOUNT NUMBER
)
RETURN VARCHAR2
IS
BEGIN
   IF P_AMOUNT > 50000 THEN
      RETURN 'HIGH RISK';
   ELSE
      RETURN 'NORMAL';
   END IF;
END;
/
SELECT TRANSACTION_ID,
      AMOUNT,
      CHECK_FRAUD(AMOUNT) AS RISK_STATUS
```

FROM CARD_TRANSACTIONS;

OUTPUT:

Results Explain Describe Saved SQL History

TRANSACTION_ID	AMOUNT	RISK_STATUS
1001	25000	NORMAL
1002	75000	HIGH RISK
1003	15000	NORMAL
1004	550000	HIGH RISK
1005	30000	NORMAL
1006	650000	HIGH RISK
1008	80000	HIGH RISK
1009	20000	NORMAL
1010	600000	HIGH RISK
1011	35000	NORMAL
More than 10 rows available. Increase rows selector to view more rows.		

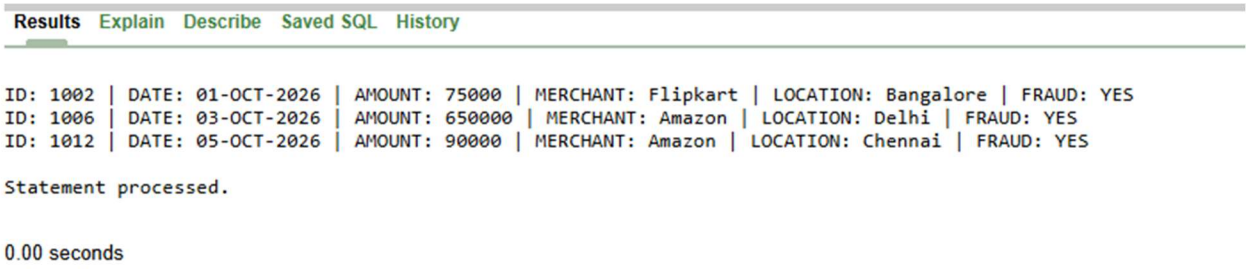
10 rows returned in 0.06 seconds CSV Export

PL/SQL PROCEDURE 2: Customer Transaction History

```
CREATE OR REPLACE PROCEDURE CUSTOMER_HISTORY (
   P_CUSTOMER_ID NUMBER
)
IS
BEGIN
   FOR R IN
   (
      SELECT TRANSACTION_ID,
           TRANSACTION_DATE,
           AMOUNT,
           MERCHANT,
           LOCATION,
```

```
            FRAUD_FLAG
        FROM CARD_TRANSACTIONS
        WHERE CUSTOMER_ID = P_CUSTOMER_ID
        ORDER BY TRANSACTION_DATE
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || R.TRANSACTION_ID ||
            ' | DATE: ' || TO_CHAR(R.TRANSACTION_DATE,'DD-MON-YYYY') ||
            ' | AMOUNT: ' || R.AMOUNT ||
            ' | MERCHANT: ' || R.MERCHANT ||
            ' | LOCATION: ' || R.LOCATION ||
            ' | FRAUD: ' || R.FRAUD_FLAG
        );
    END LOOP;
END;
/
BEGIN
    CUSTOMER_HISTORY(102);
END;
/
```

OUTPUT:

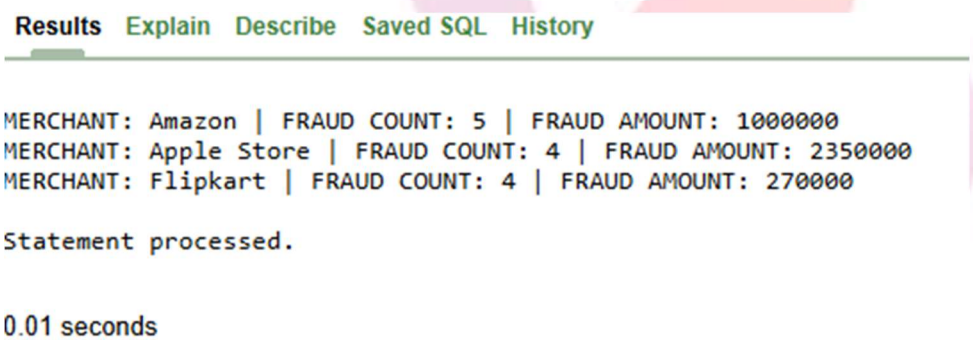


PL/SQL PROCEDURE 3: Fraud Summary by Merchant

```
CREATE OR REPLACE PROCEDURE FRAUD_SUMMARY
IS
BEGIN
  FOR R IN
  (
    SELECT MERCHANT,
        COUNT(*) AS FRAUD_COUNT,
        SUM(AMOUNT) AS FRAUD_AMOUNT
    FROM CARD_TRANSACTIONS
    WHERE FRAUD_FLAG = 'YES'
    GROUP BY MERCHANT
    ORDER BY MERCHANT
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE(
      'MERCHANT: ' || R.MERCHANT ||
      ' | FRAUD COUNT: ' || R.FRAUD_COUNT ||
      ' | FRAUD AMOUNT: ' || R.FRAUD_AMOUNT
```

```
    );
  END LOOP;
END;
/
BEGIN
  FRAUD_SUMMARY;
END;
/
```

OUTPUT:



RESULT:

Thus, the Credit Card Transaction and Fraud Flagging system was successfully implemented using SQL and PL/SQL. The required CARD_TRANSACTIONS table was created, 25 transaction records were inserted, SQL queries were executed successfully, and transaction amounts, merchants, locations, and fraud-labelled records were analyzed. PL/SQL functions and procedures were implemented to identify high-risk transactions, retrieve customer transaction history, and generate fraud summaries. The system successfully demonstrated basic credit card transaction analysis and rule-based fraud flagging.

KHAN ACADEMY

PRACTICE PROGRAM

DATABASE TABLE FOR MY FAVOURITE BOOKS

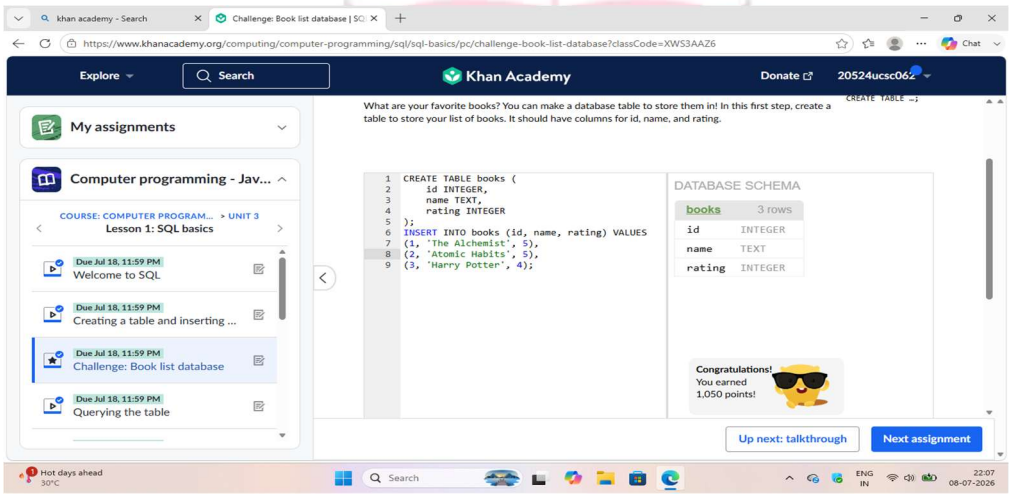
QUESTION

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.

ANSWERS

```
CREATE TABLE books (
    id INTEGER,
    name TEXT,
    rating INTEGER
);


INSERT INTO books (id, name, rating) VALUES
(1, 'The Alchemist', 5),
(2, 'Atomic Habits', 5),
(3, 'Harry Potter', 4);
```



BOX OFFICE MOVE EXPLORE

QUESTION

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

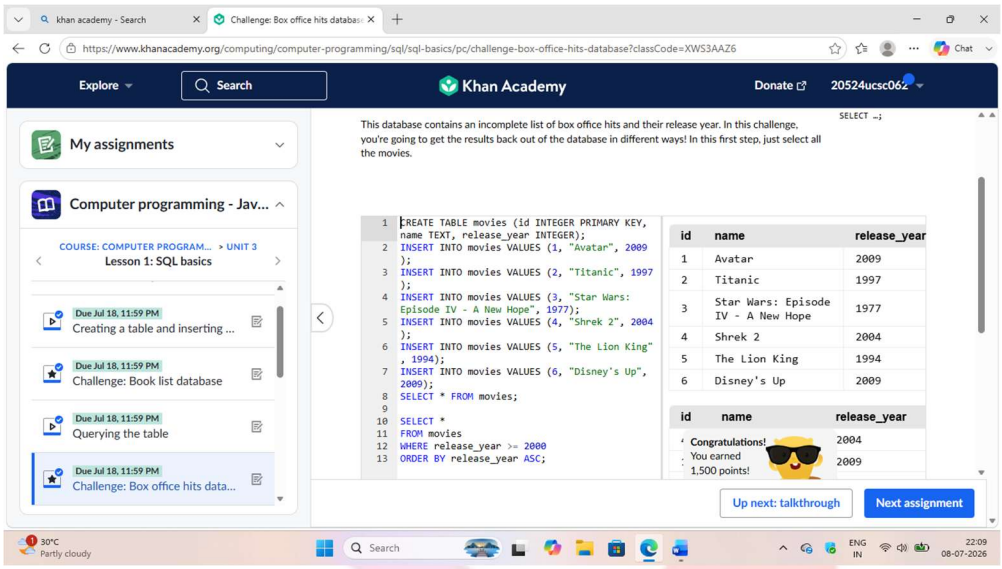
ANSWERS

```
CREATE TABLE movies (
   id INTEGER PRIMARY KEY,
   name TEXT,
   release_year INTEGER
);


INSERT INTO movies VALUES (1, "Avatar", 2009);
INSERT INTO movies VALUES (2, "Titanic", 1997);
INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);
INSERT INTO movies VALUES (4, "Shrek 2", 2004);
INSERT INTO movies VALUES (5, "The Lion King", 1994);
INSERT INTO movies VALUES (6, "Disney's Up", 2009);


SELECT * FROM movies;


SELECT *
FROM movies
WHERE release_year >= 2000
ORDER BY release_year ASC;
```



TODO LIST DATABASE STATS

QUESTION

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take.

ANSWERS

```
CREATE TABLE todo_list (
    id INTEGER PRIMARY KEY,
    item TEXT,
    minutes INTEGER
);

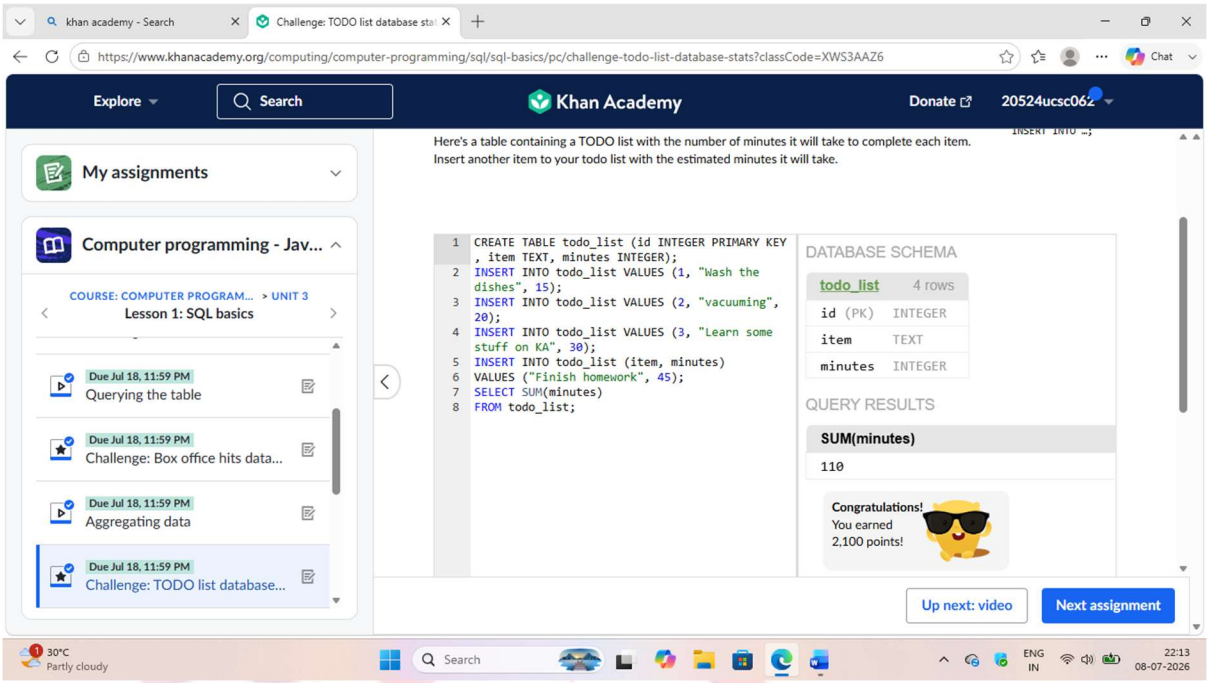

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);
INSERT INTO todo_list VALUES (2, "Vacuuming", 20);
INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);


INSERT INTO todo_list (item, minutes)
```

VALUES ("Finish homework", 45);

SELECT SUM(minutes)

FROM todo_list;



KARAOKE SONG SELECTOR

QUESTION

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

ANSWERS

CREATE TABLE songs (

id INTEGER PRIMARY KEY,

title TEXT,

artist TEXT,

mood TEXT,

duration INTEGER,

```
    released INTEGER
);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
 INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013)
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
SELECT title
FROM songs;
SELECT title
FROM songs
WHERE mood = "epic"
OR released > 1990;
```

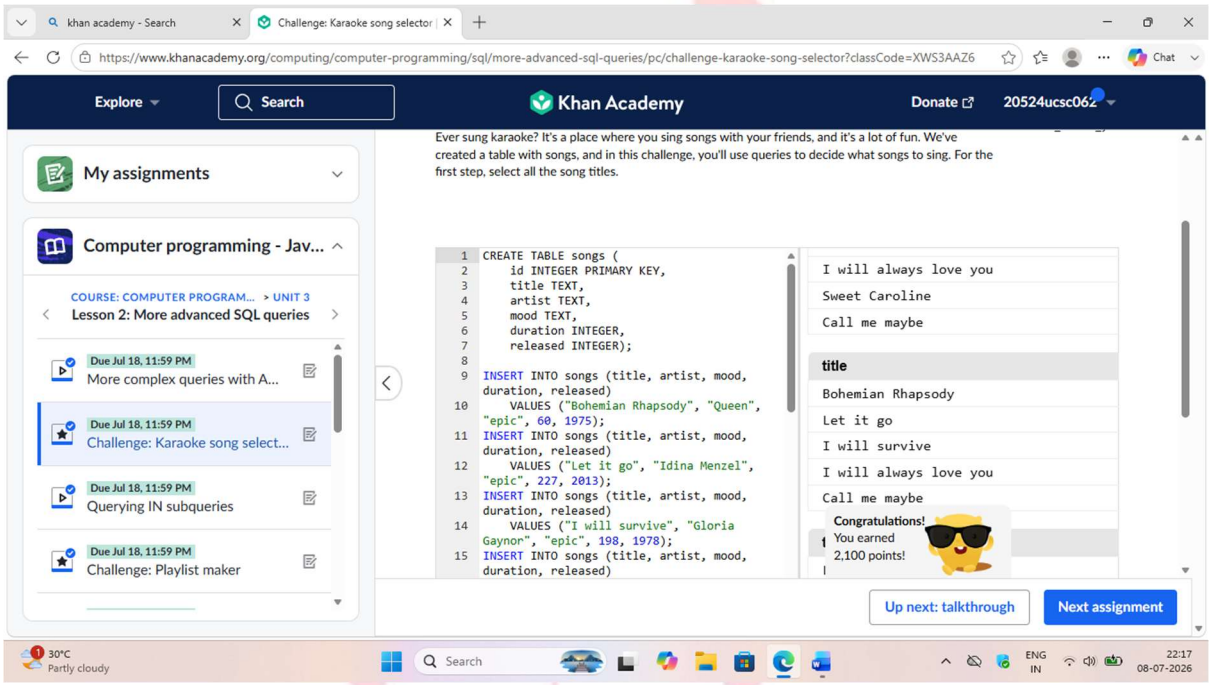
SELECT title

FROM songs

WHERE mood = "epic"

AND released > 1990

AND duration < 240;



Step 2

QUESTION

Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990.

ANSWER

CREATE TABLE songs (

id INTEGER PRIMARY KEY,

title TEXT,

artist TEXT,

```
    mood TEXT,
    duration INTEGER,
    released INTEGER
  );
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
  VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
SELECT title
  FROM songs;
SELECT title
  FROM songs
  WHERE mood = "epic"
```

OR released > 1990;

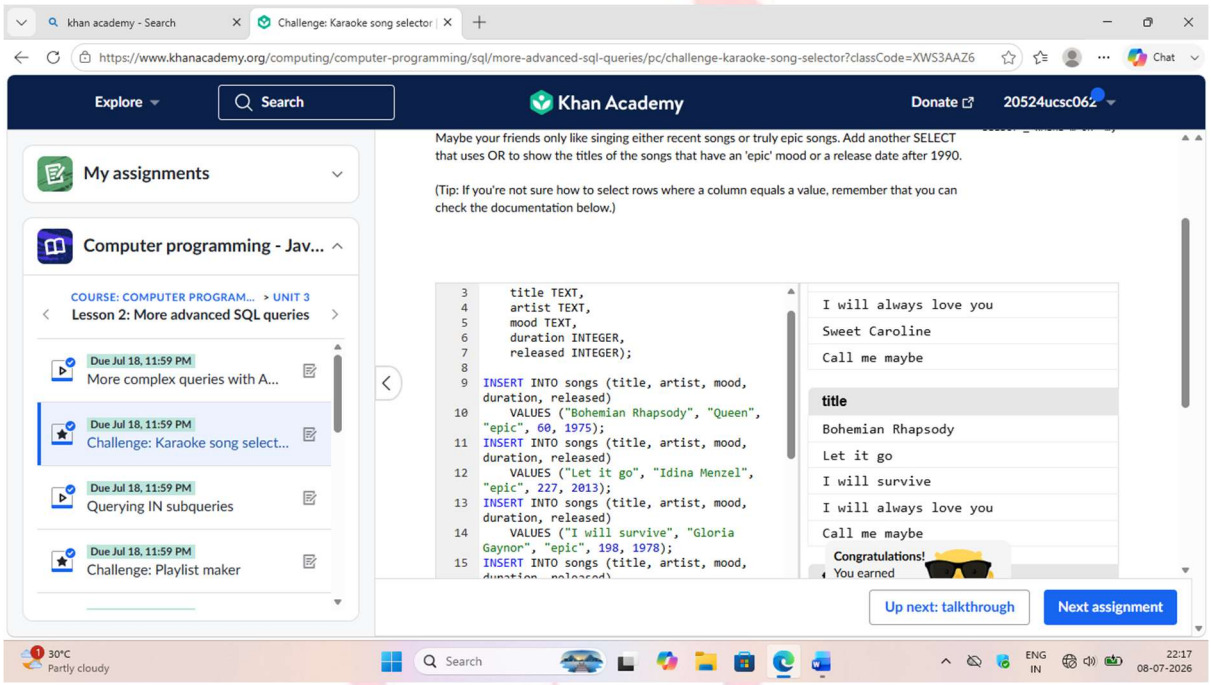
SELECT title

FROM songs

WHERE mood = "epic"

AND released > 1990

AND duration < 240;



PLAYLIST MAKER

QUESTION

Step 1

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.

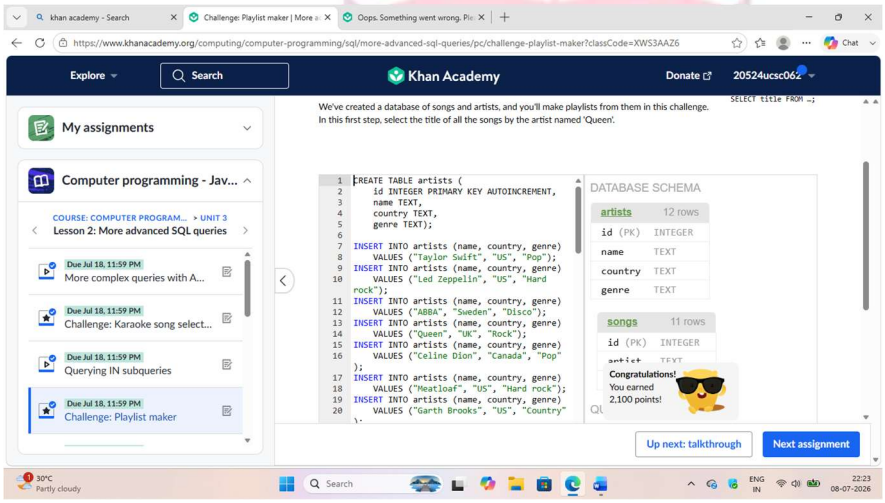
ANSWER

CREATE TABLE artists (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

```
    country TEXT,
    genre TEXT
  );
INSERT INTO artists (name, country, genre)
  VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
  VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
  VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
  VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
  VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
  VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
  VALUES ("Garth Brooks", "US", "Country");
```



Step 2

Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre.

QUESTION

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT
);


INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop");


INSERT INTO artists (name, country, genre)
VALUES ("Led Zeppelin", "US", "Hard rock");


INSERT INTO artists (name, country, genre)
VALUES ("ABBA", "Sweden", "Disco");


INSERT INTO artists (name, country, genre)
VALUES ("Queen", "UK", "Rock");


INSERT INTO artists (name, country, genre)
VALUES ("Celine Dion", "Canada", "Pop");
```

```
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");

INSERT INTO artists (name, country, genre)
VALUES ("Shania Twain", "Canada", "Country");

INSERT INTO artists (name, country, genre)
VALUES ("Rihanna", "US", "Pop");

INSERT INTO artists (name, country, genre)
VALUES ("Guns N' Roses", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");

INSERT INTO artists (name, country, genre)
VALUES ("Bob Marley", "Jamaica", "Reggae");

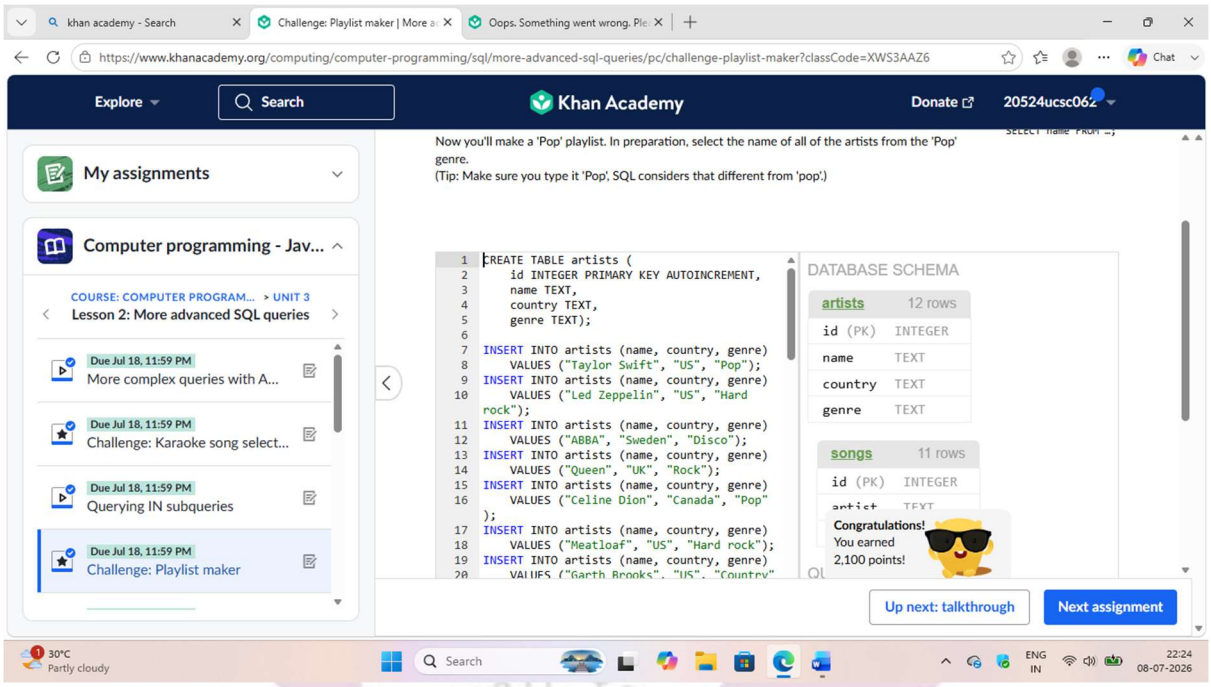

CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    artist TEXT,
    title TEXT
);


INSERT INTO songs (artist, title)
VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
VALUES ("Queen", "Bohemian Rhapsody");
```

INSERT INTO songs (artist, title)

VALUES ("Guns N' Roses", "Don't cry");



Step 3

QUESTION

To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

ANSWER

CREATE TABLE artists (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

country TEXT,

genre TEXT

);

```
INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop");

INSERT INTO artists (name, country, genre)
VALUES ("Led Zeppelin", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("ABBA", "Sweden", "Disco");

INSERT INTO artists (name, country, genre)
VALUES ("Queen", "UK", "Rock");

INSERT INTO artists (name, country, genre)
VALUES ("Celine Dion", "Canada", "Pop");

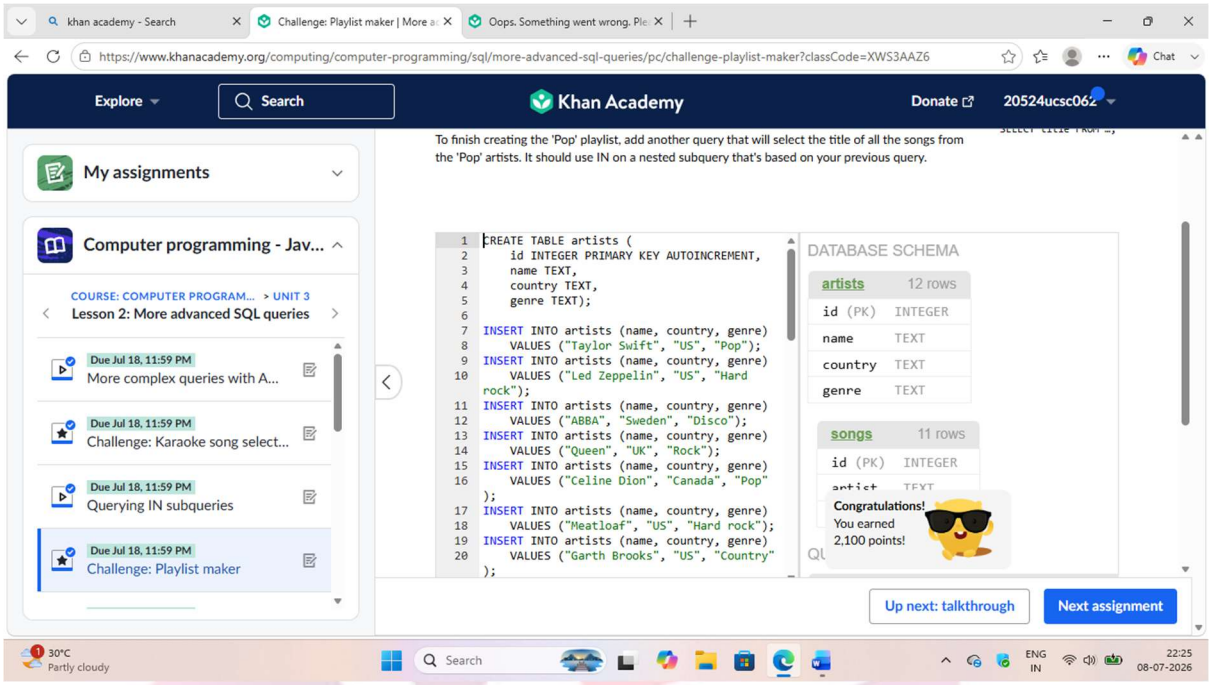
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");


CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
```

```
    artist INTEGER
  );
INSERT INTO songs (title, artist)
  VALUES ("Bicycle", 4);
 INSERT INTO songs (title, artist)
  VALUES ("Bohemian Rhapsody", 4);
 INSERT INTO songs (title, artist)
  VALUES ("Love Story", 1);
 INSERT INTO songs (title, artist)
  VALUES ("Shake It Off", 1);
 INSERT INTO songs (title, artist)
  VALUES ("My Heart Will Go On", 5);
-- Pop playlist
  SELECT title
  FROM songs
  WHERE artist IN (
    SELECT id
    FROM artists
    WHERE genre = "Pop"
  );
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS



THE WORDIEST AUTHOR

QUESTION

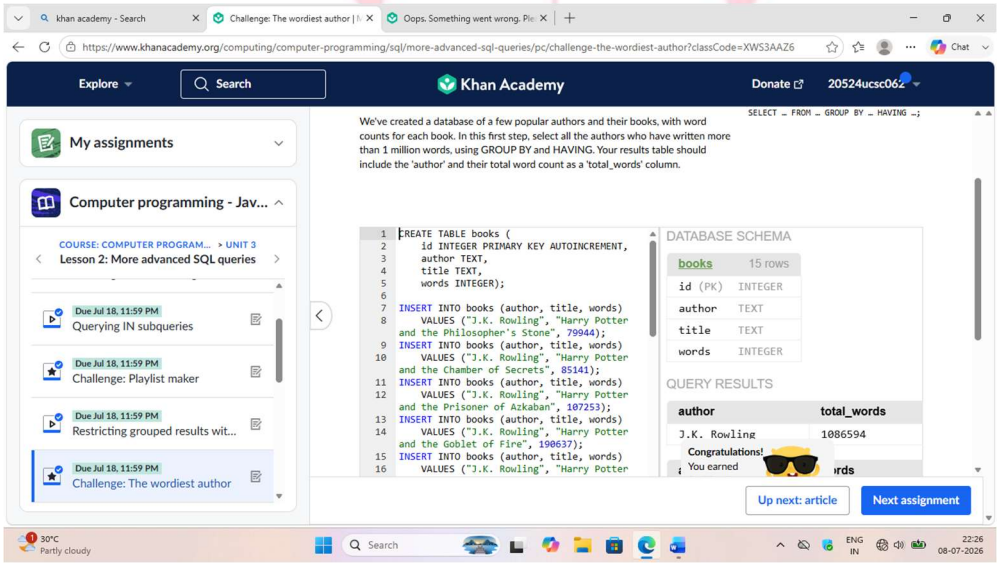
STEP 1

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

ANSWER

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER
);
```

```
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
SELECT author, SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
```



Step 2

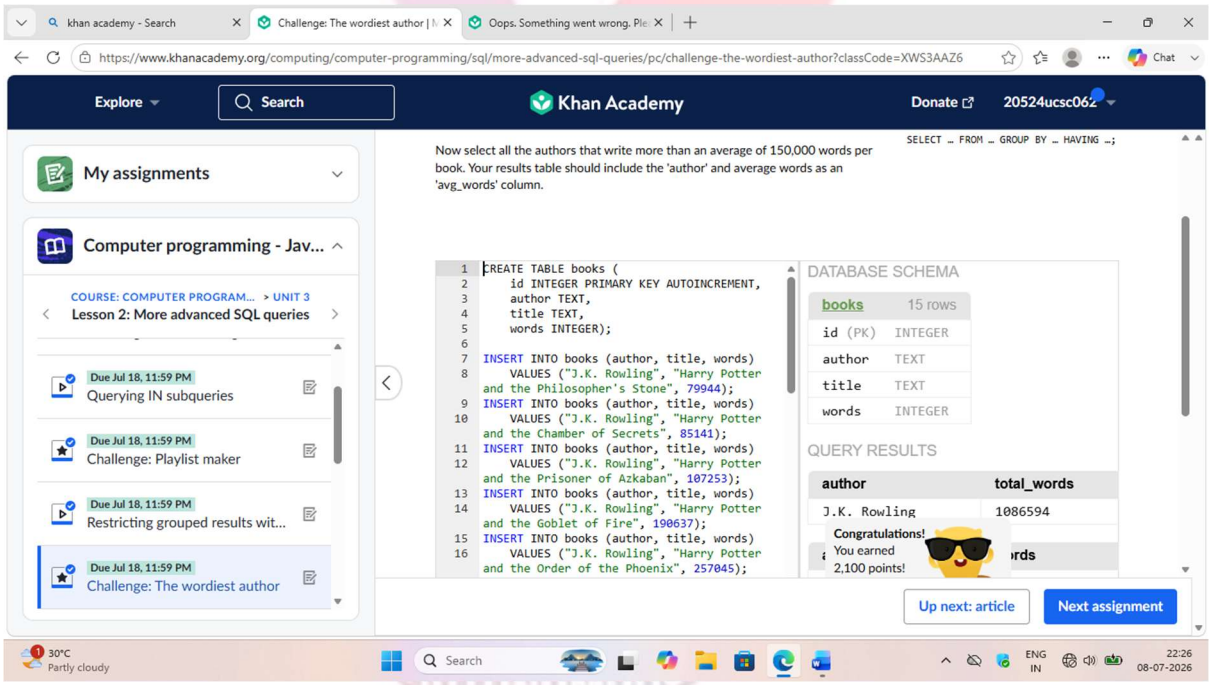
QUESTION

Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

ANSWER

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER
);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
```

```
  INSERT INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 198227);
SELECT author, AVG(words) AS avg_words
  FROM books
  GROUP BY author;
SELECT author, AVG(words) AS avg_words
  FROM books
  GROUP BY author
  HAVING AVG(words) > 150000;
```



GRADEBOOK

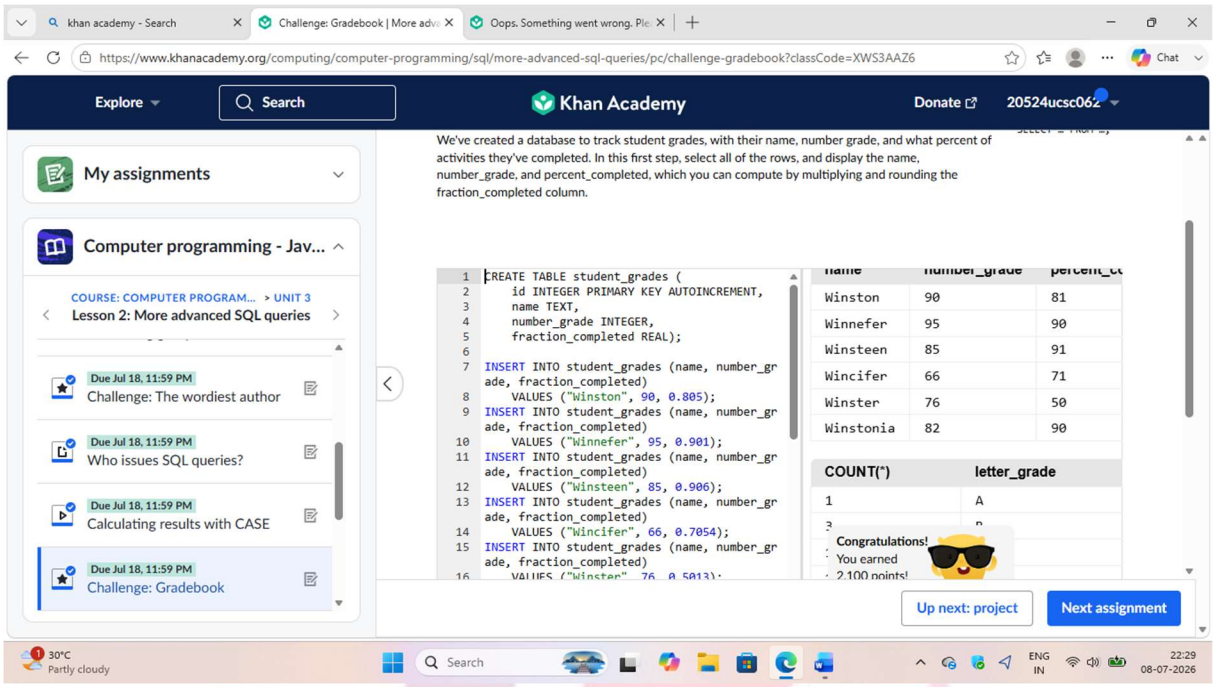
QUESTION

STEP 1

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

ANSWER

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL
);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winstonia", 82, 0.9045);
SELECT
    name,
    number_grade,
    ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
```



QUESTION

Step 2

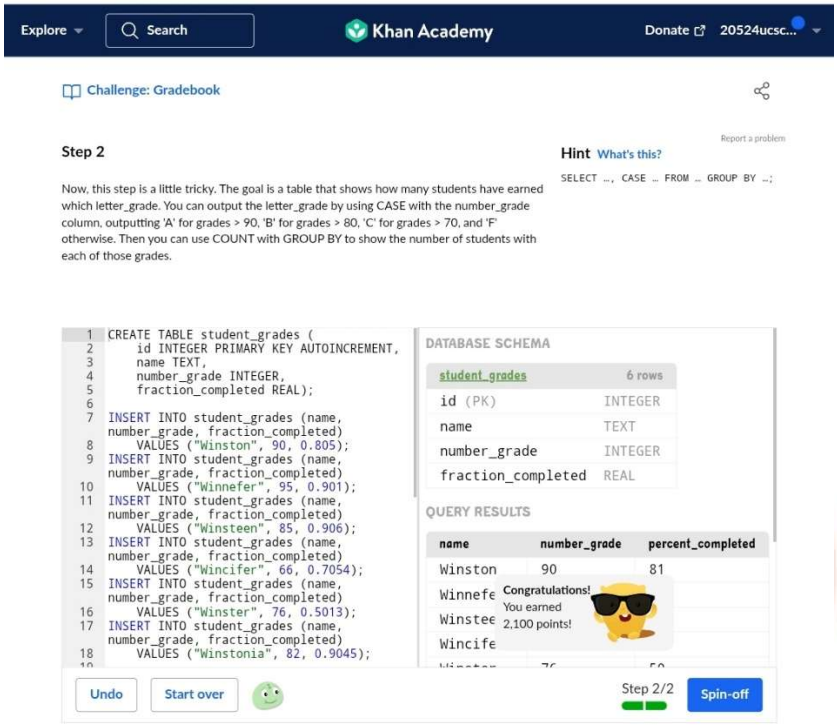
Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

ANSWER

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL
);


INSERT INTO student_grades (name, number_grade, fraction_completed)
```

```
 VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
 VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
 VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
 VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
 VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
 VALUES ("Winstonia", 82, 0.9045);
SELECT
    CASE
        WHEN number_grade > 90 THEN 'A'
        WHEN number_grade > 80 THEN 'B'
        ELSE 'C'
    END AS letter_grade,
    COUNT(*) AS number_of_students
FROM student_grades
GROUP BY letter_grade;
```



BOBBY HOBBIES

QUESTION

STEP 1

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

ANSWER

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER
);


INSERT INTO persons (name, age)
```

```
VALUES ("Bobby McBobbyFace", 12);

INSERT INTO persons (name, age)
VALUES ("Lucy BoBucie", 25);

INSERT INTO persons (name, age)
VALUES ("Banana Fana", 14);

INSERT INTO persons (name, age)
VALUES ("Shish Kabob", 20);

INSERT INTO persons (name, age)
VALUES ("Fluffy Sparkles", 8);


CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);

INSERT INTO hobbies (person_id, name)
VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name)
VALUES (1, "coding");
```

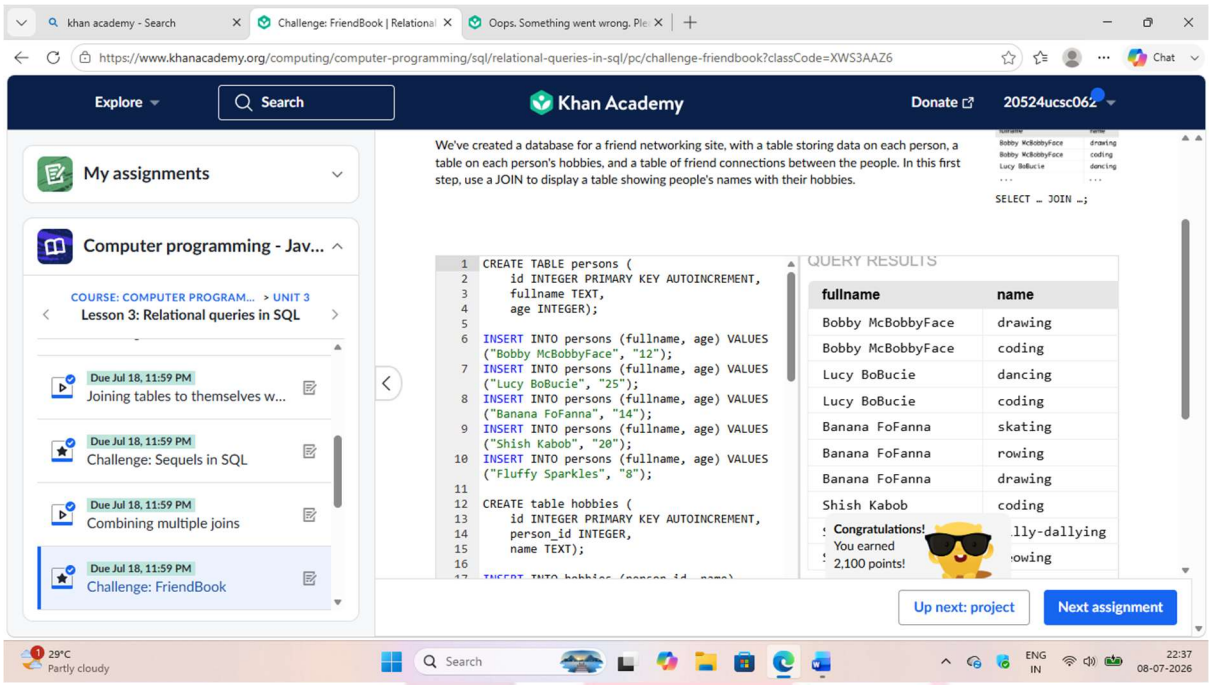
```
INSERT INTO hobbies (person_id, name)
VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name)
VALUES (2, "coding");
INSERT INTO hobbies (person_id, name)
VALUES (3, "skating");
INSERT INTO hobbies (person_id, name)
VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name)
VALUES (4, "hiking");
INSERT INTO hobbies (person_id, name)
VALUES (4, "coding");
INSERT INTO hobbies (person_id, name)
VALUES (5, "drawing");
INSERT INTO hobbies (person_id, name)
VALUES (5, "dancing");
INSERT INTO hobbies (person_id, name)
VALUES (5, "playing");
```

Add one new person

```
INSERT INTO persons (name, age)
VALUES ("New Person", 18);
```

Add one hobby for the new person

```
INSERT INTO hobbies (person_id, name)
VALUES (6, "reading");
```



CUSTOMER ORDER

QUESTION

STEP 1

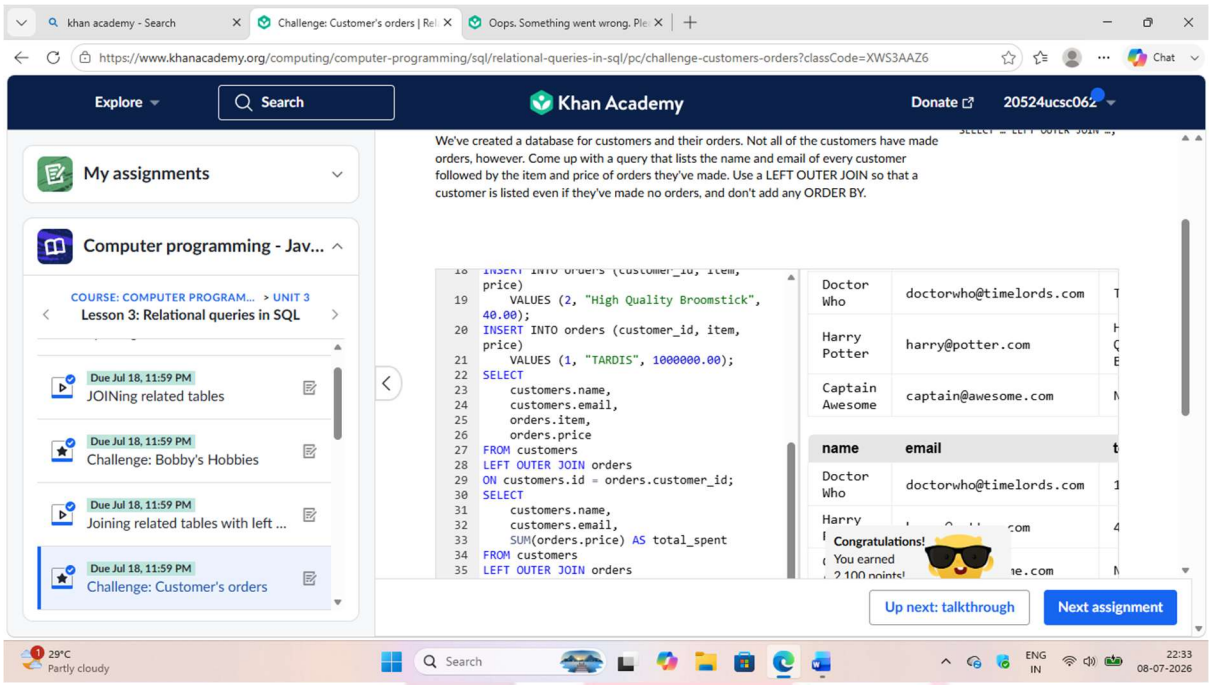
We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

ANSWER

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT
);


INSERT INTO customers (name, email)
VALUES ("Doctor Who", "doctorwho@timelords.com");
```

```
INSERT INTO customers (name, email)
VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email)
VALUES ("Captain Awesome", "captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL
);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
      customers.email,
      orders.item,
      orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
```



STEP 2

QUESTION

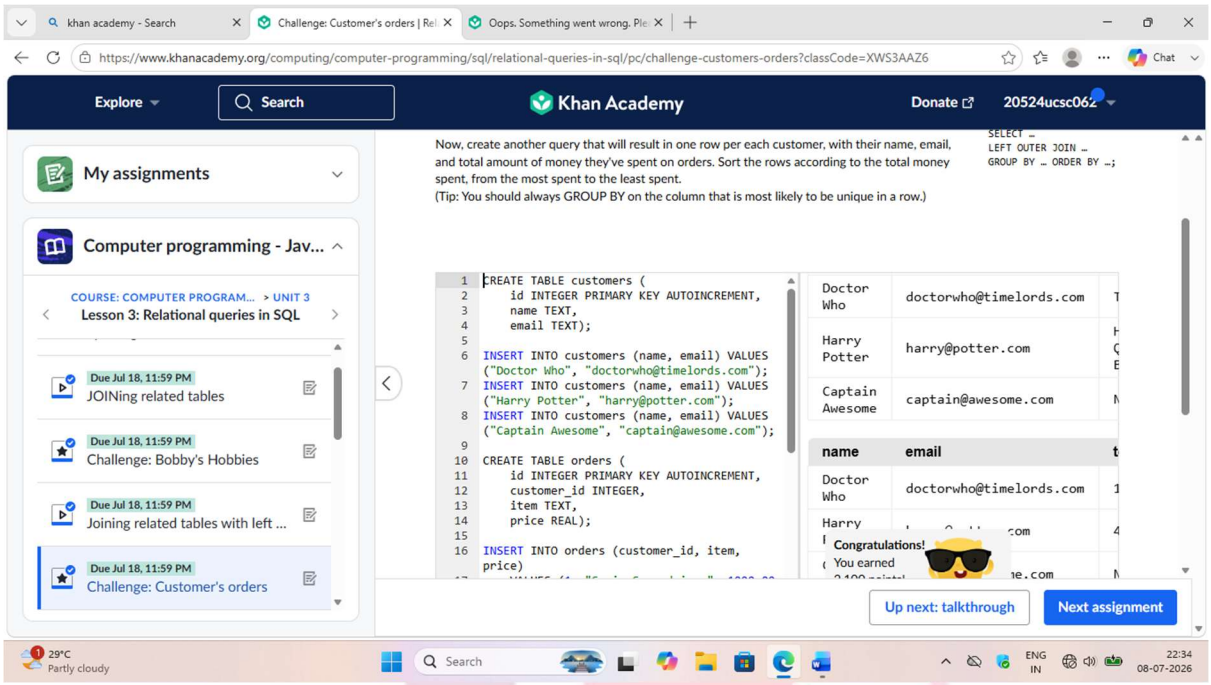
Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

ANSWER

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT
);


INSERT INTO customers (name, email)
VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email)
VALUES ("Harry Potter", "harry@potter.com");
```

```
INSERT INTO customers (name, email)
VALUES ("Captain Awesome", "captain@awesome.com");
CREATE TABLE orders (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   customer_id INTEGER,
   item TEXT,
   price REAL
);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "TARDIS", 1000000.00);
SELECT
   customers.name,
   customers.email,
   SUM(orders.price) AS total_money_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_money_spent DESC;
```



SEQUELS IN SQL

QUESTION

STEP 1

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

ANSWER

```
CREATE TABLE movies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER
);
INSERT INTO movies (id, title, released, sequel_id)
VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies (id, title, released, sequel_id)
```

```
VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (5, "Harry Potter and the Order of the Phoenix", 2007, 6);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (7, "Harry Potter and the Deathly Hallows - Part 1", 2010, 8);

INSERT INTO movies (id, title, released, sequel_id)

VALUES (8, "Harry Potter and the Deathly Hallows - Part 2", 2011, NULL);

SELECT

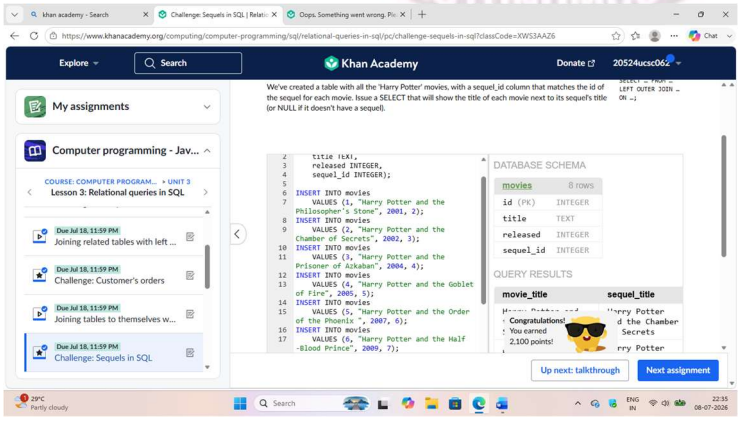
    movies.title AS movie_title,

    sequels.title AS sequel_title

FROM movies

LEFT JOIN movies AS sequels

ON movies.sequel_id = sequels.id;
```



HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the **CITY** table with populations larger than 100000. The **CountryCode** for America is USA.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'USA'
AND POPULATION > 100000;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test case

Sample Test case 0

Your Output (stdout)

```
3878 Scottsdale USA Arizona 202705
3965 Corona USA California 124966
3973 Concord USA California 121780
3977 Cedar Rapids USA Iowa 120758
3982 Coral Springs USA Florida 117549
```

Expected Output Download

```
3878 Scottsdale USA Arizona 202705
3965 Corona USA California 124966
3973 Concord USA California 121780
3977 Cedar Rapids USA Iowa 120758
3982 Coral Springs USA Florida 117549
```

REVISING THE SELECT QUERY II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than 120000. The *CountryCode* for America is USA.

The **CITY** table is described as follows:

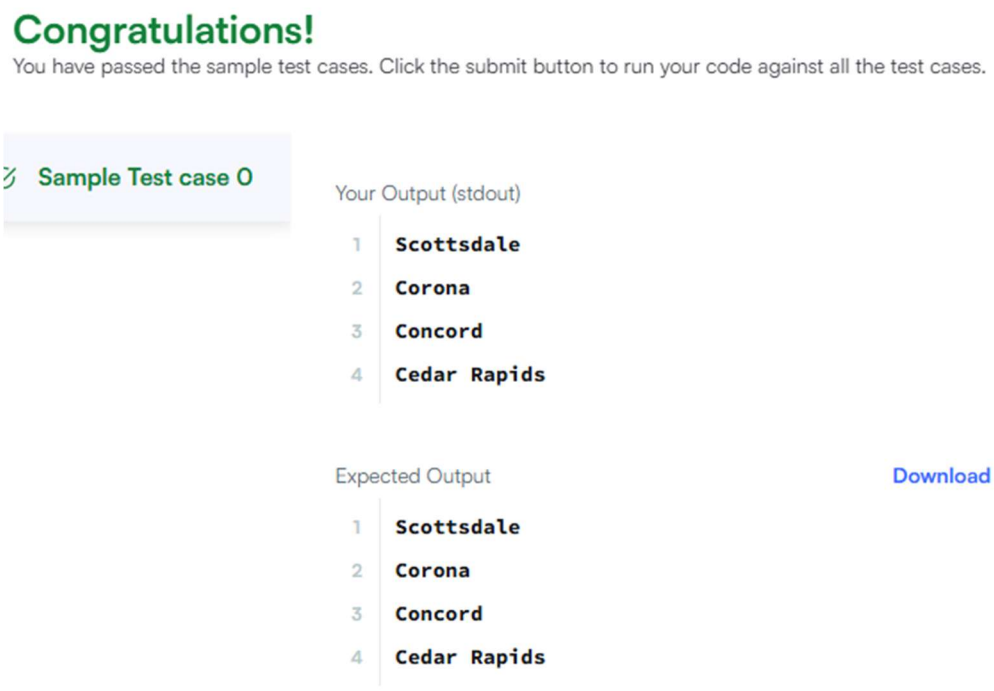
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT NAME
FROM CITY
WHERE POPULATION > 120000
AND COUNTRYCODE = 'USA';
```

OUTPUT



SELECT ALL

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

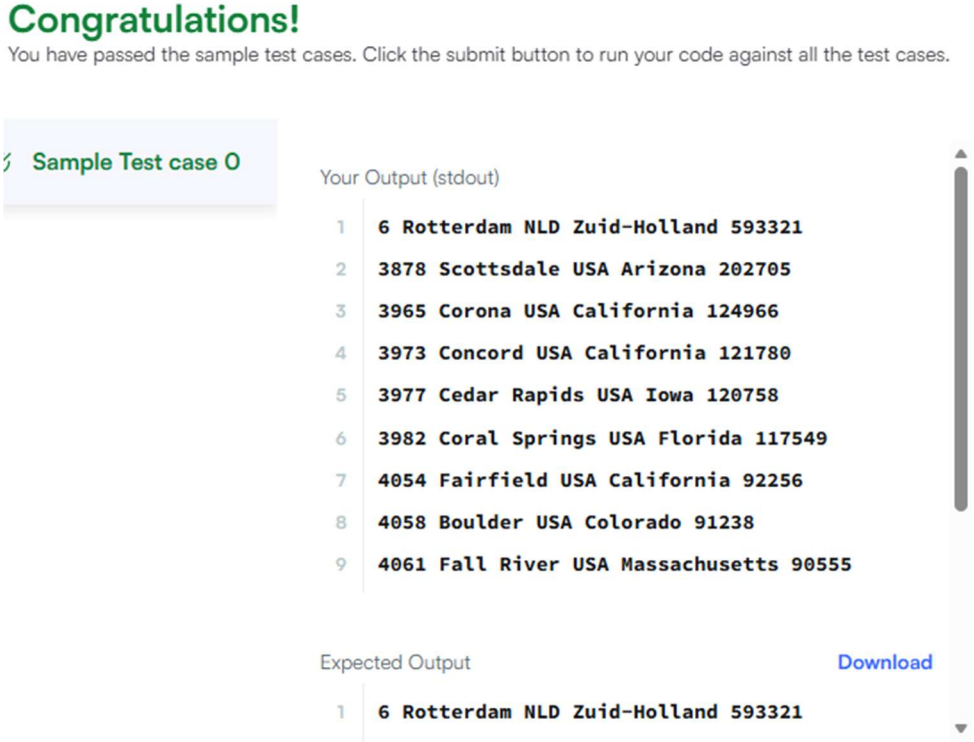
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

SELECT*FROM CITY;

OUTPUT



SELECT BY ID

Query all columns for a city in **CITY** with the *ID* 1661.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1661 Sayama JPN Saitama 162472
```

JAPANESE CITIES ATTRIBUTES

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1613 Neyagawa JPN Osaka 257315
1630 Ageo JPN Saitama 209442
1661 Sayama JPN Saitama 162472
1681 Omuta JPN Fukuoka 142889
1739 Tokuyama JPN Yamaguchi 107078
```

JAPANESE CITIES NAMES

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Neyagawa
Ageo
Sayama
Omuta
Tokuyama
```

WEATHER OBSERVATION STATION I

Query a list of CITY and STATE from the STATION table. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT CITY, STATE
FROM STATION;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Kissee Mills MO
Loma Mar CA
Sandy Hook CT
Tipton IN
Arlington CO
Turner AR
Slidell LA
Negreet LA
Glencoe KY
Chelsea IA
Chignik Lagoon AK
Pelahatchie MS
Hanna City IL
```

WEATHER OBSERVATION STATION 3

Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE MOD(ID, 2) = 0;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Kissee Mills
Loma Mar
Tipton
Glencoe
Chignik Lagoon
Albany
Manchester
Cahone
Bowdon
Watkins
Millville
Aguanga
Morenci
```

WEATHER OBSERVATION STATION 4

Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

SELECT COUNT(CITY) - COUNT(DISTINCT CITY)

FROM STATION;

OUTPUT



WEATHER OBSERVATION STATION 6

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

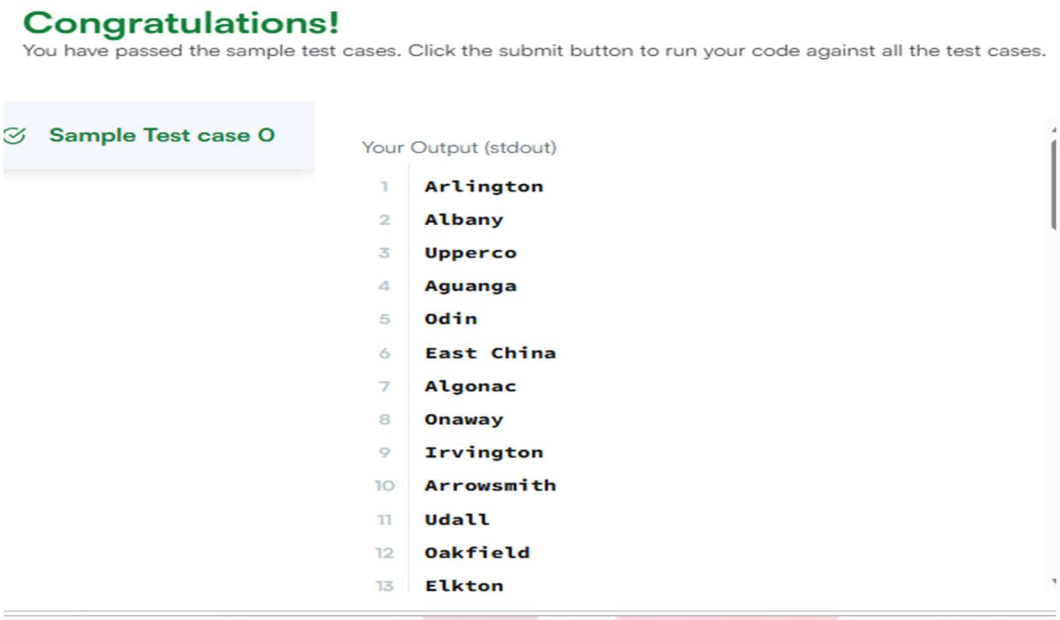
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



WEATHER OBSERVATION STSTION 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

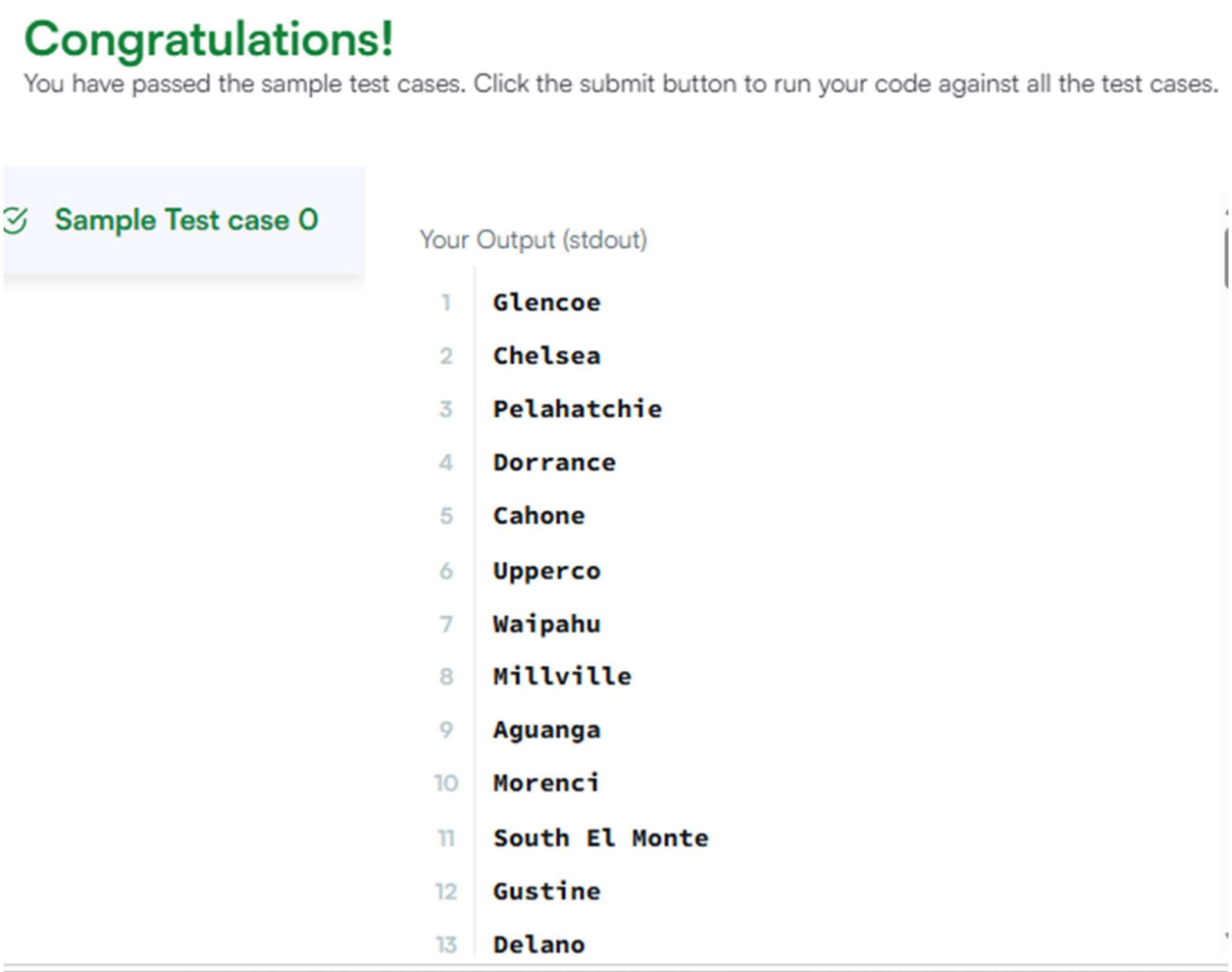
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE RIGHT(CITY, 1) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



WEATHER OBSERVATION STATION 8

Query the list of *CITY* names from STATION which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

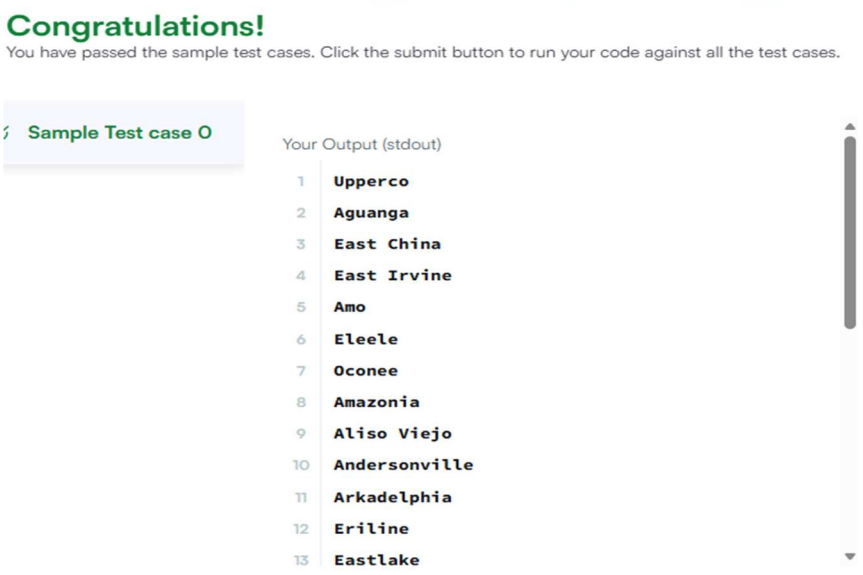
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(LEFT(CITY, 1)) IN ('a', 'e', 'i', 'o', 'u')
  AND LOWER(RIGHT(CITY, 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



WEATHER OBSERVATION STATION 9

Query the list of *CITY* names from STATION that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

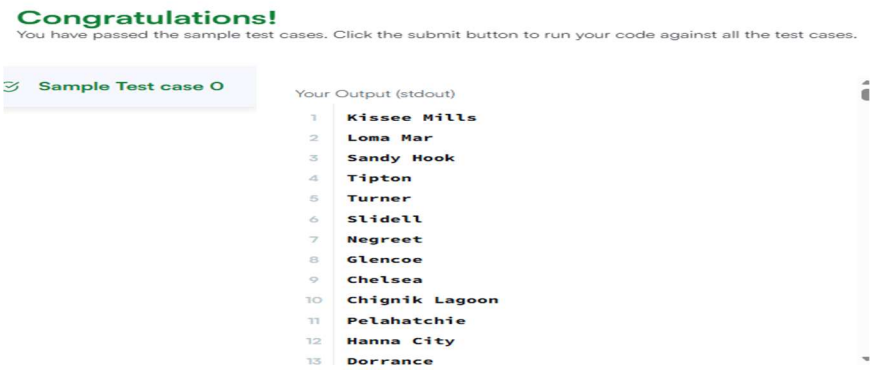
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



WEATHER OBSERVATIPON STATION 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

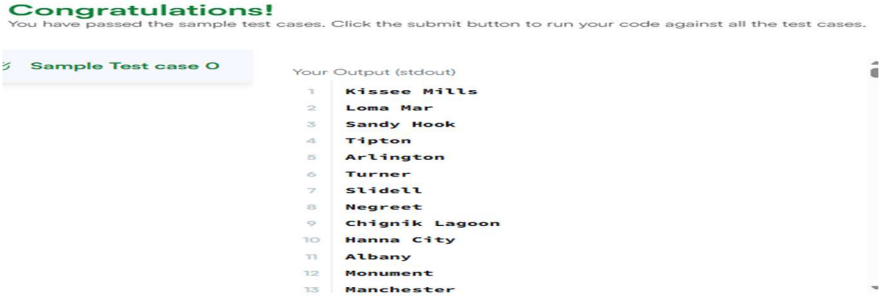
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, -1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



WEATHER OBSERVATION STATION 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

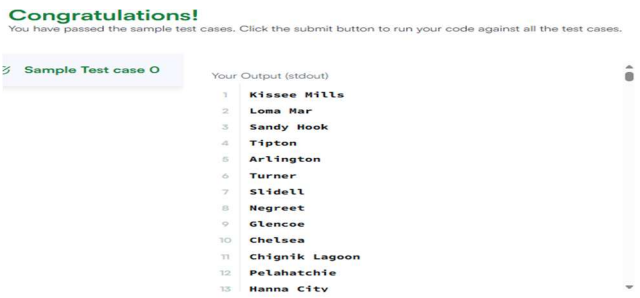
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(LEFT(CITY, 1)) NOT IN ('A', 'E', 'I', 'O', 'U')
   OR UPPER(RIGHT(CITY, 1)) NOT IN ('A', 'E', 'I', 'O', 'U');
```

OUTPUT



HIGHER THAN 75 MARKS

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

The **STUDENTS** table is described as follows:

Column	Type
ID	Integer
Name	String
Marks	Integer

The *Name* column only contains uppercase (A-Z) and lowercase (a-z) letters.

Sample Input

ID	Name	Marks
1	Ashley	81
2	Samantha	75
4	Julia	76
3	Belvet	84

Sample Output

Ashley

Julia

Belvet

Explanation

Only Ashley, Julia, and Belvet have *Marks* >

INPUT

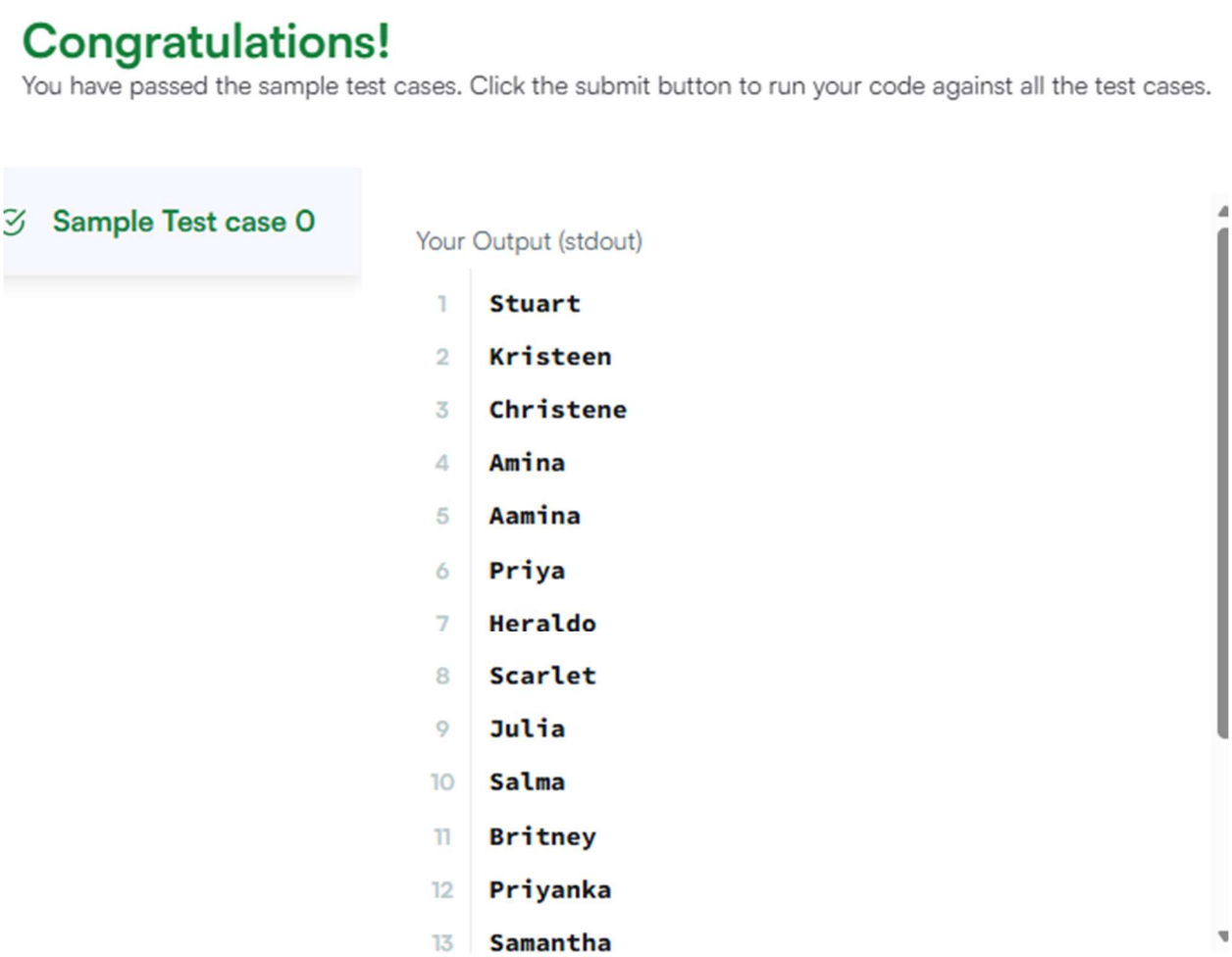
SELECT Name

FROM STUDENTS

WHERE Marks > 75

ORDER BY SUBSTR(Name, -3), ID;

OUTPUT



EMPLOYEE NAMES

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the Employee table in alphabetical order.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

Angela

Bonnie

Frank

Joe

Kimberly

Lisa

Michael

Patrick

Rose

Todd

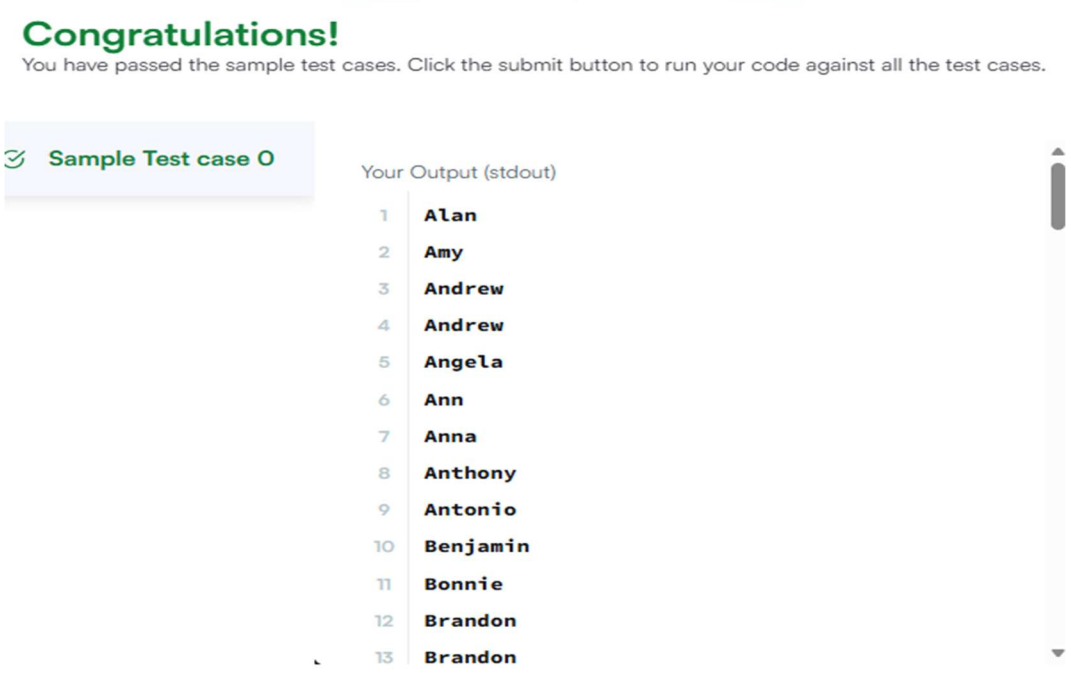
INPUT

SELECT name

FROM Employee

ORDER BY name ASC;

OUTPUT



EMPLOYEE SALARIES

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in Employee having a salary greater than per month who have been employees for less than

months. Sort your result by ascending *employee_id*.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly s

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

Angela

Michael

Todd

Joe

Explanation

Angela has been an employee for salaries

INPUT

SELECT name

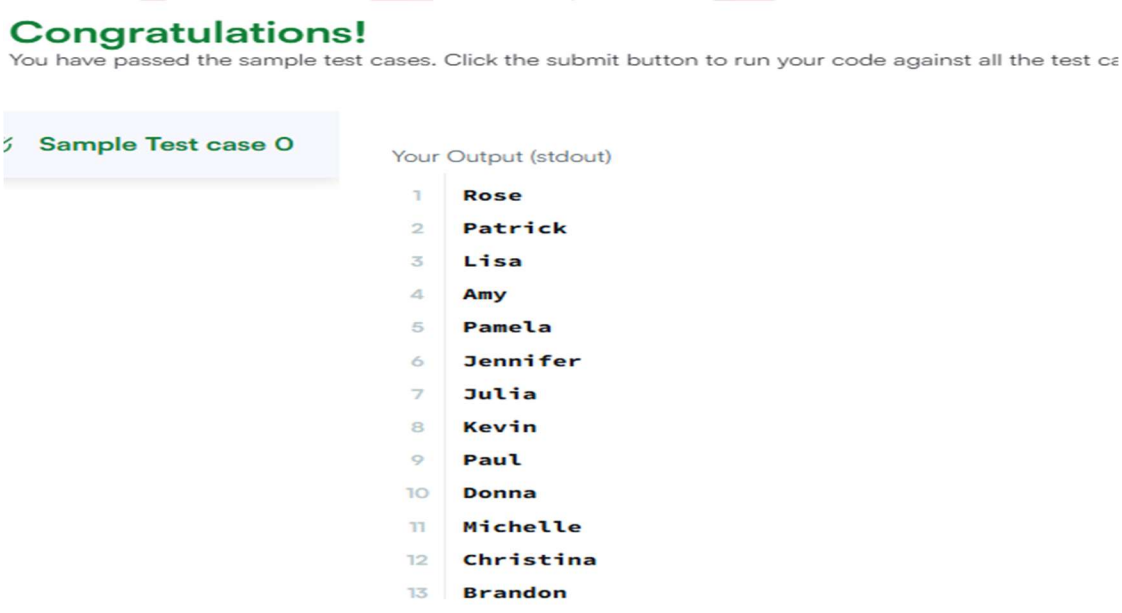
FROM Employee

WHERE salary > 2000

AND months < 10

ORDER BY employee_id ASC;

OUTPUT



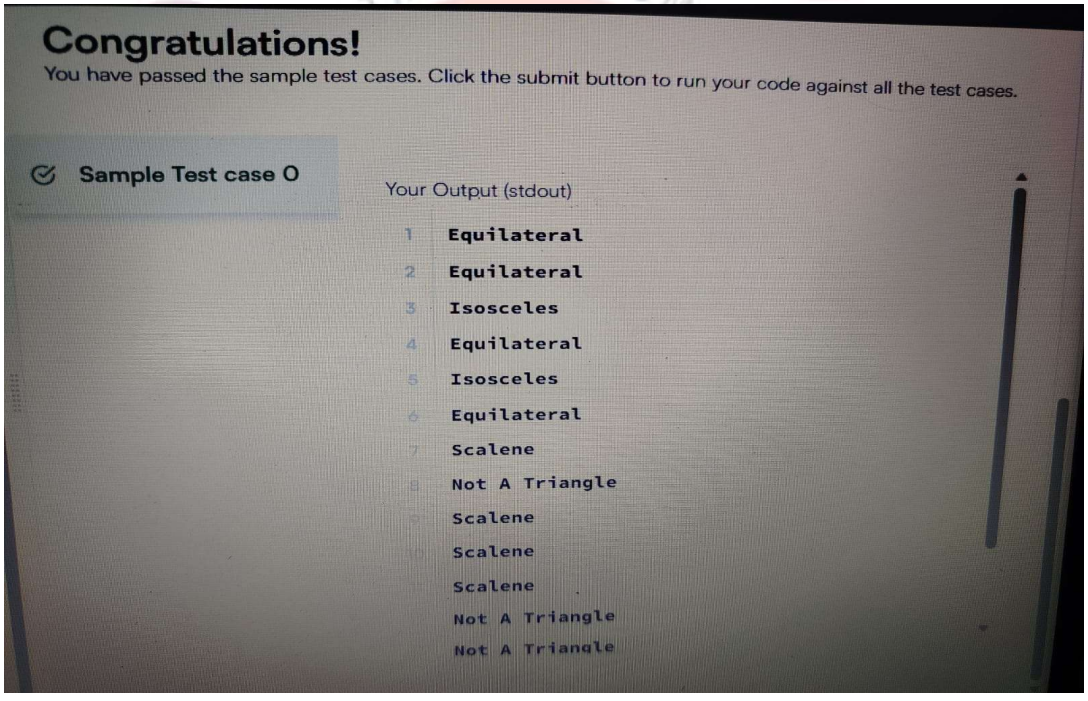
TYPES OF TRIANGLE

Write a query identifying the *type* of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:

INPUT

```
SELECT
  CASE
    WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
    WHEN A = B AND B = C THEN 'Equilateral'
    WHEN A = B OR B = C OR A = C THEN 'Isosceles'
    ELSE 'Scalene'
  END
FROM TRIANGLES;
```

OUTPUT



THE PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in *ascending order*, and output them in the following format:
3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: Doctor, Professor, Singer or Actor.

Sample Input

An OCCUPATIONS table that contains the following records:

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

Ashely(P)

Christeen(P)

Jane(A)

Jenny(D)

Julia(A)

Ketty(P)

Maria(A)

Meera(S)

Priya(S)

Samantha(D)

There are a total of 2 doctors.

There are a total of 2 singers.

There are a total of 3 actors.

There are a total of 3 professors.

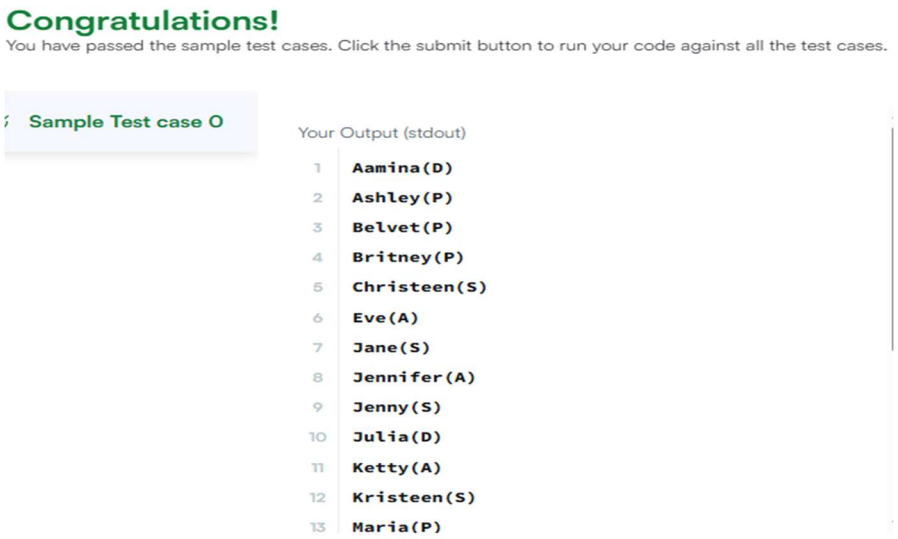
Explanation

The results of the first query are formatted to the problem description's specifications.
The results of the second query are ascendingly ordered first by number of names corresponding to each profession (

INPUT

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(*), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(*), Occupation;
```

OUTPUT



OCCUPATIONS

Pivot the *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: Doctor, Professor, Singer or Actor.

Sample Input

TRUTH-TOIL-TRIUMPH AUTONOMOUS

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

Jenny Ashley Meera Jane

Samantha Christeen Priya Julia

NULL Ketty NULL Maria

Explanation

The first column is an alphabetically ordered list of Doctor names.
The second column is an alphabetically ordered list of Professor names.
The third column is an alphabetically ordered list of Singer names.
The fourth column is an alphabetically ordered list of Actor names.
The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with NULL values.

INPUT

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM
(
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS T
GROUP BY rn
ORDER BY rn;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Aamina Ashley Christeen Eve
Julia Belvet Jane Jennifer
Priya Britney Jenny Ketty
NULL Maria Kristeen Samantha
NULL Meera NULL NULL
NULL Naomi NULL NULL
NULL Priyanka NULL NULL
```

BINARY TREE NODES

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

Sample Output

1 Leaf

2 Inner

3 Leaf

5 Root

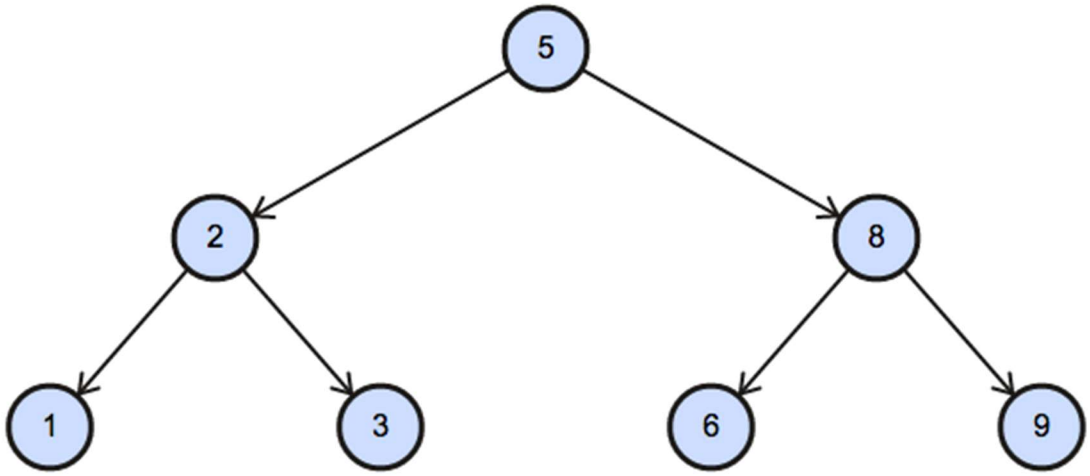
6 Leaf

8 Inner

9 Leaf

Explanation

The *Binary Tree* below illustrates the sample:

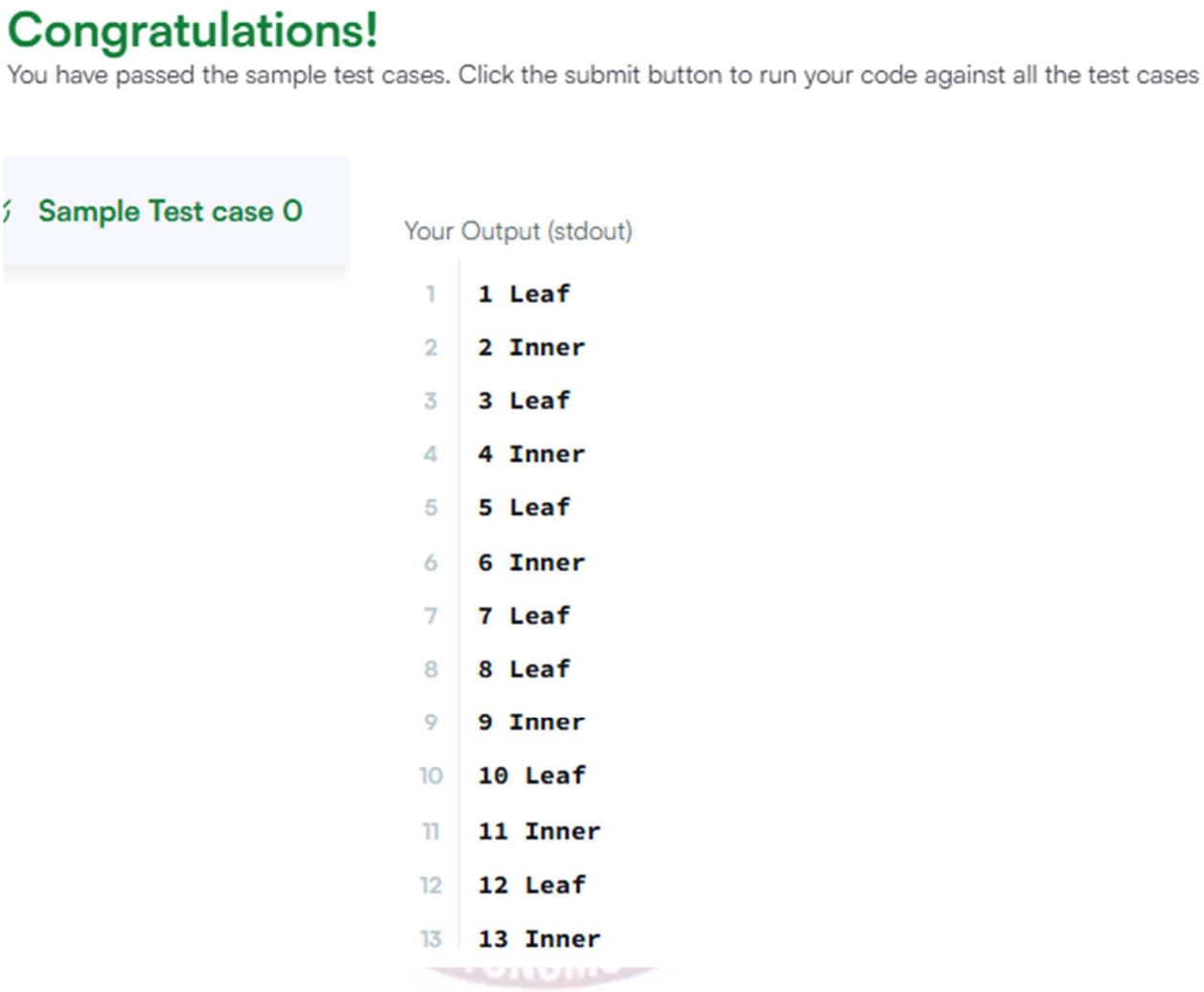


INPUT

```
SELECT
   N,
   CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N NOT IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN
'Leaf'
      ELSE 'Inner'
```

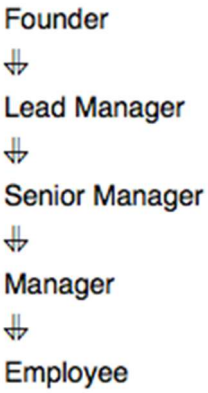
```
    END AS NodeType
FROM BST
ORDER BY N;
```

OUTPUT



NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- **The tables may contain duplicate records.**
- **The *company_code* is string, so the sorting should not be numeric. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.**

Input Format

The following tables contain company data:

- ***Company:* The *company_code* is the code of the company and *founder* is the founder of the company.**

Column	Type
company_code	String
founder	String

- ***Lead_Manager:*** **The *lead_manager_code* is the code of the lead manager, and the *company_code* is the code of the working company.**

Column	Type
lead_manager_code	String
company_code	String

- ***Senior_Manager:*** **The *senior_manager_code* is the code of the senior manager, the *lead_manager_code* is the code of its lead manager, and the *company_code* is the code of the working company.**

Column	Type
senior_manager_code	String
lead_manager_code	String
company_code	String

- ***Manager:*** **The *manager_code* is the code of the manager, the *senior_manager_code* is the code of its senior manager, the *lead_manager_code* is the code of its lead manager, and the *company_code* is the code of the working company.**

Column	Type
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

- ***Employee:*** **The *employee_code* is the code of the employee, the *manager_code* is the code of its manager, the *senior_manager_code* is the code of its senior manager, the**

lead_manager_code **is the code of its lead manager, and the** ***company_code*** **is the code of the working company.**

Column	Type
employee_code	String
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

Sample Input

Company **Table:**

company_code	founder
C1	Monika
C2	Samantha

Lead_Manager **Table:**

lead_manager_code	company_code
LM1	C1
LM2	C2

Senior_Manager **Table:**

senior_manager_code	lead_manager_code	company_code
SM1	LM1	C1
SM2	LM1	C1
SM3	LM2	C2

Manager **Table:**

manager_code	senior_manager_code	lead_manager_code	company_code
M1	SM1	LM1	C1
M2	SM3	LM2	C2
M3	SM3	LM2	C2

Employee **Table:**

employee_code	manager_code	senior_manager_code	lead_manager_code	company_code
E1	M1	SM1	LM1	C1
E2	M1	SM1	LM1	C1
E3	M2	SM3	LM2	C2
E4	M3	SM3	LM2	C2

Sample Output

C1 Monika 1 2 1 2

C2 Samantha 1 1 2 2

Explanation

In company *C1*, the only lead manager is *LM1*. There are two senior managers, *SM1* and *SM2*, under *LM1*. There is one manager, *M1*, under senior manager *SM1*. There are two employees, *E1* and *E2*, under manager *M1*.

In company *C2*, the only lead manager is *LM2*. There is one senior manager, *SM3*, under *LM2*. There are two managers, *M2* and *M3*, under senior manager *SM3*. There is one employee, *E3*, under manager *M2*, and another employee, *E4*, under manager, *M3*.

INPUT

SELECT

```
c.company_code,
c.founder,
COUNT(DISTINCT lm.lead_manager_code),
COUNT(DISTINCT sm.senior_manager_code),
```

```
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager lm
    ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm
    ON c.company_code = sm.company_code
LEFT JOIN Manager m
    ON c.company_code = m.company_code
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY
    c.company_code,
    c.founder
ORDER BY
    c.company_code;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
C2 Frank 1 1 1 3
```

REVISING AGGREGATION-THE COUNT FUNCTION

Query a *count* of the number of cities in **CITY** having a *Population* larger than

Input Format

AUTONOMOUS

The **CITY** table is described as follows:

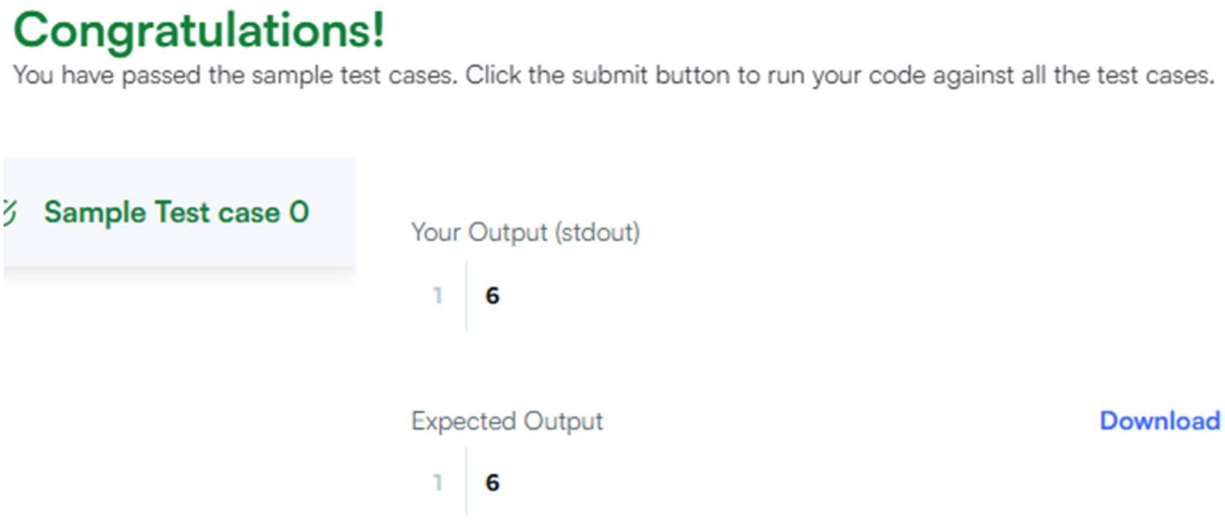
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT COUNT(*)
FROM CITY
WHERE POPULATION > 100000;
```

OUTPUT



REVISING AGGREGATIONS-THE SUM FUNCTION

Query the total population of all cities in CITY where *District* is California.

Input Format

The CITY table is described as follows:

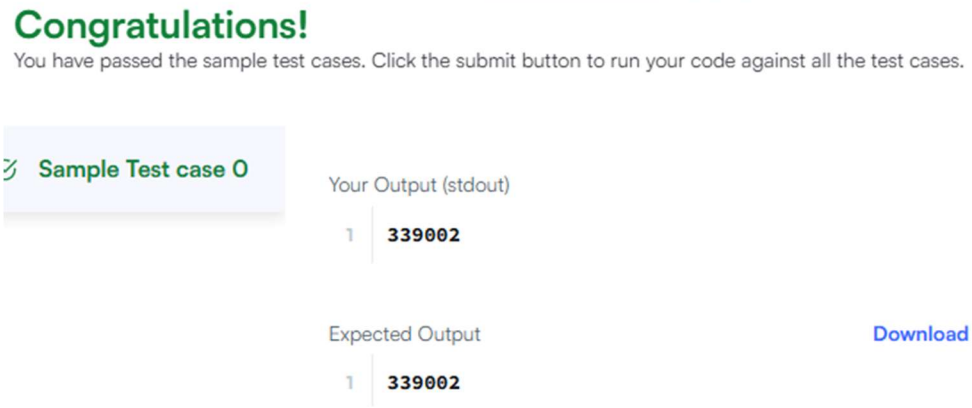
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT SUM(Population)
FROM CITY
WHERE District = 'California';
```

OUTPUT



REVISING AGGREGATIONS-AVERAGES

Query the average population of all cities in CITY where *District* is California.

Input Format

The CITY table is described as follows:

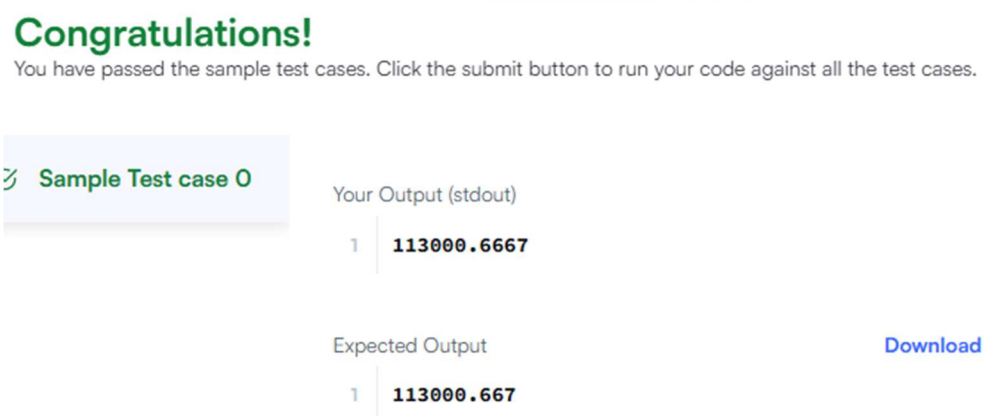
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



AVERAGE POPULATION

Query the average population for all cities in CITY, rounded *down* to the nearest integer.

Input Format

The CITY table is described as follows:

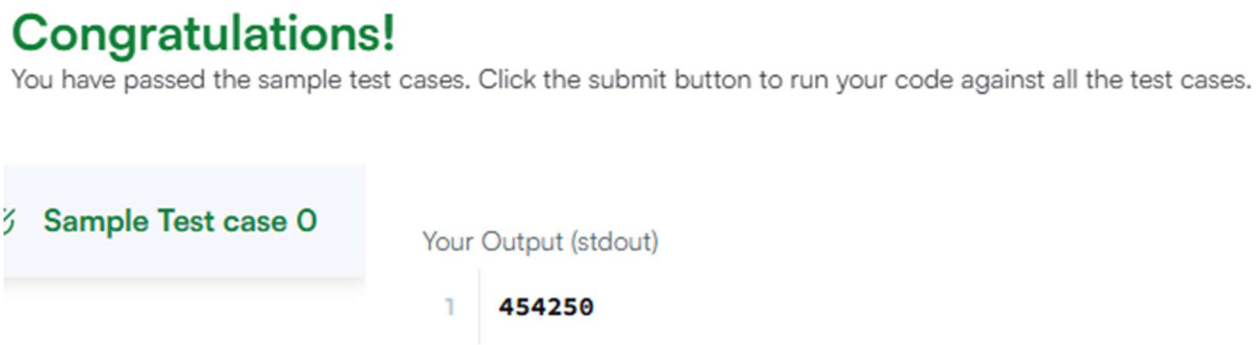
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in CITY. The *COUNTRYCODE* for Japan is JPN.

Input Format

The CITY table is described as follows:

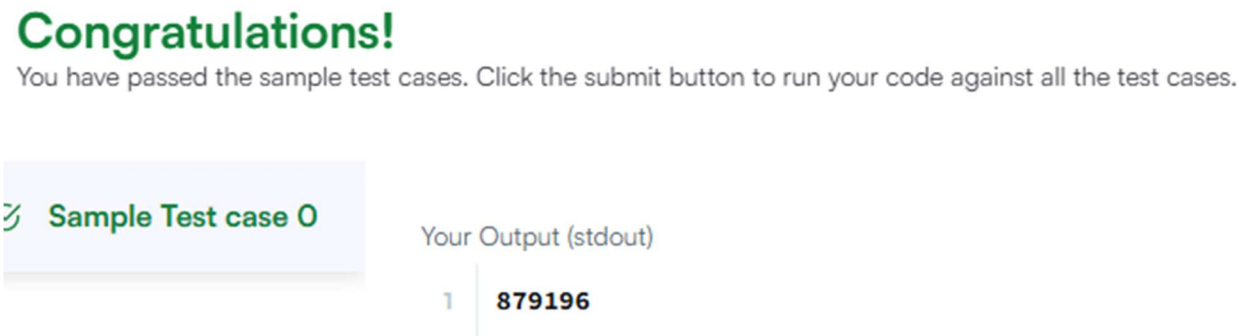
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in CITY.

Input Format

The CITY table is described as follows:

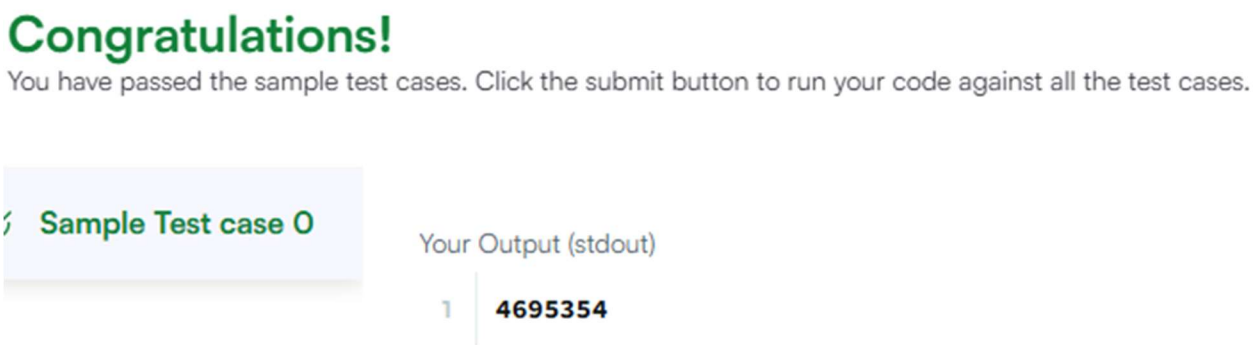
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT MAX(POPULATION) - MIN(POPULATION)
FROM CITY;
```

OUTPUT



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's

key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

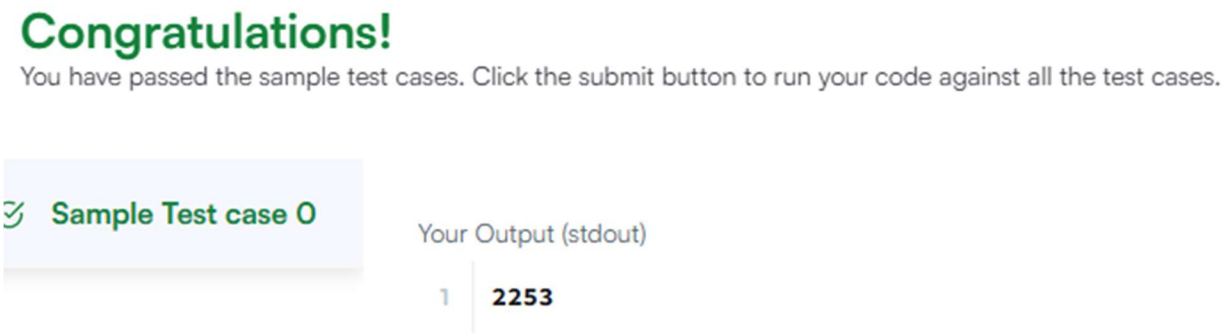
The EMPLOYEES table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

INPUT

```
SELECT CEIL(
   AVG(SALARY) - AVG(
      CAST(REPLACE(SALARY, '0', '') AS DECIMAL(10,2))
   )
)
FROM EMPLOYEES;
```

OUTPUT



TOP EARNER

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the Employee table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

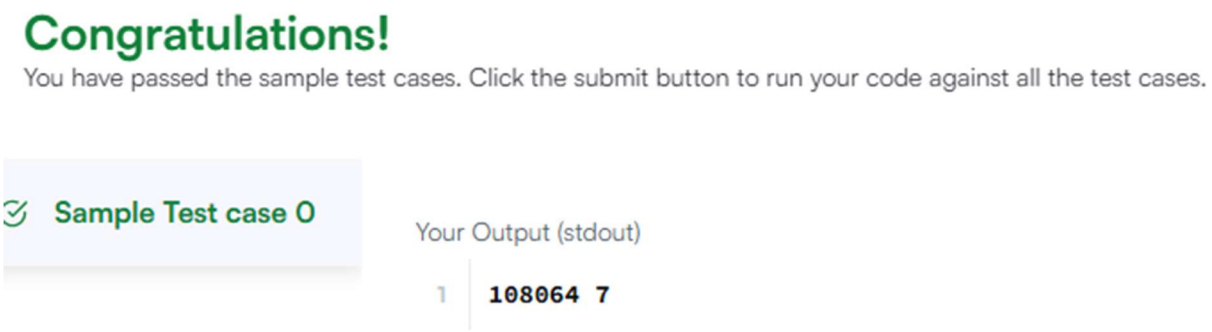
Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

INPUT

```
SELECT MAX(earnings), COUNT(*)
FROM (
    SELECT salary * months AS earnings
    FROM Employee
) AS emp
WHERE earnings = (
    SELECT MAX(salary * months)
    FROM Employee
);
```

OUTPUT



WEATHER OBSERVATION STATION 2

Query the following two values from the STATION table:

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

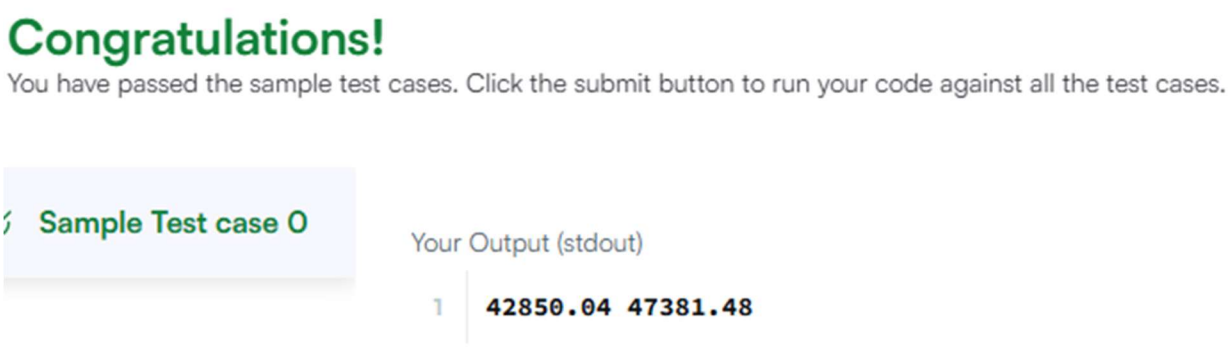
where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
```

FROM STATION;

OUTPUT



WEATHER OBSERVATION STATION 13

Query the sum of *Northern Latitudes* (*LAT_N*) from STATION having values greater than

Truncate your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

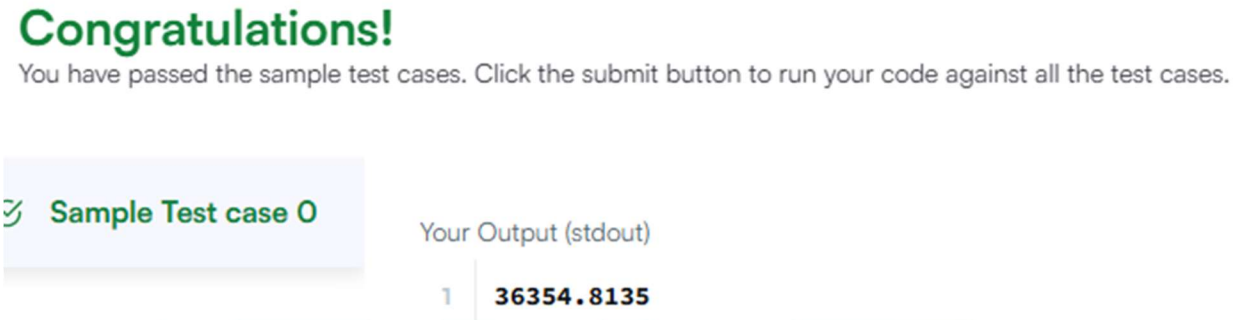
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

INPUT

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT



WEATHER OBSERVATION STATION 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than decimal places.

Input Format

The **STATION** table is described as follows:

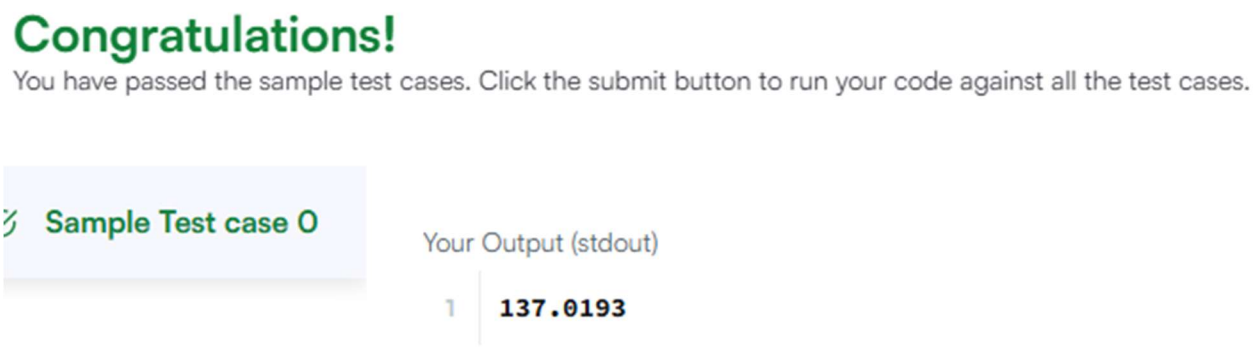
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT



WEATHER OBSERVATION STATION 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in STATION that is less than decimal places.

Input Format

The STATION table is described as follows:

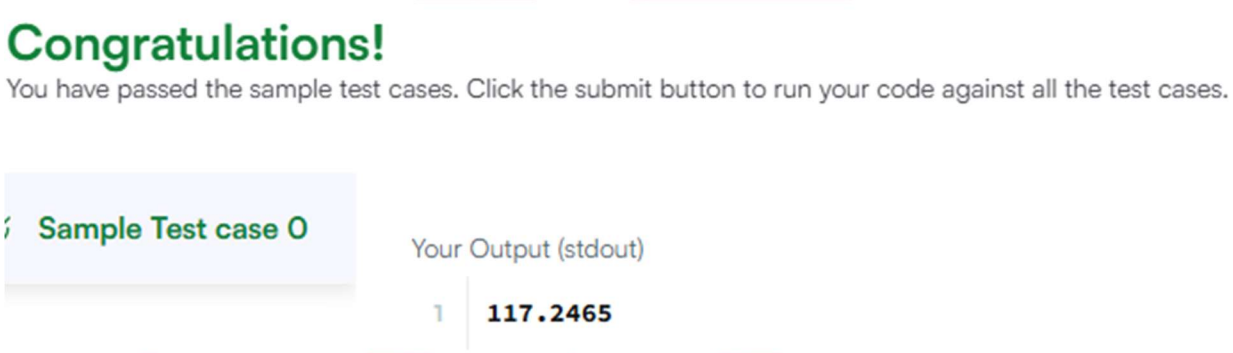
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
);
```

OUTPUT



WEATHER OBSERVATION STATION 16

Query the smallest *Northern Latitude* (*LAT_N*) from STATION that is greater than

decimal places.

Input Format

The STATION table is described as follows:

STATION

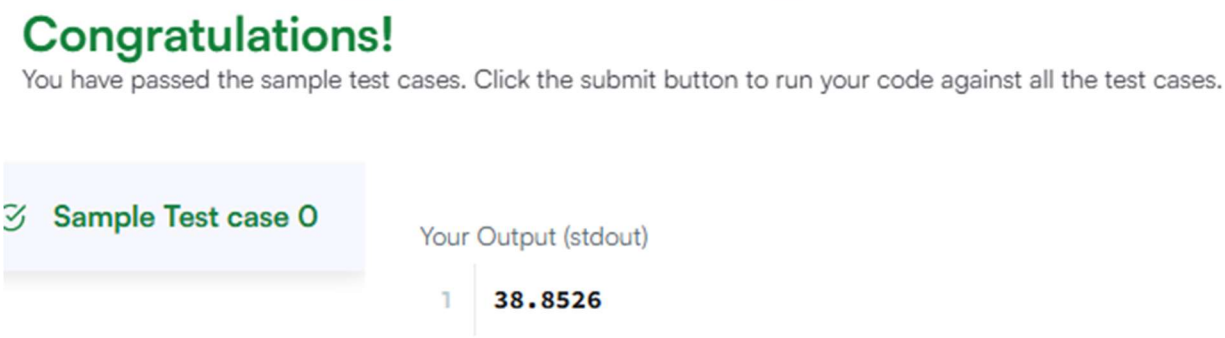
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7780;
```

OUTPUT



WEATHER OBSERVATION STATION 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in STATION is greater than decimal places.

Input Format

The STATION table is described as follows:

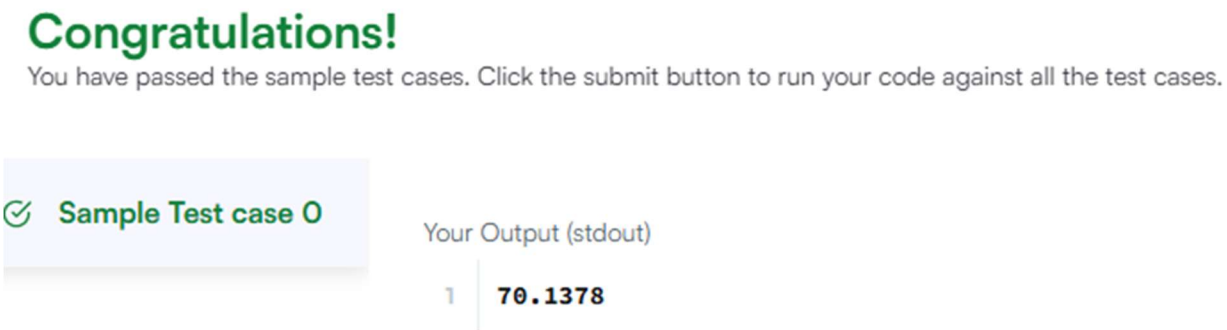
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT



WEATHER OBSERVATION STATION 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from STATION and round your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

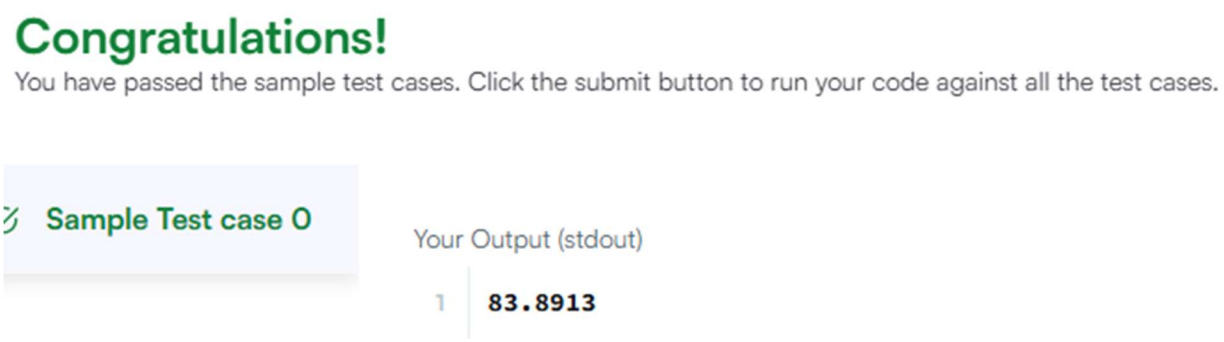
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

INPUT

```
SELECT ROUND(LAT_N, 4)
FROM (
   SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn,
        COUNT(*) OVER () AS total_rows
   FROM STATION
) t
WHERE rn = CEIL(total_rows / 2);
```

OUTPUT



POPULATION CENSUS

Given the CITY and COUNTRY tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The CITY and COUNTRY tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

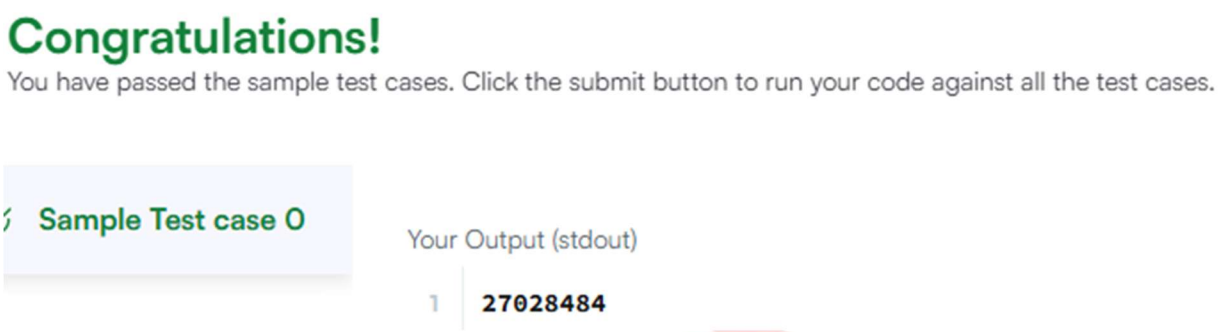
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

INPUT

```
SELECT SUM(CITY.Population)
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Asia';
```

OUTPUT



AFRICAN CITIES

Given the CITY and COUNTRY tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The CITY and COUNTRY tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

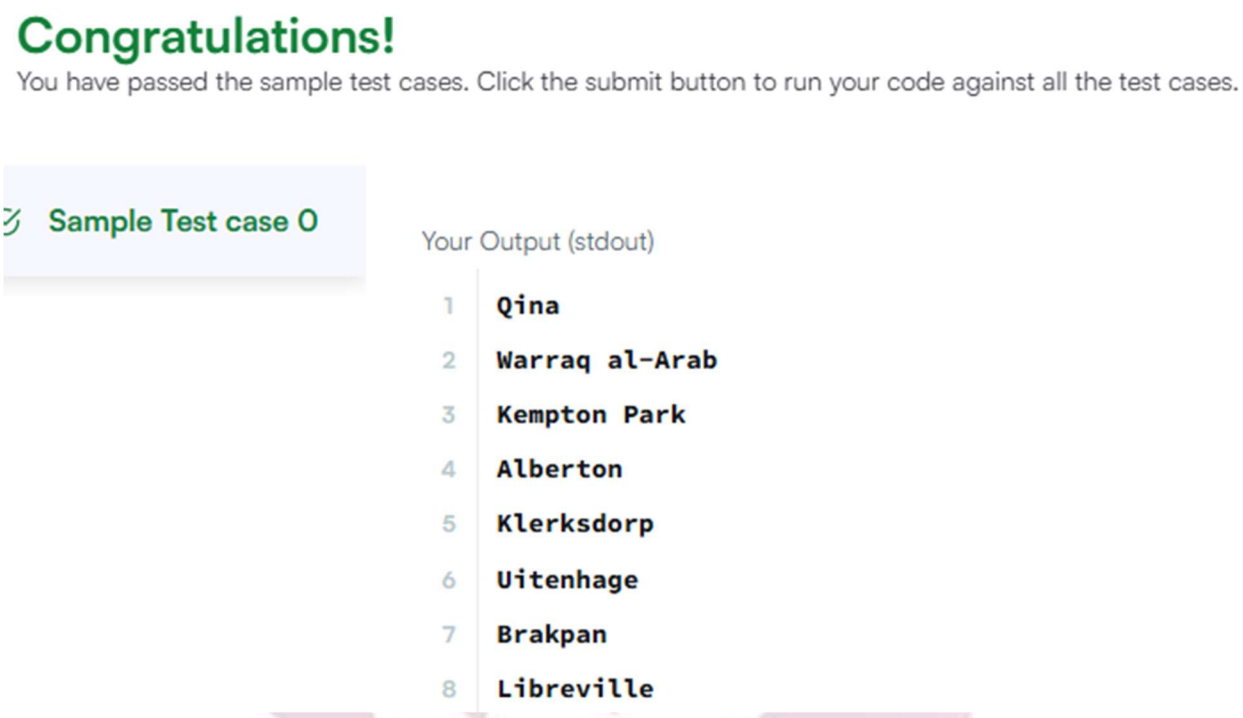
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

INPUT

```
SELECT CITY.Name
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Africa';
```

OUTPUT



AVERAGE POPULATION OF EACH CONTINENT

Given the CITY and COUNTRY tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The CITY and COUNTRY tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

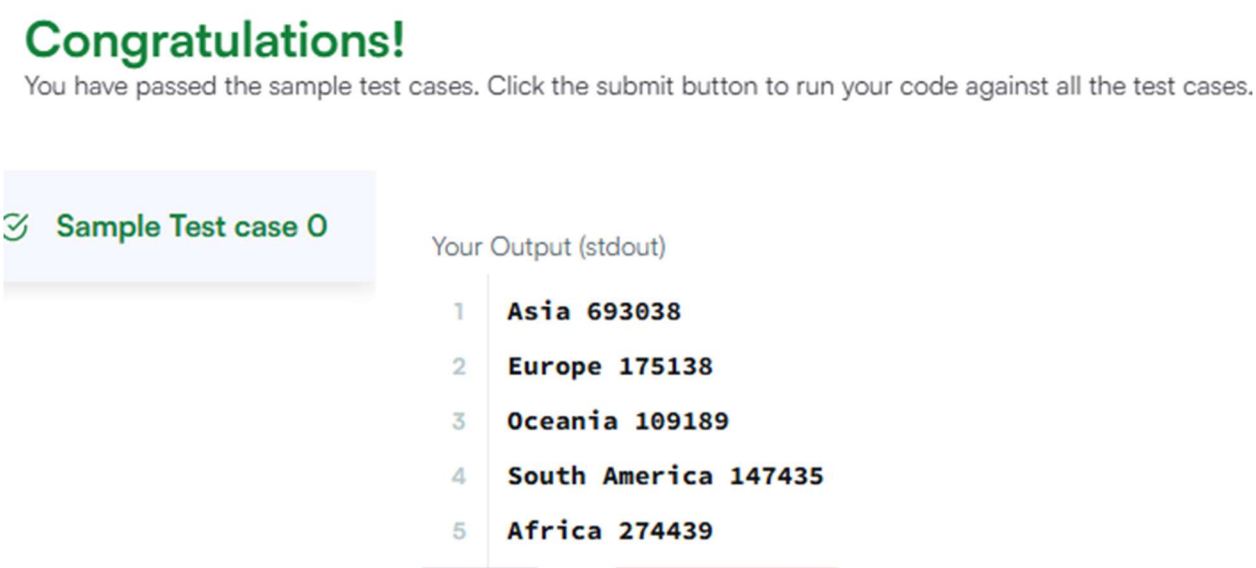
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

INPUT

```
SELECT COUNTRY.Continent,
    FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

OUTPUT



THE REPORT

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

Sample Output

Maria 10 99

Jane 9 81

Julia 9 88

Scarlet 8 78

NULL 7 63

NULL 7 68

Note

Print "NULL" as the name if the grade is less than 8.

Explanation

Consider the following table with the grades assigned to the students:

ID	Name	Marks	Grade
1	Julia	88	9
2	Samantha	68	7
3	Maria	99	10
4	Scarlet	78	8
5	Ashley	63	7
6	Jane	81	9

So, the following students got *8*, *9* or *10* grades:

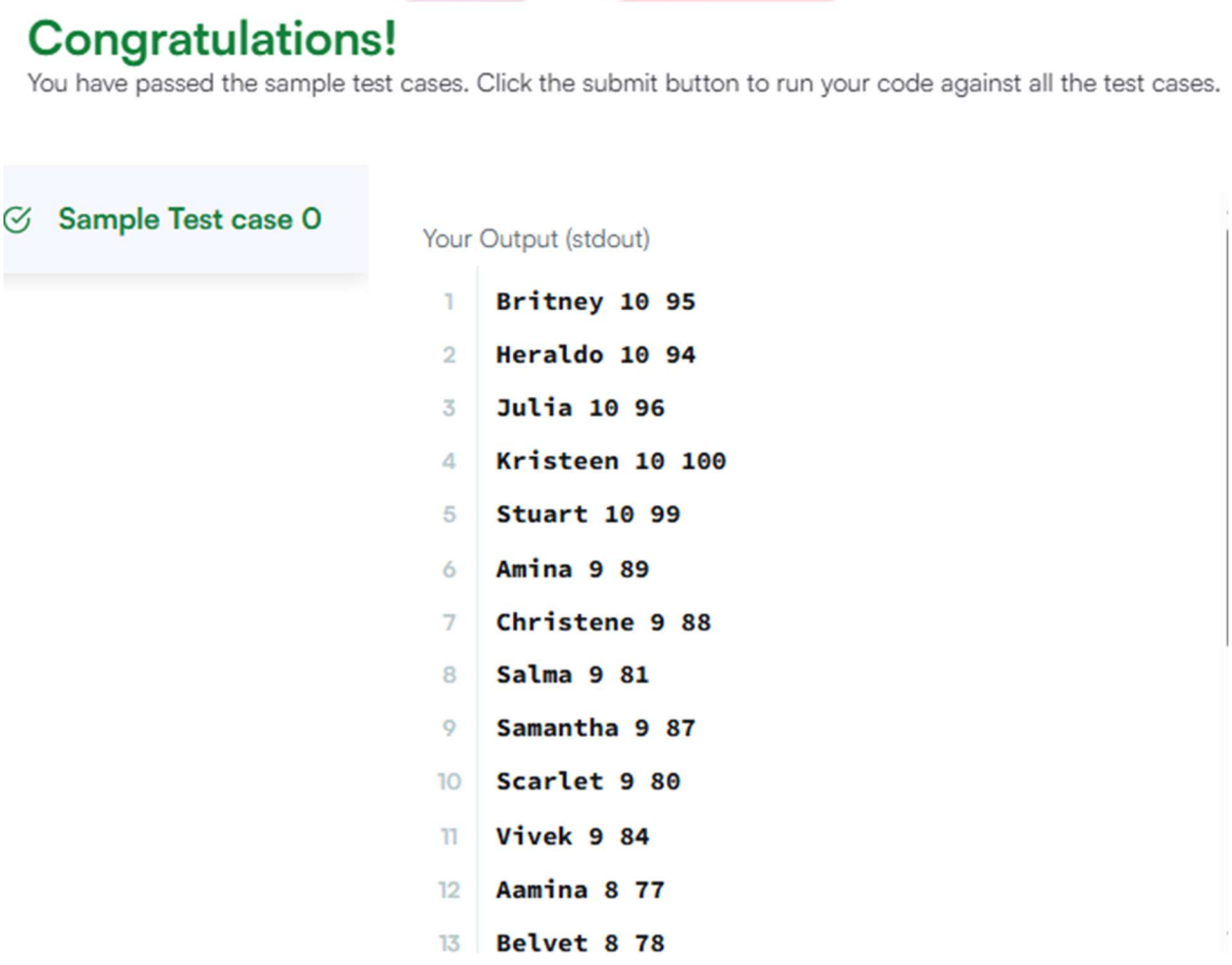
- *Maria (grade 10)*
- *Jane (grade 9)*
- *Julia (grade 9)*
- *Scarlet (grade 8)*

INPUT

```
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
    ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
```

```
ORDER BY
    g.Grade DESC,
    CASE
        WHEN g.Grade >= 8 THEN s.Name
    END ASC,
    CASE
        WHEN g.Grade < 8 THEN s.Marks
    END ASC;
```

OUTPUT



TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge that the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

hacker_id	name
5580	Rose
8439	Angela
27205	Frank
52243	Patrick
52348	Lisa
57645	Kimberly
77726	Bonnie
83082	Michael
86870	Todd
90411	Joe

challenge_id	hacker_id	difficulty_level
4810	77726	4
21089	27205	1
36566	5580	7
66730	52243	6
71055	52243	2

submission_id	hacker_id	challenge_id	score
68628	77726	36566	30
65300	77726	21089	10
40326	52243	36566	77
8941	27205	4810	4
83554	77726	66730	30
43353	52243	66730	0
55385	52348	71055	20
39784	27205	71055	23
94613	86870	71055	30
45788	52348	36566	0
93058	86870	36566	30
7344	8439	66730	92
2721	8439	4810	36
523	5580	71055	4
49105	52348	66730	0
55877	57645	66730	80
38355	27205	66730	35
3924	8439	36566	80
97397	90411	66730	100
84162	83082	4810	40
97431	90411	71055	30

Sample Output

```
90411 Joe
```

Explanation

Hacker *86870* got a score of *30* for challenge *71055* with a difficulty level of *2*, so *86870* earned a full score for this challenge.

Hacker *90411* got a score of *30* for challenge *71055* with a difficulty level of *2*, so *90411* earned a full score for this challenge.

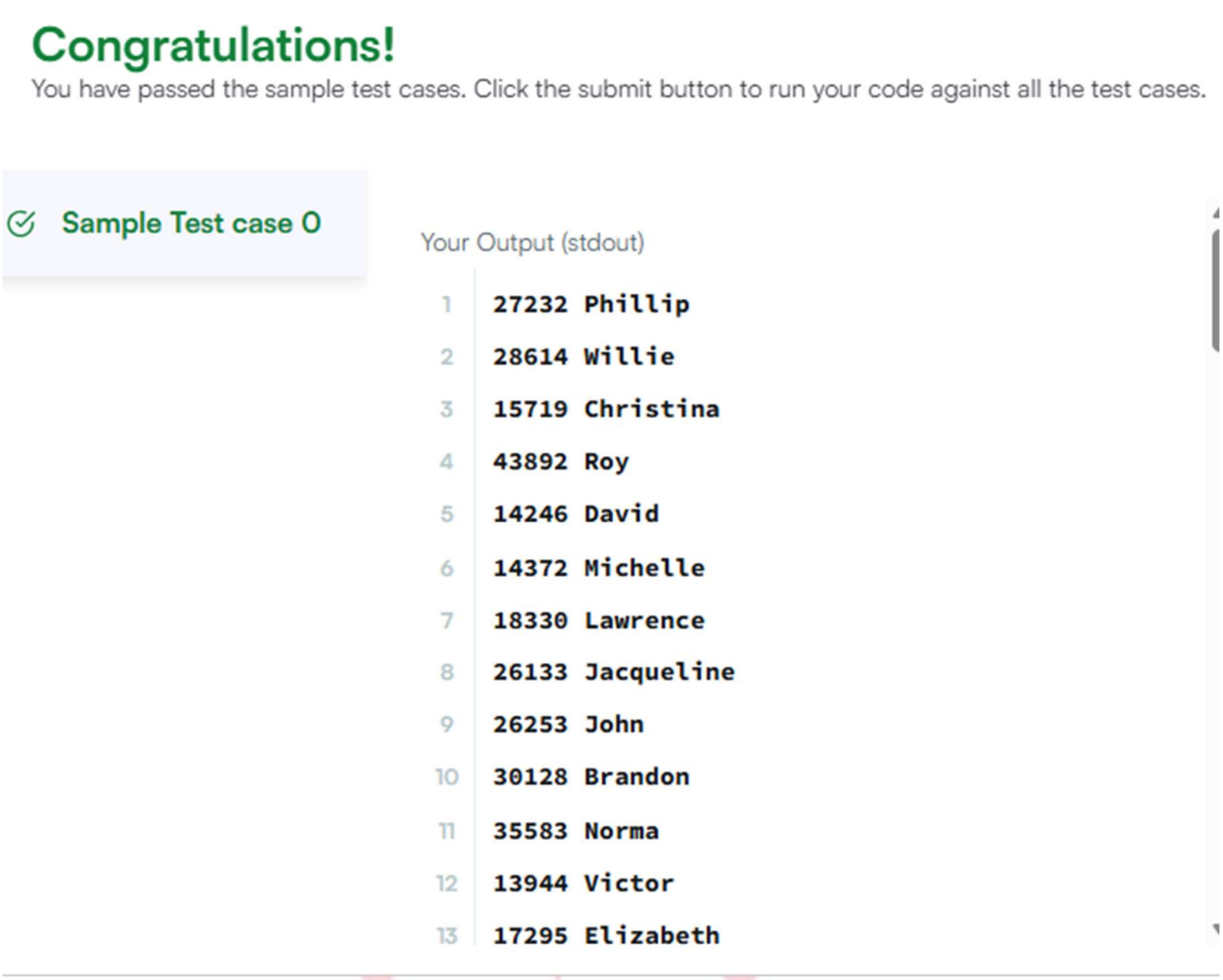
Hacker *90411* got a score of *100* for challenge *66730* with a difficulty level of *6*, so *90411* earned a full score for this challenge.

Only hacker *90411* managed to earn a full score for more than one challenge, so we print the their *hacker_id* and *name* as Values

INPUT

```
SELECT h.hacker_id,
       h.name
FROM Hackers h
JOIN Submissions s
   ON h.hacker_id = s.hacker_id
JOIN Challenges c
   ON s.challenge_id = c.challenge_id
JOIN Difficulty d
   ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1
ORDER BY COUNT(DISTINCT s.challenge_id) DESC,
        h.hacker_id ASC;
```

OUTPUT



OLLIVANDER INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs,

Column	Type
code	Integer
age	Integer
is_evil	Integer

- .

id	code	coins_needed	power
1	4	3688	8
2	3	9365	3
3	3	7187	10
4	3	734	8
5	1	6020	2
6	2	6773	7
7	3	9873	9
8	3	7721	7
9	1	1647	10
10	4	504	5
11	2	7587	5
12	5	9897	10
13	3	4651	8
14	2	5408	1
15	2	6018	7
16	4	7710	5
17	2	8798	7
18	2	3312	3
19	4	7651	6
20	5	5689	3

code	age	is_evil
1	45	0
2	40	0
3	4	1
4	20	0
5	17	0

Sample Output

```
9 45 1647 10
12 17 9897 10
1 20 3688 8
15 40 6018 7
19 20 7651 6
11 40 7587 5
10 20 504 5
18 40 3312 3
20 17 5689 3
5 45 6020 2
14 40 5408 1
```

Explanation

id	age	coins_needed	power
5	45	6020	2
9	45	1647	10

INPUT

SELECT w.id,

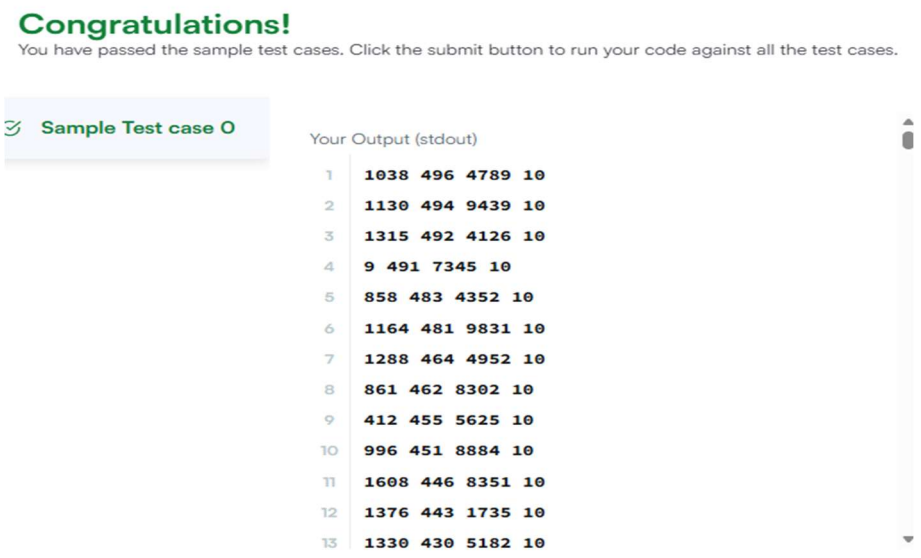
wp.age,

w.coins_needed,

w.power

```
FROM Wands w
JOIN Wands_Property wp
    ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
        SELECT MIN(w2.coins_needed)
        FROM Wands w2
        JOIN Wands_Property wp2
            ON w2.code = wp2.code
        WHERE wp2.is_evil = 0
          AND w2.power = w.power
          AND wp2.age = wp.age
    )
ORDER BY w.power DESC,
         wp.age DESC;
```

OUTPUT



CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the

Column	Type
challenge_id	Integer
hacker_id	Integer

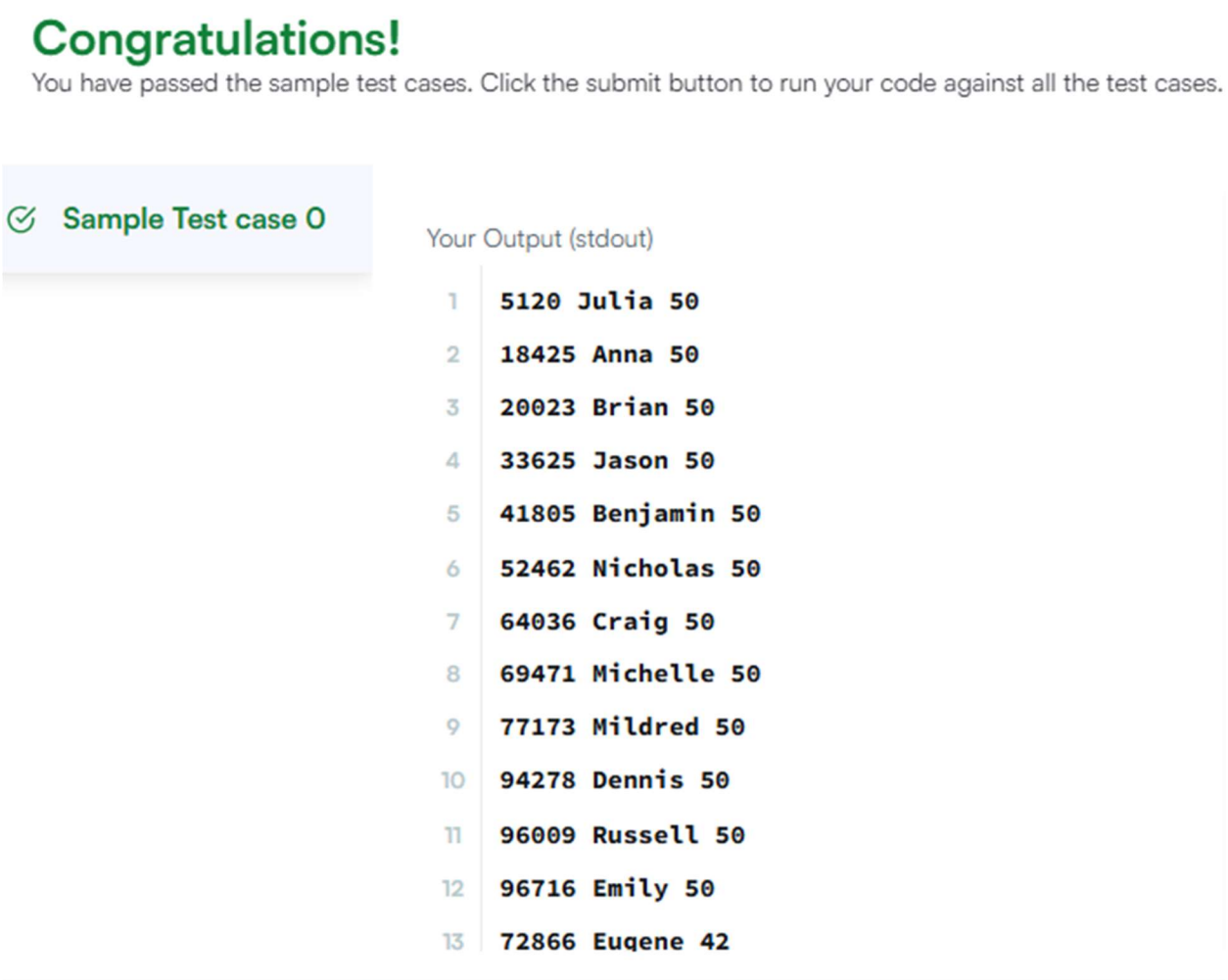
INPUT

```
SELECT h.hacker_id,
    h.name,
    c.challenge_count
FROM Hackers h
JOIN (
  SELECT hacker_id,
      COUNT(*) AS challenge_count
  FROM Challenges
```

```
    GROUP BY hacker_id
) c
ON h.hacker_id = c.hacker_id
WHERE c.challenge_count = (
        SELECT MAX(cnt)
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) t
    )
OR c.challenge_count IN (
        SELECT cnt
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) t
        GROUP BY cnt
        HAVING COUNT(*) = 1
    )
ORDER BY c.challenge_count DESC,
         h.hacker_id ASC;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

OUTPUT



CONTENT LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

Hackers Table:

hacker_id	name
4071	Rose
4806	Angela
26071	Frank
49438	Patrick
74842	Lisa
80305	Kimberly
84072	Bonnie
87868	Michael
92118	Todd
95895	Joe

submission_id	hacker_id	challenge_id	score
67194	74842	63132	76
64479	74842	19797	98
40742	26071	49593	20
17513	4806	49593	32
69846	80305	19797	19
41002	26071	89343	36
52826	49438	49593	9
31093	26071	19797	2
81614	84072	49593	100
44829	26071	89343	17
75147	80305	49593	48
14115	4806	49593	76
6943	4071	19797	95
12855	4806	25917	13
73343	80305	49593	42
84264	84072	63132	0
9951	4071	49593	43
45104	49438	25917	34
53795	74842	19797	5
26363	26071	19797	29
10063	4071	49593	96

Submissions Table:

Sample Output

4071 Rose 191

74842 Lisa 174

84072 Bonnie 100

4806 Angela 89

26071 Frank 85

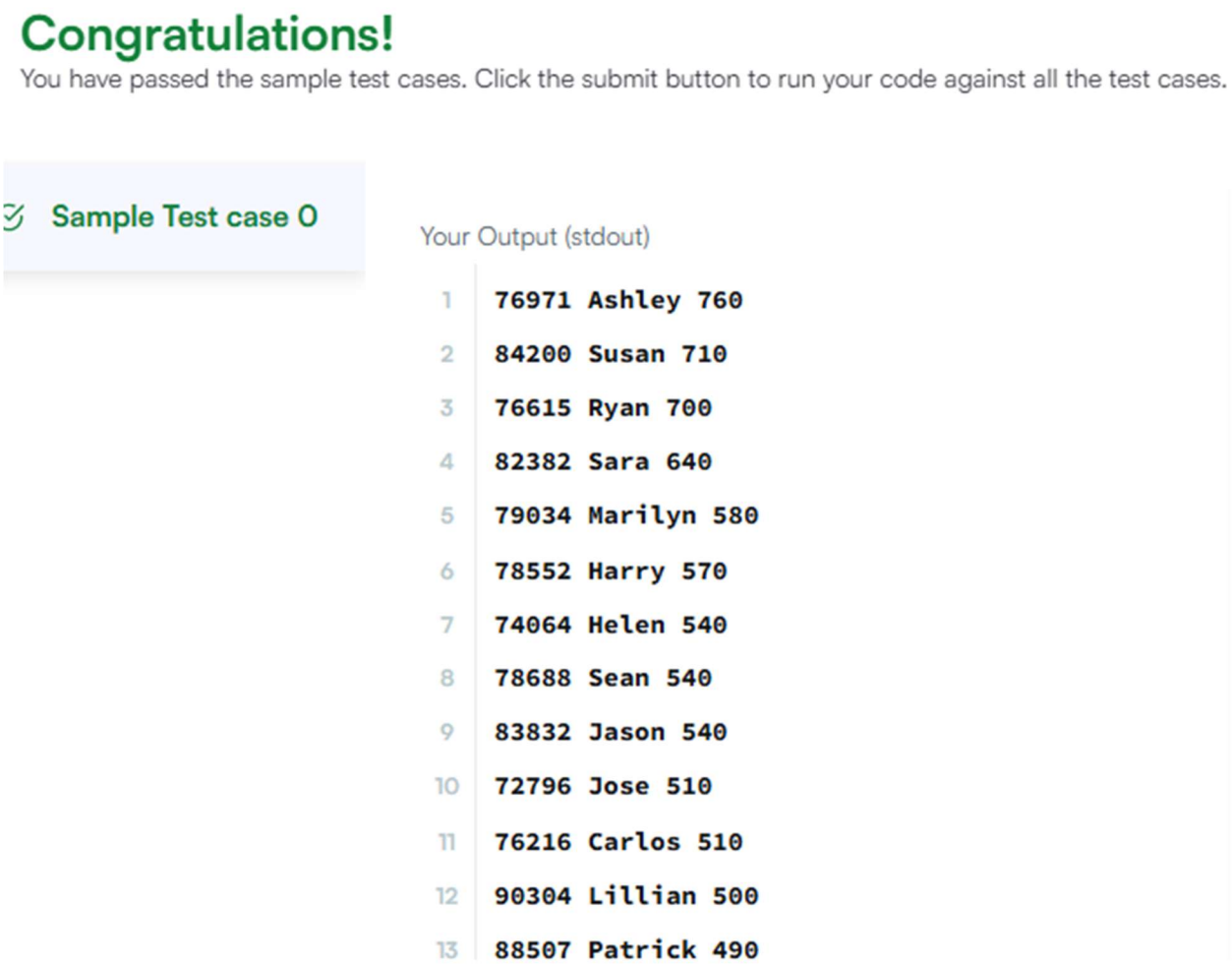
80305 Kimberly 67

49438 Patrick 43

INPUT

```
SELECT h.hacker_id,
       h.name,
       SUM(s.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id,
           challenge_id,
           MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) s
ON h.hacker_id = s.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(s.max_score) > 0
ORDER BY total_score DESC,
         h.hacker_id ASC;
```

OUTPUT



SQL PROJECT PLANNING

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

2015-10-28 2015-10-29

2015-10-30 2015-10-31

2015-10-13 2015-10-15

2015-10-01 2015-10-04

Explanation

The example describes following *four* projects:

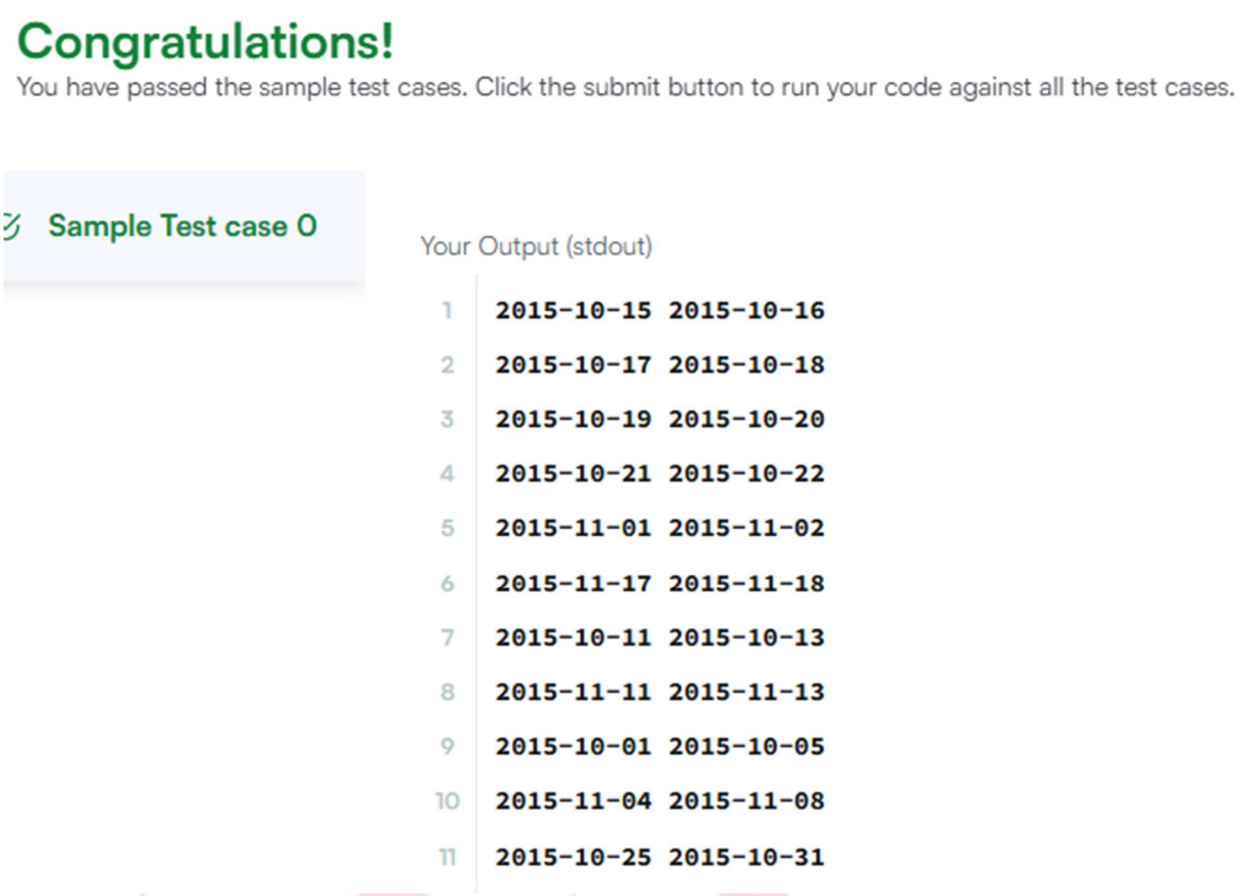
- *Project 1*: Tasks *1*, *2* and *3* are completed on consecutive days, so these are part of the project. Thus start date of project is *2015-10-01* and end date is *2015-10-04*, so it took *3 days* to complete the project.

- *Project 2*: Tasks *4* and *5* are completed on consecutive days, so these are part of the project. Thus, the start date of project is *2015-10-13* and end date is *2015-10-15*, so it took *2 days* to complete the project.
- *Project 3*: Only task *6* is part of the project. Thus, the start date of project is *2015-10-28* and end date is *2015-10-29*, so it took *1 day* to complete the project.
- *Project 4*: Only task *7* is part of the project. Thus, the start date of project is *2015-10-30* and end date is *2015-10-31*, so it took *1 day* to complete the project.

INPUT

```
SELECT
   MIN(Start_Date),
   MAX(End_Date)
FROM (
   SELECT
      Start_Date,
      End_Date,
      DATE_SUB(Start_Date, INTERVAL ROW_NUMBER() OVER (ORDER BY Start_Date)
DAY) AS grp
   FROM Projects
) p
GROUP BY grp
ORDER BY DATEDIFF(MAX(End_Date), MIN(Start_Date)),
       MIN(Start_Date);
```

OUTPUT



PLACEMENTS

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

Samantha

Julia

Scarlet

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

The name output, when ordered by the salary offered to their friends, will be:

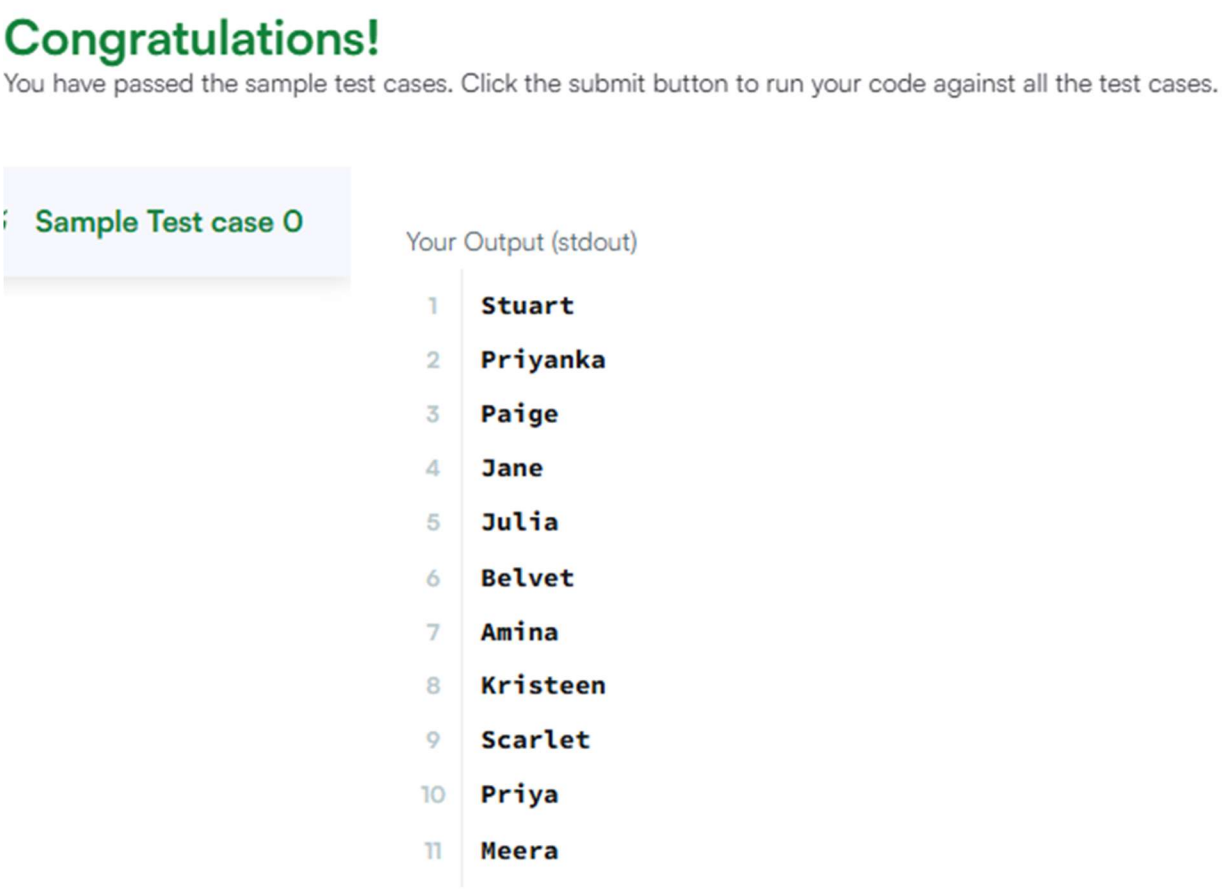
- *Samantha*
- ***Julia***

INPUT

```
SELECT s.Name
FROM Students s
JOIN Friends f
   ON s.ID = f.ID
JOIN Packages p1
   ON s.ID = p1.ID
JOIN Packages p2
   ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
```

ORDER BY p2.Salary;

OUTPUT



SYMMETRIC PAIRS

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Sample Output

20 20

20 21

22 23

INPUT

```
SELECT DISTINCT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
ON f1.X = f2.Y
AND f1.Y = f2.X
WHERE f1.X <= f1.Y
AND (
   f1.X < f1.Y
```

```
    OR (
        f1.X = f1.Y
        AND (
            SELECT COUNT(*)
            FROM Functions
            WHERE X = f1.X AND Y = f1.Y
        ) > 1
    )
)
ORDER BY f1.X;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2 24
4 22
5 21
6 20
8 18
9 17
11 15
13 13
```

INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are 0

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Sample Input

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Sample Output

66406 17973 Rose 111 39 156 56

66556 79153 Angela 0 0 11 10

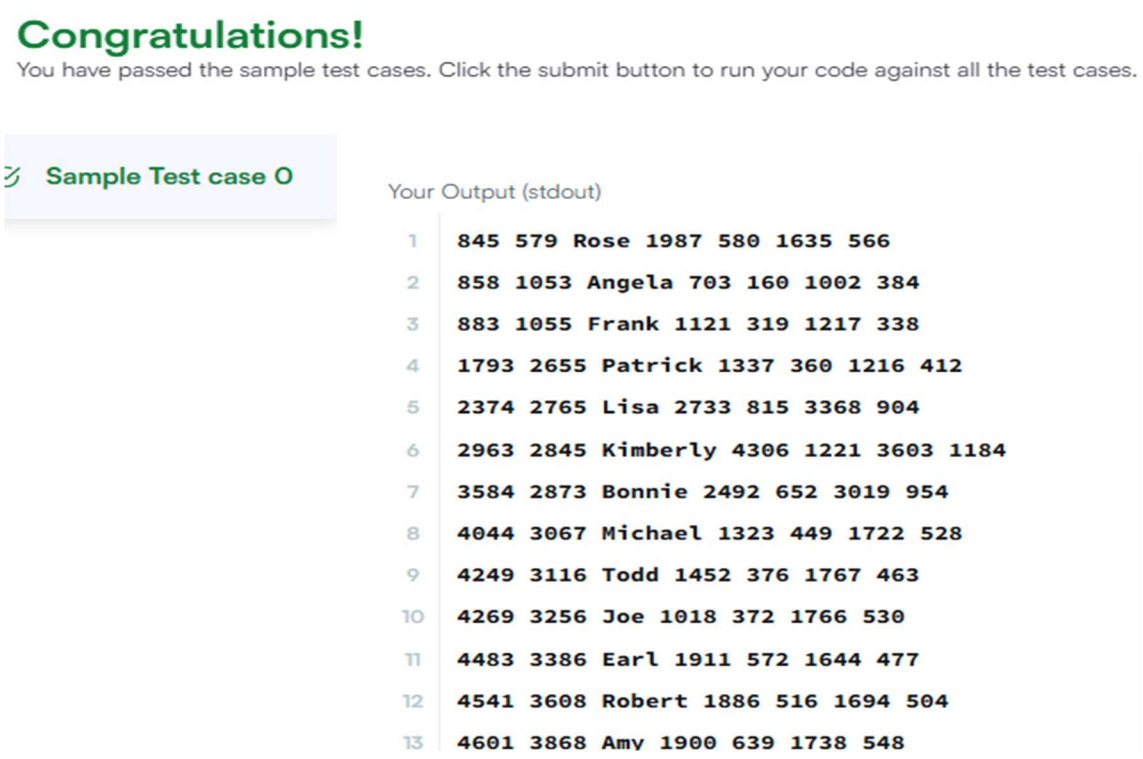
94828 80275 Frank 150 38 41 15

INPUT

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(ss.total_submissions),
    SUM(ss.total_accepted_submissions),
    SUM(vs.total_views),
    SUM(vs.total_unique_views)
FROM Contests c
JOIN Colleges co
    ON c.contest_id = co.contest_id
JOIN Challenges ch
    ON co.college_id = ch.college_id
LEFT JOIN (
    SELECT challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) ss
ON ch.challenge_id = ss.challenge_id
LEFT JOIN (
    SELECT challenge_id,
        SUM(total_views) AS total_views,
```

```
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) vs
ON ch.challenge_id = vs.challenge_id
GROUP BY c.contest_id, c.hacker_id, c.name
HAVING
    SUM(COALESCE(ss.total_submissions, 0)) +
    SUM(COALESCE(ss.total_accepted_submissions, 0)) +
    SUM(COALESCE(vs.total_views, 0)) +
    SUM(COALESCE(vs.total_unique_views, 0)) > 0
ORDER BY c.contest_id;
```

OUTPUT



15 DAYS OF LEARNING SQL

Julia conducted a of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

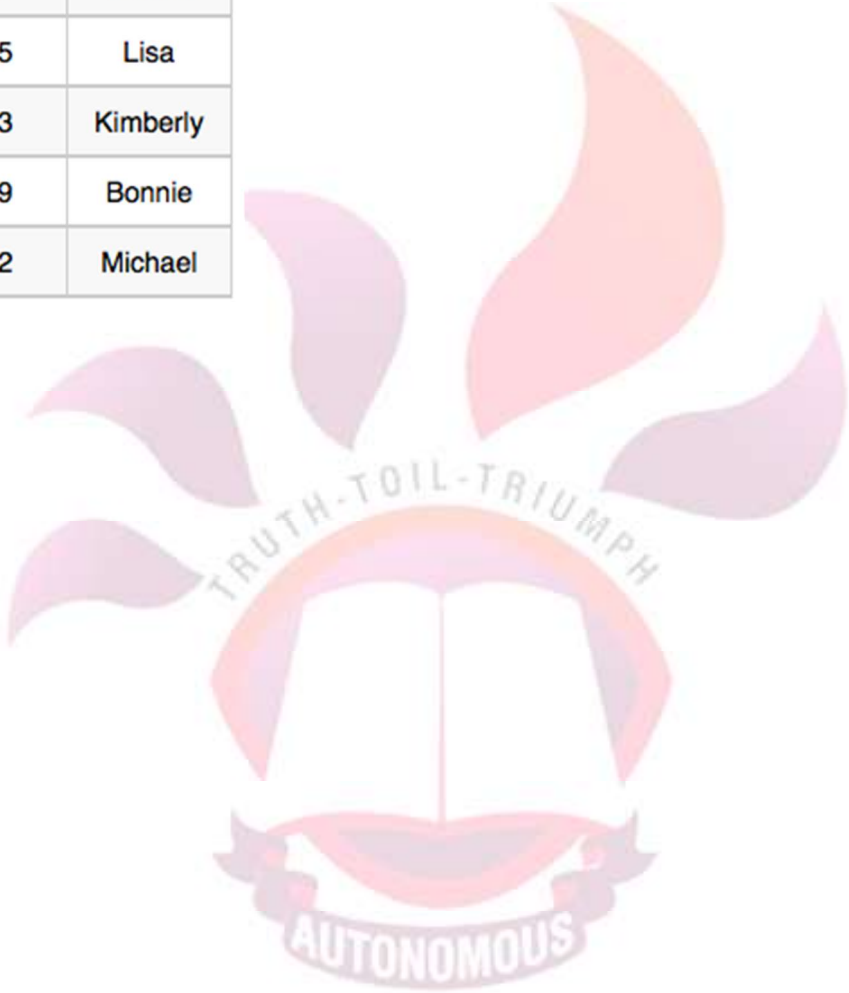
- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael



submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Sample Output

2016-03-01 4 20703 Angela

2016-03-02 2 79722 Michael

2016-03-03 2 20703 Angela

2016-03-04 2 20703 Angela

2016-03-05 1 36396 Frank

2016-03-06 1 20703 Angela

INPUT

```
SELECT
   s.submission_date,

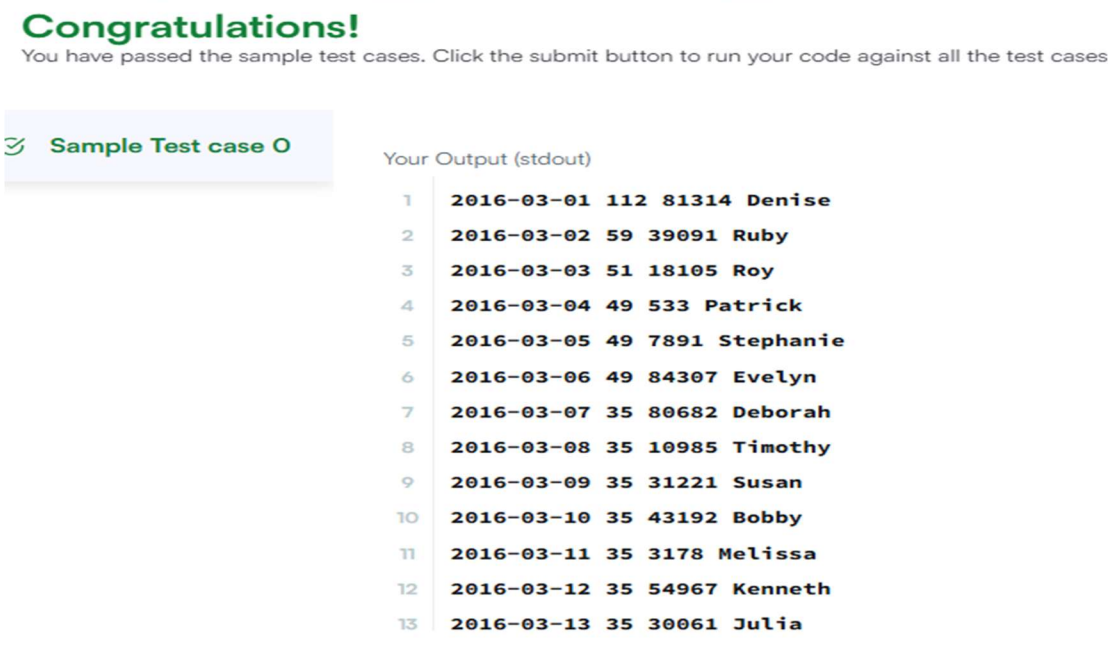
   (
      SELECT COUNT(*)
      FROM (
         SELECT hacker_id
         FROM Submissions s2
         WHERE s2.submission_date <= s.submission_date
         GROUP BY hacker_id
         HAVING COUNT(DISTINCT s2.submission_date) =
               DATEDIFF(s.submission_date, '2016-03-01') + 1
      ) t
   ) AS unique_hackers,

   h.hacker_id,
   h.name

FROM (
```

```
    SELECT submission_date,
        hacker_id,
        ROW_NUMBER() OVER (
            PARTITION BY submission_date
            ORDER BY COUNT(*) DESC, hacker_id ASC
        ) AS rn
    FROM Submissions
    GROUP BY submission_date, hacker_id
) s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.rn = 1
ORDER BY s.submission_date;
```

OUTPUT



DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

* * * * *

* * * *

* * *

* *

*

Write a query to print the pattern *P(20)*.

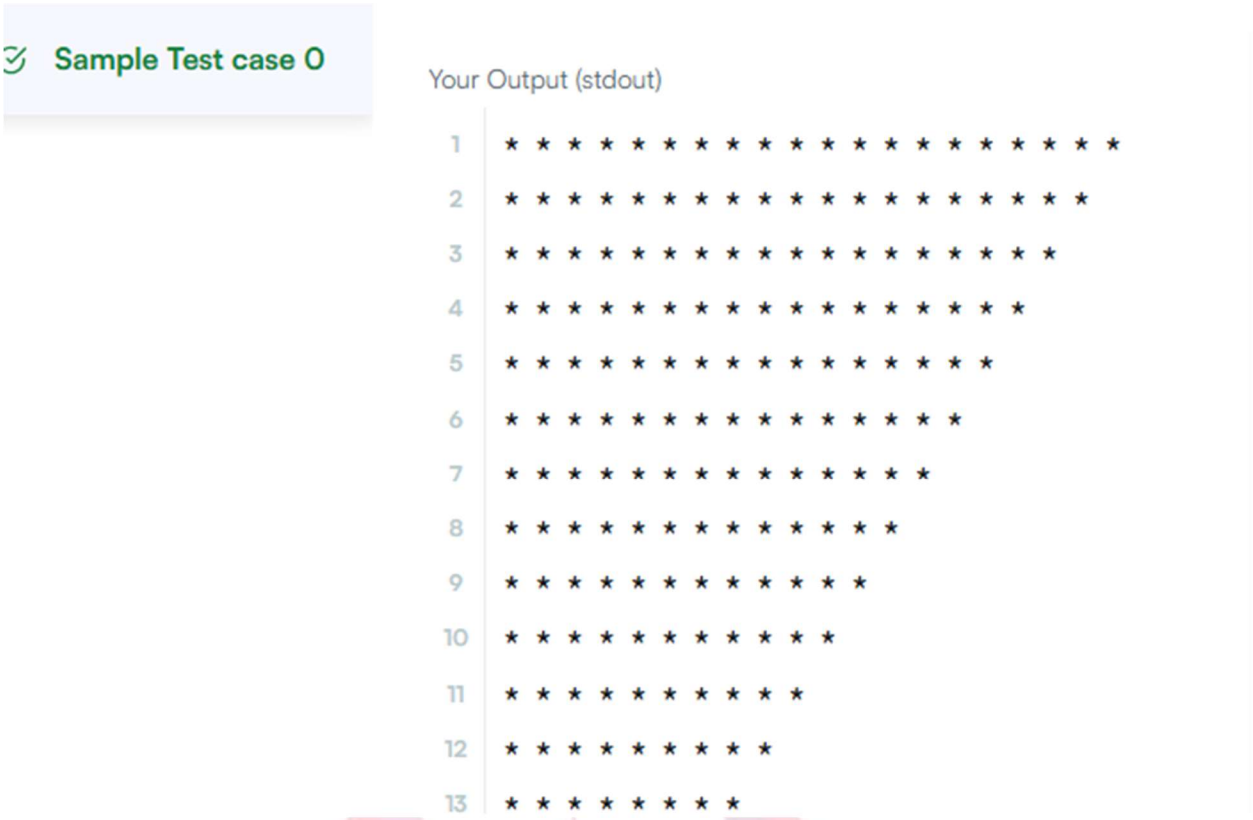
INPUT

```
WITH RECURSIVE pattern AS (
   SELECT 20 AS n
   UNION ALL
   SELECT n - 1
   FROM pattern
   WHERE n > 1
)
SELECT REPEAT('* ', n) AS pattern
FROM pattern;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.



DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

INPUT

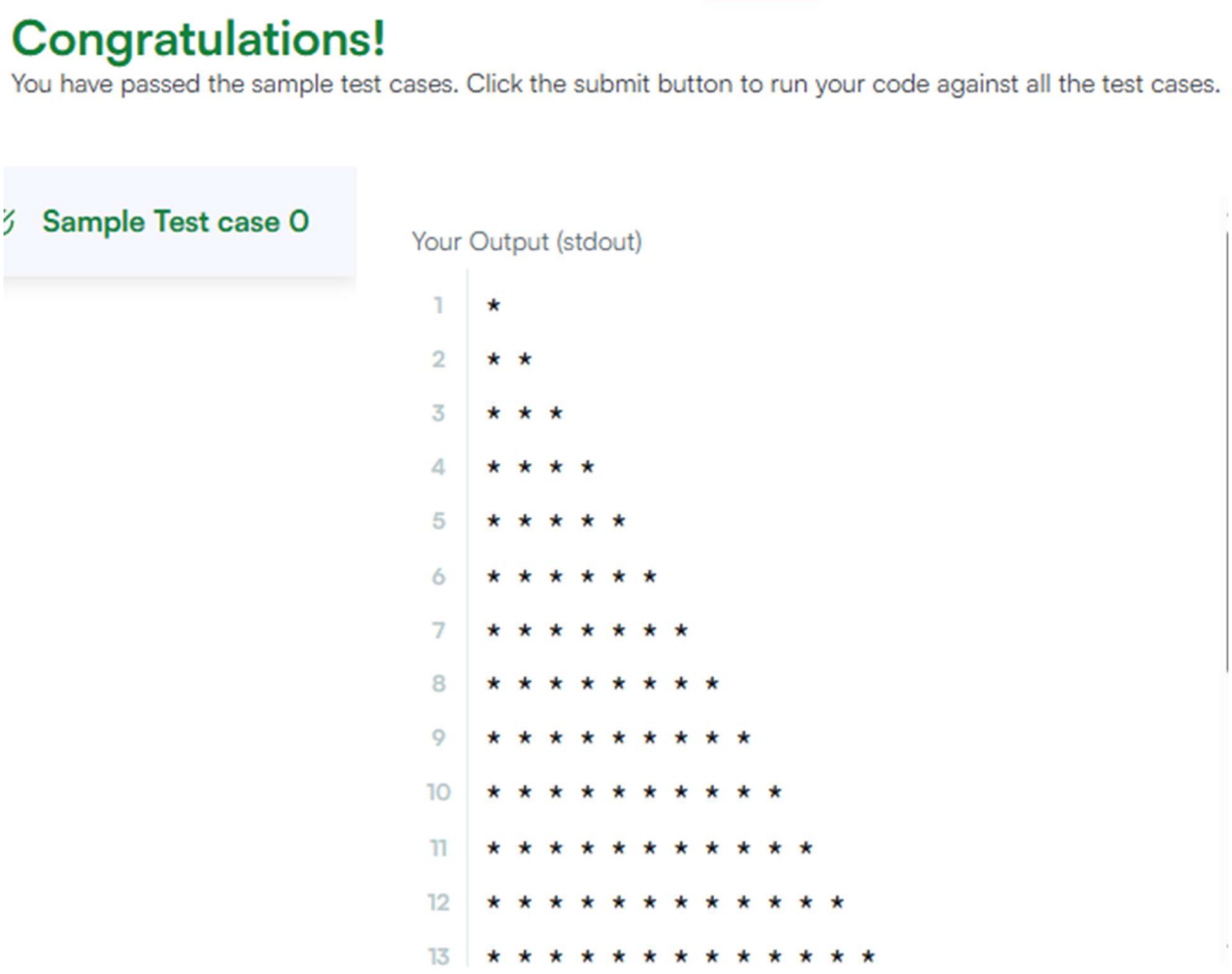
```
SET @row = 0;


SELECT REPEAT('* ', @row := @row + 1)

FROM information_schema.tables

LIMIT 20;
```

OUTPUT



PRINT PRIME NUMBERS

Write a query to print all *prime numbers* less than or equal to 1000 Print your result on a single line, and use the ampersand (&)character as your separator (instead of a space).

For example, the output for all prime numbers

2&3&5&7

INPUT

```
WITH RECURSIVE nums AS (
   SELECT 2 AS n
   UNION ALL
   SELECT n + 1
   FROM nums
   WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums
WHERE NOT EXISTS (
   SELECT 1
   FROM nums d
   WHERE d.n <= SQRT(nums.n)
     AND d.n < nums.n
     AND MOD(nums.n, d.n) = 0
);
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  2&3&5&7&11&13&17&19&23&29&31&37&41&43&47&53&59&
   61&67&71&73&79&83&89&97&101&103&107&109&113&127
   &131&137&139&149&151&157&163&167&173&179&181&19
   1&193&197&199&211&223&227&229&233&239&241&251&2
   57&263&269&271&277&281&283&293&307&311&313&317&
   331&337&347&349&353&359&367&373&379&383&389&397
   &401&409&419&421&431&433&439&443&449&457&461&46
   3&467&479&487&491&499&503&509&521&523&541&547&5
   57&563&569&571&577&587&593&599&601&607&613&617&
   619&631&641&643&647&653&659&661&673&677&683&691
   &701&709&719&727&733&739&743&751&757&761&769&77
   3&787&797&809&811&821&823&827&829&839&853&857&8
   59&863&877&881&883&887&907&911&919&929&937&941&
   947&953&967&971&977&983&991&997
```

CODE CHEF

All products

Question

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics

product_id	product_name	price	category
10	Water Bottle	12.99	Stationery

Input

SELECT *

FROM Products;

Output

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✔ Well done, it's correct! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

High Price of Products

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Expected columns

- product_name
- category

Input

```
SELECT product_name, category
FROM Products
WHERE price > 100.00;
```

Output

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

✔ Great job, keep it up! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Average Salary

Task

Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected column

- avg_salary

Input

SELECT AVG(salary) AS avg_salary

FROM works;

OUTPUT

Your Output

```
┌─────────────┐
│ avg_salary  │
├─────────────┤
│ 62600.155   │
└─────────────┘
```

✔ Perfect answer! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Locate People

Task

Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: departments

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco

department_id	department_name	location
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

Expected column

- department_name
- location

INPUT

```
SELECT department_name, location
FROM departments
WHERE location LIKE 'S%';
```

OUTPUT

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

✔ Hooray, you did it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Distinct Companies

Task

Write a query to select all the distinct companies (company_name) in the Works table.
Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50

employee_id	company_name	salary
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column

- company_name

INPUT

```
SELECT DISTINCT company_name
FROM works;
```

OUTPUT

```
| company_name |
├──────────────┤
| TechCorp     |
| InnovateLtd  |
| HealthPlus   |
| EduWorks     |
| GreenTech    |
└──────────────┘
```

✔ Awesome, you nailed it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Fiction Collection Size

Task

Write a query to find the total count of books whose genre is Fiction.
Note: Output column name should be fiction_count.

Table name: Books

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925

id	title	author	genre	price	published_year
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950

id	title	author	genre	price	published_year
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected column

- fiction_count

INPUT

```
SELECT COUNT(*) AS fiction_count
FROM Books
WHERE genre = 'Fiction';
```

OUTPUT

Your Output

fiction_count
7

Great job, keep it up! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

List of Movies with Ratings

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: Cinema

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

Expected column

- Movie_name

INPUT

SELECT Movie_name

FROM Cinema

WHERE Rating > 7 AND Rating < 9;

OUTPUT

Your Output

```
┌──────────────┐
│ Movie_name   │
├──────────────┤
│ War          │
│ Science      │
│ Ice Song     │
│ The Escape   │
│ Solar Flare  │
│ Lost Dreams  │
│ Galaxy War   │
```

✔ You got it right! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Handling NULL Values

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

book_idtitle author published_yearrating

1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2

book_id	title	author	published_year	rating
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

Expected column

- book_id
- title
- author
- published_year

INPUT

```
SELECT book_id, title, author, published_year
FROM Library
WHERE rating IS NULL;
```

OUTPUT

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

✔ Well done, it's correct! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Salary of Employees

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in descending order

Table name: Employees

employee_idemployee_namecompany category department salary

1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000

employee_idemployee_namecompany category department salary

4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected column

- employee_name
- company
- salary

INPUT

SELECT employee_name, company, salary

FROM employees

WHERE category = 'Full Time'

ORDER BY salary DESC;

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✔ You got it right! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Department of Each Employee

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_idemployee_namecompany category department salary

1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000

employee_id	employee_name	company	category	department	salary
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Column

- department
- total_employees

INPUT

```
SELECT department, COUNT(*) AS total_employees
FROM Employees
GROUP BY department;
```

OUTPUT

Your Output

```
┌──────────────┬──────────────────┐
│ department   │ total_employees  │
├──────────────┼──────────────────┤
│ Engineering  │ 2                │
│ Finance      │ 2                │
│ HR           │ 2                │
│ IT           │ 1                │
│ Marketing    │ 2                │
│ Sales        │ 1                │
└──────────────┴──────────────────┘
```

Great job, keep it up! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

Article views

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in ascending order.

Table name: Views

article_id author_id author_name viewer_id view_count publication_name

101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News

article_id	author_id	author_name	viewer_id	view_count	publication_name
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected column

- author_id
- author_name
- publication_name

INPUT

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

OUTPUT

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

Awesome, you nailed it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		Result - Correct

Player Performance Insights

Task

Write a query to find the names of the top 3 distinct players by highest score who have won matches, including their scores.

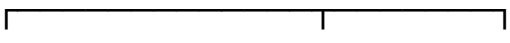
Table 1: Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: Matches

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

Expected Output Columns:



player_name	score
David	1600
Bob	1500
Charlie	1300

INPUT

```
SELECT DISTINCT p.player_name, p.score
FROM Players p
JOIN Matches m
    ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Hooray, you did it! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)
Subtask Score: 100%		**Result** - Correct
		Total Score = 100%

Player Details

Task

Write a query to retrieve the details of the last five matches played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named Players and Matches.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

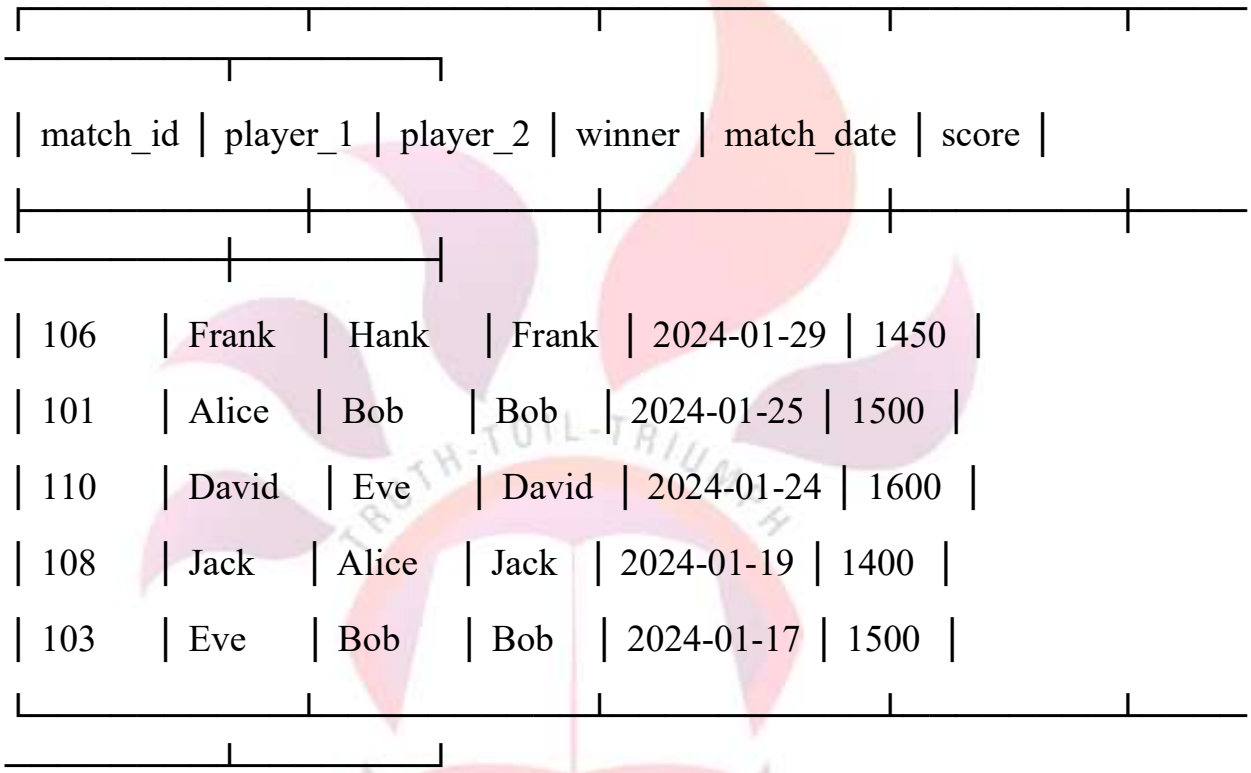
match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

- match_id
- player_1

- player_2
- winner
- match_date
- score

Expected Output:



match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

INPUT

```
SELECT
   m.match_id,
   m.player_1,
   m.player_2,
   m.winner,
   m.match_date,
```

```
    p.score
FROM Matches m
JOIN Players p
    ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

✔ You got it right! AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.00)

SQL PRATICE PROGRAMS

QUESTION:1

```
CREATE TABLE CABHUB
(
   VehicleNo NUMBER PRIMARY KEY,
   VehicleName VARCHAR2(30),
   Make VARCHAR2(30),
   Color VARCHAR2(20),
   Capacity NUMBER,
   Charges NUMBER
);
INSERT INTO CABHUB VALUES (101, 'Swift', 'Maruti', 'White', 4, 1200);
INSERT INTO CABHUB VALUES (102, 'Innova', 'Toyota', 'Black', 7, 2500);
INSERT INTO CABHUB VALUES (103, 'Verna', 'Hyundai', 'White', 5, 1800);
INSERT INTO CABHUB VALUES (104, 'City', 'Honda', 'Red', 4, 2000);
INSERT INTO CABHUB VALUES (105, 'Ertiga', 'Maruti', 'White', 6, 2200);
COMMIT;
CREATE TABLE CUSTOMER
(
   CustomerID NUMBER PRIMARY KEY,
   CustomerName VARCHAR2(30),
   VehicleNo NUMBER
);
INSERT INTO CUSTOMER VALUES (1, 'Arun', 101);
INSERT INTO CUSTOMER VALUES (2, 'Meena', 103);
```

INSERT INTO CUSTOMER VALUES (3, 'Kavin', 104);

INSERT INTO CUSTOMER VALUES (4, 'Divya', 102);

INSERT INTO CUSTOMER VALUES (5, 'Priya', 105);

COMMIT;

QUERY 1

SELECT VehicleName

FROM CABHUB

WHERE Color = 'White';

Results Explain Describe Saved SQL History

VEHICLENAME
Maruti Swift
Toyota Innova
Tata Nexon
Maruti Swift
Toyota Innova

5 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT VehicleName, Capacity

FROM CABHUB

ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	CAPACITY
Indigo	3
Honda City	4
Maruti Swift	4
Maruti Swift	4
Honda City	4
Hyundai i20	5
Tata Nexon	5
Hyundai i20	5
Innova	7
Toyota Innova	7
More than 10 rows available. Increase rows selector to view more rows.	

10 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT MAX(Charges) AS Highest_Charges
FROM CABHUB;
```

Results Explain Describe Saved SQL History

HIGHEST_CHARGES
3000

1 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT COUNT(DISTINCT Make) AS Number_Of_Makes
FROM CABHUB;
```

Results Explain Describe Saved SQL History

NUMBER_OF_MAKES
5

1 rows returned in 0.02 seconds CSV Export

QUERY 5

```
SELECT MAX(Charges) AS Maximum_Charges,
    MIN(Charges) AS Minimum_Charges
FROM CABHUB;
```

Results Explain Describe Saved SQL History

MAXIMUM_CHARGES	MINIMUM_CHARGES
3000	12

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT Make, COUNT(*) AS Vehicle_Count
FROM CABHUB
GROUP BY Make;
```

Results Explain Describe Saved SQL History

MAKE	VEHICLE_COUNT
Hyundai	2
Toyota	4
Maruti	2
Honda	2
Tata	2

5 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT VehicleName
FROM CABHUB
WHERE Capacity = 4;
```

Results Explain Describe Saved SQL History

VEHICLENAME
Maruti Swift
Honda City
Maruti Swift
Honda City

4 rows returned in 0.00 seconds CSV Export

QUESTION 2

```
CREATE TABLE CO2_EMISSION
(
   VehicleID NUMBER PRIMARY KEY,
   Company VARCHAR2(30),
   Model VARCHAR2(30),
   FuelType VARCHAR2(20),
   EngineSize NUMBER(3,1),
   CO2Emission NUMBER
);
```

```
INSERT INTO CO2_EMISSION VALUES
(101, 'Toyota', 'Etios', 'Diesel', 1.4, 120);

INSERT INTO CO2_EMISSION VALUES
(102, 'Honda', 'City', 'Petrol', 1.5, 140);

INSERT INTO CO2_EMISSION VALUES
(103, 'Hyundai', 'Creta', 'Diesel', 1.6, 155);

INSERT INTO CO2_EMISSION VALUES
(104, 'Maruti', 'Swift', 'Petrol', 1.2, 110);

INSERT INTO CO2_EMISSION VALUES
(105, 'Kia', 'Seltos', 'Diesel', 1.5, 145);

INSERT INTO CO2_EMISSION VALUES
(106, 'Tata', 'Nexon', 'Petrol', 1.2, 130);

INSERT INTO CO2_EMISSION VALUES
(107, 'Mahindra', 'XUV500', 'Diesel', 2.0, 180);

COMMIT;
QUERY 1
SELECT *
FROM CO2_EMISSION
```

WHERE FuelType = 'Petrol';

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
108	MG	Hector	Petrol	1.5	160

2 rows returned in 0.03 seconds CSV Export

QUERY 2

SELECT Company, Model

FROM CO2_EMISSION

WHERE EngineSize > 1.5;

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla

1 rows returned in 0.01 seconds CSV Export

QUERY 3

SELECT Company, Model, CO2Emission

FROM CO2_EMISSION

ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
MG	Hector	160
Toyota	Corolla	165

2 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT *

FROM CO2_EMISSION

WHERE CO2Emission > 150;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
108	MG	Hector	Petrol	1.5	160

2 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*) AS Diesel_Vehicles
FROM CO2_EMISSION
WHERE FuelType = 'Diesel';
```

Results Explain Describe Saved SQL History

DIESEL_VEHICLES
0

1 rows returned in 0.02 seconds CSV Export

QUERY 6

```
INSERT INTO CO2_EMISSION
VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);
COMMIT;
DESC CO2_EMISSION;
```

Results Explain Describe Saved SQL History

Object Type TABLE Object CO2_EMISSION

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CO2_EMISSION	VEHICLEID	Number	-	-	-	-	✓	-	-
	COMPANY	Varchar2	20	-	-	-	✓	-	-
	MODEL	Varchar2	20	-	-	-	✓	-	-
	FUELTYPE	Varchar2	10	-	-	-	✓	-	-
	ENGINESIZE	Number	-	3	1	-	✓	-	-
	CO2EMISSION	Number	-	-	-	-	✓	-	-

1 - 6

QUERY 7

(i)SELECT MAX(CO2Emission)

FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MAX(CO2EMISSION)
165

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MIN(EngineSize)

FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MIN(ENGINESIZE)
1.5

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT AVG(CO2Emission)

FROM CO2_EMISSION

WHERE FuelType = 'Petrol';

Results Explain Describe Saved SQL History

AVG(CO2EMISSION)
161.66666666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT COUNT(*)

FROM CO2_EMISSION

WHERE EngineSize >= 1.5;

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION 4

```
CREATE TABLE PRODUCTS (
   PID NUMBER PRIMARY KEY,
   PNAME VARCHAR2(50),
   QTY NUMBER,
   PRICE NUMBER,
   COMPANY VARCHAR2(50),
   SUPCODE VARCHAR2(5)
);
CREATE TABLE SUPPLIERS (
   SUPCODE VARCHAR2(5) PRIMARY KEY,
   SNAME VARCHAR2(50),
   CITY VARCHAR2(30)
);
INSERT INTO PRODUCTS VALUES
(101, 'DIGITAL CAMERA 14X', 120, 12000, 'RENIX', 'S01');

INSERT INTO PRODUCTS VALUES
(102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP', 'S02');

INSERT INTO PRODUCTS VALUES
```

(104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', 'S01');

INSERT INTO PRODUCTS VALUES

(106, 'LED SCREEN 32', 70, 28000, 'DISEXPERTS', 'S02');

INSERT INTO PRODUCTS VALUES

(105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON', 'S03');

COMMIT;

INSERT INTO SUPPLIERS VALUES

('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS VALUES

('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS VALUES

('S02', 'DIGI BUSY GROUP', 'CHENNAI');

COMMIT;

QUERY 1

SELECT *

FROM PRODUCTS

ORDER BY PNAME ASC;

Results Explain Describe Saved SQL History

PID	PNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA 14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
106	LED SCREEN 32	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

5 rows returned in 0.02 seconds CSV Export

QUERY 2

```
SELECT PNAME, PRICE
FROM PRODUCTS
WHERE PRICE BETWEEN 10000 AND 15000;
```

Results Explain Describe Saved SQL History

PNAME	PRICE
DIGITAL CAMERA 14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.02 seconds CSV Export

QUERY 3

```
SELECT SUPCODE, COUNT(*)
FROM PRODUCTS
GROUP BY SUPCODE
ORDER BY SUPCODE
```

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT PRICE, PNAME, QTY
FROM PRODUCTS
WHERE QTY > 100;
```

Results Explain Describe Saved SQL History

PRICE	PNAME	QTY
12000	DIGITAL CAMERA 14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.02 seconds CSV Export

QUERY 5

```
SELECT COMPANY, PNAME
FROM PRODUCTS
ORDER BY COMPANY DESC;
```

Results Explain Describe Saved SQL History

COMPANY	PNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA 14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN 32
DIGI POP	DIGITAL PAD 11i

5 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT DISTINCT SUPCODE
FROM PRODUCTS;
```

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.01 seconds CSV Export

QUERY 7

```
SELECT MAX(PRICE), MIN(PRICE)
FROM PRODUCTS;
```

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT PRICE * QTY AS AMOUNT
FROM PRODUCTS
WHERE PID = 104;
```

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

QUESTION 5

```
CREATE TABLE SHOPPE (
   ID VARCHAR2(5),
   SNAME VARCHAR2(50),
   AREA VARCHAR2(30)
);

CREATE TABLE ACCESSORIES (
   NO VARCHAR2(5),
   NAME VARCHAR2(50),
```

```
    PRICE NUMBER,
    SNO VARCHAR2(5)
);
INSERT INTO SHOPPE VALUES ('S01', 'ABC Computeronics', 'CP');
INSERT INTO SHOPPE VALUES ('S02', 'All Infotech Media', 'GK II');
INSERT INTO SHOPPE VALUES ('S03', 'Tech Shoppe', 'CP');
INSERT INTO SHOPPE VALUES ('S04', 'Geeks Tecno Soft', 'Nehru Place');
INSERT INTO SHOPPE VALUES ('S05', 'Hitech Tech Store', 'Nehru Place');

INSERT INTO ACCESSORIES VALUES ('A01', 'Mother Board', 12000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A02', 'Hard Disk', 5000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A03', 'Keyboard', 500, 'S02');
INSERT INTO ACCESSORIES VALUES ('A04', 'Mouse', 300, 'S01');
INSERT INTO ACCESSORIES VALUES ('A05', 'Mother Board', 13000, 'S02');
INSERT INTO ACCESSORIES VALUES ('A06', 'Keyboard', 400, 'S03');
INSERT INTO ACCESSORIES VALUES ('A07', 'LCD', 6000, 'S04');
INSERT INTO ACCESSORIES VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ACCESSORIES VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ACCESSORIES VALUES ('T10', 'Hard Disk', 4500, 'S03');

COMMIT;
QUERY 1
SELECT NAME, PRICE
FROM ACCESSORIES
ORDER BY PRICE ASC
```

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

QUERY 2

```
SELECT ID, SNAME
FROM SHOPPE
WHERE AREA = 'Nehru Place'
```

Results Explain Describe Saved SQL History

ID	SNAME
S04	Geeks Tecno Soft
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT NAME, MIN(PRICE) AS MINIMUM_PRICE,
    MAX(PRICE) AS MAXIMUM_PRICE
FROM ACCESSORIES
GROUP BY NAME
```

Results Explain Describe Saved SQL History

NAME	MINIMUM_PRICE	MAXIMUM_PRICE
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

QYERY 4

```
SELECT A.NAME, A.PRICE, S.SNAME
FROM ACCESSORIES A, SHOPPE S
WHERE A.SNO = S.ID
```

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All Infotech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All Infotech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Tecno Soft
LCD	5500	Hitech Tech Store
Hard Disk	4500	Tech Shoppe
Mouse	350	Hitech Tech Store

10 rows returned in 0.01 seconds CSV Export

QUERY 5

```
SELECT DISTINCT NAME
FROM ACCESSORIES
WHERE PRICE >= 5000
```

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT AREA, COUNT(*)

FROM SHOPPE

GROUP BY AREA

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(DISTINCT AREA)

FROM SHOPPE

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT NAME, PRICE * 0.05 AS DISCOUNT

FROM ACCESSORIES

WHERE SNO IN ('S02', 'S03')

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

QUESTION 6

```
CREATE TABLE HOSPITAL (
   NO NUMBER,
   NAME VARCHAR2(30),
   AGE NUMBER,
   DEPARTMENT VARCHAR2(30),
   DATEOFADM DATE,
   CHARGES NUMBER,
   SEX CHAR(1)
);
INSERT INTO HOSPITAL VALUES
(1, 'Arpit', 62, 'Surgery', TO_DATE('21/01/98','DD/MM/RR'), 300, 'M');

INSERT INTO HOSPITAL VALUES
(2, 'Zareena', 22, 'ENT', TO_DATE('12/12/97','DD/MM/RR'), 250, 'F');

INSERT INTO HOSPITAL VALUES
(3, 'Kareem', 32, 'Orthopedic', TO_DATE('19/02/98','DD/MM/RR'), 200, 'M');

INSERT INTO HOSPITAL VALUES
```

(4, 'Arun', 12, 'Surgery', TO_DATE('11/01/98','DD/MM/RR'), 300, 'M');

INSERT INTO HOSPITAL VALUES

(5, 'Zubin', 30, 'ENT', TO_DATE('12/01/98','DD/MM/RR'), 250, 'M');

INSERT INTO HOSPITAL VALUES

(6, 'Ketika', 16, 'ENT', TO_DATE('24/02/98','DD/MM/RR'), 250, 'F');

INSERT INTO HOSPITAL VALUES

(7, 'Ankita', 29, 'Cardiology', TO_DATE('20/02/98','DD/MM/RR'), 800, 'F');

INSERT INTO HOSPITAL VALUES

(8, 'Zareen', 45, 'Gynecology', TO_DATE('22/02/98','DD/MM/RR'), 300, 'F');

INSERT INTO HOSPITAL VALUES

(9, 'Kush', 19, 'Cardiology', TO_DATE('13/01/98','DD/MM/RR'), 800, 'M');

INSERT INTO HOSPITAL VALUES

(10, 'Shilpa', 23, 'Nuclear Medicine',

TO_DATE('21/02/98','DD/MM/RR'), 400, 'F');

COMMIT;

QUERY 1

SELECT *

FROM HOSPITAL

WHERE DEPARTMENT = 'Cardiology';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	SEX
7	Ankita	29	Cardiology	20-FEB-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT NAME

FROM HOSPITAL

WHERE DEPARTMENT = 'ENT'

AND SEX = 'F';

Results Explain Describe Saved SQL History

NAME
Zareena
Ketika

2 rows returned in 0.01 seconds CSV Export

QUERY 3

SELECT NAME, DATEOFADM

FROM HOSPITAL

ORDER BY DATEOFADM ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFADM
Zareena	12-DEC-97
Arun	11-JAN-98
Zubin	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Ketika	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT NAME, CHARGES, AGE
FROM HOSPITAL
WHERE SEX = 'F';
```

Results | Explain | Describe | Saved SQL | History

NAME	CHARGES	AGE
Zareena	250	22
Ketika	250	16
Ankita	800	29
Zareen	300	45
Shilpa	400	23

5 rows returned in 0.00 seconds CSV Export

QYERY 5

```
SELECT COUNT(*)
FROM HOSPITAL
WHERE AGE < 30;
```

Results | Explain | Describe | Saved SQL | History

COUNT(*)
6

1 rows returned in 0.02 seconds CSV Export

QUERY 6

```
INSERT INTO HOSPITAL
VALUES
(11, 'Aftab', 24, 'Surgery',
 TO_DATE('25/02/98','DD/MM/RR'),
 300, 'M');
```

COMMIT;

SELECT * FROM HOSPITAL;

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	SEX
1	Arpit	62	Surgery	21-JAN-98	300	M
2	Zareena	22	ENT	12-DEC-97	250	F
3	Kareem	32	Orthopedic	19-FEB-98	200	M
4	Arun	12	Surgery	11-JAN-98	300	M
5	Zubin	30	ENT	12-JAN-98	250	M
6	Ketika	16	ENT	24-FEB-98	250	F
7	Ankita	29	Cardiology	20-FEB-98	800	F
8	Zareen	45	Gynecology	22-FEB-98	300	F
9	Kush	19	Cardiology	13-JAN-98	800	M
10	Shilpa	23	Nuclear Medicine	21-FEB-98	400	F
More than 10 rows available. Increase rows selector to view more rows.						

10 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(DISTINCT CHARGES)

FROM HOSPITAL;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCHARGES)
5

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MIN(AGE)

FROM HOSPITAL

WHERE SEX = 'F';

Results Explain Describe Saved SQL History

MIN(AGE)
16

1 rows returned in 0.00 seconds CSV Export

QUERY 9

```
SELECT SUM(CHARGES)
FROM HOSPITAL
WHERE DEPARTMENT = 'ENT';
```

Results Explain Describe Saved SQL History

SUM(CHARGES)
750

1 rows returned in 0.00 seconds CSV Export

QUERY 10

```
SELECT AVG(CHARGES)
FROM HOSPITAL
WHERE DATEOFADM < TO_DATE('12/08/98','DD/MM/RR');
```

Results Explain Describe Saved SQL History

AVG(CHARGES)
377.272727272727272727272727272727272727

1 rows returned in 0.00 seconds CSV Export

QUESTION 7

```
CREATE TABLE STORE (
   SNO VARCHAR2(5) PRIMARY KEY,
   SNAME VARCHAR2(50),
   AREA VARCHAR2(30)
);
CREATE TABLE ITEM (
   INO VARCHAR2(5) PRIMARY KEY,
   INAME VARCHAR2(50),
   PRICE NUMBER,
```

```
    SNO VARCHAR2(5)
);
INSERT INTO STORE VALUES ('S01', 'ABC Computronics', 'GK II');
INSERT INTO STORE VALUES ('S02', 'All Infotech Media', 'CP');
INSERT INTO STORE VALUES ('S03', 'Tech Shoppe', 'Nehru Place');
INSERT INTO STORE VALUES ('S04', 'Geeks Techno Soft', 'Nehru Place');
INSERT INTO STORE VALUES ('S05', 'Hitech Tech Store', 'CP');

COMMIT;
INSERT INTO ITEM VALUES ('T01', 'Mother Board', 12000, 'S01');
INSERT INTO ITEM VALUES ('T02', 'Hard Disk', 5000, 'S01');
INSERT INTO ITEM VALUES ('T03', 'Keyboard', 500, 'S02');
INSERT INTO ITEM VALUES ('T04', 'Mouse', 300, 'S01');
INSERT INTO ITEM VALUES ('T05', 'Mother Board', 13000, 'S02');
INSERT INTO ITEM VALUES ('T06', 'Keyboard', 400, 'S03');
INSERT INTO ITEM VALUES ('T07', 'LCD', 6000, 'S04');
INSERT INTO ITEM VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ITEM VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ITEM VALUES ('T10', 'Hard Disk', 4500, 'S03');

COMMIT;
QUERY 1
SELECT INAME, PRICE
FROM ITEM
ORDER BY PRICE ASC;
```

Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

QUERY 2

```
SELECT SNO, SNAME
FROM STORE
WHERE AREA = 'CP';
```

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All Infotech Media
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT INAME,
    MIN(PRICE) AS MIN_PRICE,
    MAX(PRICE) AS MAX_PRICE
FROM ITEM
GROUP BY INAME;
```

Results Explain Describe Saved SQL History

INAME	MIN_PRICE	MAX_PRICE
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT I.INAME, I.PRICE, S.SNAME
FROM ITEM I
JOIN STORE S
ON I.SNO = S.SNO;
```

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computronics
Hard Disk	5000	ABC Computronics
Keyboard	500	All Infotech Media
Mouse	300	ABC Computronics
Mother Board	13000	All Infotech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT DISTINCT INAME
FROM ITEM
WHERE PRICE >= 5000;
```

Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT AREA, COUNT(*)
FROM STORE
GROUP BY AREA;
```

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT COUNT(DISTINCT AREA)
FROM STORE;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT INAME, PRICE * 0.05 AS DISCOUNT
FROM ITEM
WHERE SNO IN ('S02', 'S03');
```

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

QUESTION 8

```
CREATE TABLE TEACHER (
   NO NUMBER,
   NAME VARCHAR2(30),
   AGE NUMBER,
   DEPARTMENT VARCHAR2(30),
   DATEOFJOIN DATE,
   SALARY NUMBER,
   SEX CHAR(1)
);
INSERT INTO TEACHER VALUES
(1, 'Jigal', 34, 'Computer',
 TO_DATE('10/01/97','DD/MM/RR'), 12000, 'M');

INSERT INTO TEACHER VALUES
(2, 'Sharmila', 31, 'History',
 TO_DATE('24/03/98','DD/MM/RR'), 20000, 'F');

INSERT INTO TEACHER VALUES
(3, 'Sandeep', 32, 'Maths',
```

TO_DATE('12/12/96','DD/MM/RR'), 30000, 'M');

INSERT INTO TEACHER VALUES

(4, 'Sangeeta', 35, 'History',

TO_DATE('01/07/99','DD/MM/RR'), 40000, 'F');

INSERT INTO TEACHER VALUES

(5, 'Rakesh', 42, 'Maths',

TO_DATE('05/09/97','DD/MM/RR'), 25000, 'M');

INSERT INTO TEACHER VALUES

(6, 'Shyam', 50, 'History',

TO_DATE('27/02/97','DD/MM/RR'), 30000, 'M');

INSERT INTO TEACHER VALUES

(7, 'Shiv Om', 44, 'Computer',

TO_DATE('25/02/97','DD/MM/RR'), 21000, 'M');

INSERT INTO TEACHER VALUES

(8, 'Shalakha', 33, 'Maths',

TO_DATE('31/07/97','DD/MM/RR'), 20000, 'F');

COMMIT;

QUERY 1

SELECT *

FROM TEACHER

WHERE DEPARTMENT = 'History';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	SEX
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-FEB-97	30000	M

3 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT NAME

FROM TEACHER

WHERE DEPARTMENT = 'Maths'

AND SEX = 'F';

Results Explain Describe Saved SQL History

NAME
Shalakha

1 rows returned in 0.02 seconds CSV Export

QUERY 3

SELECT NAME, DATEOFJOIN

FROM TEACHER

ORDER BY DATEOFJOIN ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFJOIN
Sandeep	12-DEC-96
Jigal	10-JAN-97
Shiv Om	25-FEB-97
Shyam	27-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Sangeeta	01-JUL-99

8 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT NAME, SALARY, AGE
FROM TEACHER
WHERE SEX = 'M';
```

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jigal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

5 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*)
FROM TEACHER
WHERE AGE > 23;
```

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
INSERT INTO TEACHER
VALUES
(9, 'Raja', 26, 'Computer',
 TO_DATE('13/05/95','DD/MM/RR'),
 2300, 'M');

COMMIT;
```

SELECT * FROM TEACHER;

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	SEX
1	Jigal	34	Computer	10-JAN-97	12000	M
2	Sharmila	31	History	24-MAR-98	20000	F
3	Sandeep	32	Maths	12-DEC-96	30000	M
4	Sangeeta	35	History	01-JUL-99	40000	F
5	Rakesh	42	Maths	05-SEP-97	25000	M
6	Shyam	50	History	27-FEB-97	30000	M
7	Shiv Om	44	Computer	25-FEB-97	21000	M
8	Shalakha	33	Maths	31-JUL-97	20000	F
9	Raja	26	Computer	13-MAY-95	2300	M

9 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(DISTINCT DEPARTMENT)

FROM TEACHER;

Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MAX(AGE)

FROM TEACHER

WHERE SEX = 'F';

Results Explain Describe Saved SQL History

MAX(AGE)
35

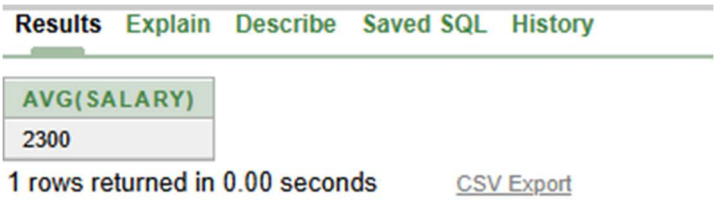
1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT AVG(SALARY)

FROM TEACHER

WHERE DATEOFJOIN < TO_DATE('12/07/96','DD/MM/RR');

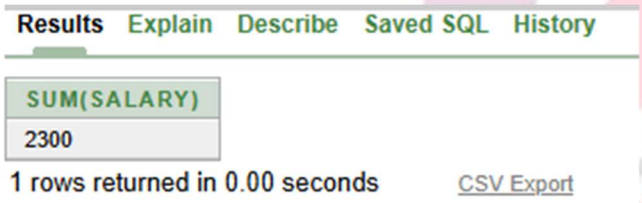


QUERY 10

SELECT SUM(SALARY)

FROM TEACHER

WHERE DATEOFJOIN < TO_DATE('12/07/96','DD/MM/RR');



QUESTION 9

```
CREATE TABLE BOOK (
   BOOK_ID VARCHAR2(10) PRIMARY KEY,
   BOOK_NAME VARCHAR2(50),
   AUTHOR_NAME VARCHAR2(50),
   PRICE NUMBER,
   PUBLISHERS VARCHAR2(30),
   TYPE VARCHAR2(20),
   QUANTITY NUMBER
);
```

```
CREATE TABLE ISSUED (
   BOOK_ID VARCHAR2(10),
   QUANTITY_ISSUED NUMBER
);
INSERT INTO BOOKS VALUES
('F0001', 'Computer Fundamentals', 'R.K. Jain', 450, 'EPB', 'Fiction', 10);

INSERT INTO BOOKS VALUES
('F0002', 'Database Concepts', 'Korth', 600, 'EPB', 'Non-Fiction', 8);

INSERT INTO BOOKS VALUES
('F0003', 'The Lost World', 'Arthur Doyle', 350, 'Penguin', 'Fiction', 6);

INSERT INTO BOOKS VALUES
('F0004', 'SQL Programming', 'Ivan Bayross', 550, 'EPB', 'Technical', 4);

INSERT INTO BOOKS VALUES
('F0005', 'Web Designing', 'Thomas Powell', 700, 'TechBooks', 'Technical', 7);

INSERT INTO BOOKS VALUES
('F0006', 'Great Stories', 'Mark Twain', 400, 'Penguin', 'Fiction', 9);

COMMIT;
INSERT INTO ISSUED VALUES ('F0001', 2);
INSERT INTO ISSUED VALUES ('F0003', 3);
```

INSERT INTO ISSUED VALUES ('F0005', 1);

COMMIT;

QUERY 1

```
SELECT BOOK_NAME, AUTHOR_NAME, PRICE
FROM BOOK
WHERE PUBLISHERS = 'EPB';
```

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
Database Concepts	Korth	600

1 rows returned in 0.02 seconds CSV Export

QUERY 2

```
SELECT BOOK_NAME
FROM BOOK
WHERE TYPE = 'Fiction';
```

Results Explain Describe Saved SQL History

BOOK_NAME
The Lost World
Great Stories

2 rows returned in 0.02 seconds CSV Export

QUERY 3

```
SELECT BOOK_NAME, PRICE
FROM BOOK
ORDER BY PRICE DESC;
```

Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
Web Designing	700
Database Concepts	600
Great Stories	400
The Lost World	350

4 rows returned in 0.00 seconds CSV Export

QUERY 4

```
INSERT INTO ISSUED
VALUES ('F0002', 4);


COMMIT;
SELECT * FROM ISSUED;
```

Results Explain Describe Saved SQL History

BOOK_ID	QUANTITY_ISSUED
F0001	2
F0003	3
F0005	1
F0002	4

4 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(DISTINCT PUBLISHERS)
FROM BOOK;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT SUM(PRICE)
FROM BOOK
WHERE QUANTITY > 5;
```

Results Explain Describe Saved SQL History

SUM(PRICE)
2050

1 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT BOOK_NAME, AUTHOR_NAME
FROM BOOK
WHERE PRICE < 500;
```

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
The Lost World	Arthur Doyle
Great Stories	Mark Twain

2 rows returned in 0.02 seconds CSV Export

QUERY 8

```
SELECT COUNT(*)
FROM BOOK;
```

Results Explain Describe Saved SQL History

COUNT(*)
0

1 rows returned in 0.01 seconds CSV Export

QUESTION 10

```
CREATE TABLE CARDEN (
   Ccode NUMBER PRIMARY KEY,
   CarName VARCHAR2(30),
   Make VARCHAR2(30),
   Color VARCHAR2(20),
   Capacity NUMBER,
   Charges NUMBER
);

CREATE TABLE CUSTOMER (
   Code NUMBER PRIMARY KEY,
   Cname VARCHAR2(30),
   Ccode NUMBER
);
INSERT INTO CARDEN VALUES (501, 'A-Star', 'Suzuki', 'RED', 3, 14);
INSERT INTO CARDEN VALUES (503, 'Indigo', 'Tata', 'SILVER', 3, 12);
INSERT INTO CARDEN VALUES (502, 'Innova', 'Toyota', 'WHITE', 7, 15);
INSERT INTO CARDEN VALUES (509, 'SX4', 'Suzuki', 'SILVER', 4, 14);
INSERT INTO CARDEN VALUES (510, 'C-Class', 'Mercedes', 'RED', 4, 35);

COMMIT;
INSERT INTO CUSTOMER VALUES (1001, 'Hemant Sahu', 501);
INSERT INTO CUSTOMER VALUES (1002, 'Raj Lal', 509);
INSERT INTO CUSTOMER VALUES (1003, 'Feroza Shah', 503);
```

INSERT INTO CUSTOMER VALUES (1004, 'Ketan Dhal', 502);

COMMIT;

QUERY 1

SELECT CarName

FROM CARDEN

WHERE Color = 'SILVER';

Results Explain Describe Saved SQL History

CARNAME
Indigo
SX4

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT CarName, Make, Capacity

FROM CARDEN

ORDER BY Capacity DESC;

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-Star	Suzuki	3

5 rows returned in 0.01 seconds CSV Export

QUERY 3

SELECT MAX(Charges) AS Highest_Charges

FROM CARDEN;

Results Explain Describe Saved SQL History

HIGHEST_CHARGES
35

1 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT COUNT(DISTINCT Make)
FROM CARDEN;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT MAX(Charges), MIN(Charges)
FROM CARDEN;
```

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT COUNT(*), Make
FROM CARDEN
GROUP BY Make;
```

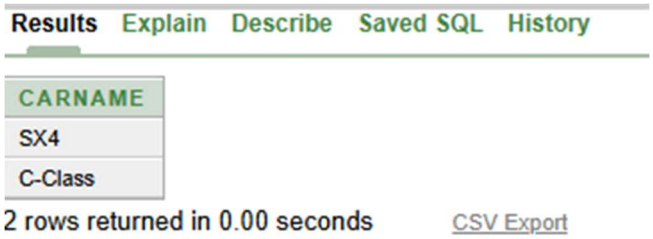
Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT CarName
FROM CARDEN
WHERE Capacity = 4;
```



Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

QUESTION 11

```
CREATE TABLE EMPLOYEE9
(
   ECODE NUMBER(3),
   NAME VARCHAR2(30),
   DESIGN VARCHAR2(20),
   SGRADE VARCHAR2(3),
   DOJ DATE,
   DOB DATE
);

INSERT INTO EMPLOYEE9 VALUES
(101,'Arun Kumar','MANAGER','S01',
TO_DATE('15-JAN-2005','DD-MON-YYYY'),
TO_DATE('10-MAR-1978','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9 VALUES
```

```
(102,'Meena Roy','ANALYST','S02',
TO_DATE('20-JUN-2007','DD-MON-YYYY'),
TO_DATE('25-MAY-1982','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9  VALUES
(103,'Ravi Kumar','CLERK','S03',
TO_DATE('08-AUG-2009','DD-MON-YYYY'),
TO_DATE('12-DEC-1985','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9  VALUES
(104,'Priya Das','EXECUTIVE','S02',
TO_DATE('12-MAR-2006','DD-MON-YYYY'),
TO_DATE('18-JUL-1980','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9 VALUES
(105,'Karthik Raj','OFFICER','S03',
TO_DATE('25-OCT-2010','DD-MON-YYYY'),
TO_DATE('20-JAN-1987','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9 VALUES
(106,'Divya Shah','HR','S01',
TO_DATE('05-MAY-2004','DD-MON-YYYY'),
TO_DATE('30-NOV-1979','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9 VALUES
```

```
(107,'Suresh Babu','DEVELOPER','S02',
TO_DATE('18-JUL-2008','DD-MON-YYYY'),
TO_DATE('14-FEB-1984','DD-MON-YYYY'));

INSERT INTO EMPLOYEE9 VALUES
(108,'Nisha Roy','TESTER','S03',
TO_DATE('10-DEC-2011','DD-MON-YYYY'),
TO_DATE('22-AUG-1988','DD-MON-YYYY'));

COMMIT;
CREATE TABLE SALGRADE
(
   SGRADE VARCHAR2(3),
   SALARY NUMBER(8,2),
   HRA NUMBER(8,2)
);

INSERT INTO SALGRADE VALUES ('S01',40000,8000);
INSERT INTO SALGRADE VALUES ('S02',50000,10000);
INSERT INTO SALGRADE VALUES ('S03',60000,12000);

COMMIT;
QUERY 1
SELECT *
FROM EMPLOYEE9
```

ORDER BY DOJ DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
108	Nisha Roy	TESTER	S03	10-DEC-11	22-AUG-88
105	Karthik Raj	OFFICER	S03	25-OCT-10	20-JAN-87
103	Ravi Kumar	CLERK	S03	08-AUG-09	12-DEC-85
107	Suresh Babu	DEVELOPER	S02	18-JUL-08	14-FEB-84
102	Meena Roy	ANALYST	S02	20-JUN-07	25-MAY-82
104	Priya Das	EXECUTIVE	S02	12-MAR-06	18-JUL-80
101	Arun Kumar	MANAGER	S01	15-JAN-05	10-MAR-78
106	Divya Shah	HR	S01	05-MAY-04	30-NOV-79

8 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT NAME, DESIGN

FROM EMPLOYEE

WHERE SGRADE IN ('S02','S03');

Results Explain Describe Saved SQL History

NAME	DESIGN
Meena Roy	ANALYST
Ravi Kumar	CLERK
Priya Das	EXECUTIVE
Karthik Raj	OFFICER
Suresh Babu	DEVELOPER
Nisha Roy	TESTER

6 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT *

FROM EMPLOYEE

WHERE DOJ BETWEEN

TO_DATE('09-FEB-2006','DD-MON-YYYY')

AND

TO_DATE('08-AUG-2009','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
102	Meena Roy	ANALYST	S02	20-JUN-07	25-MAY-82
103	Ravi Kumar	CLERK	S03	08-AUG-09	12-DEC-85
104	Priya Das	EXECUTIVE	S02	12-MAR-06	18-JUL-80
107	Suresh Babu	DEVELOPER	S02	18-JUL-08	14-FEB-84

4 rows returned in 0.00 seconds CSV Export

QUERY 4

INSERT INTO EMPLOYEE

VALUES

(109,'Harish Roy','HEAD-IT','S02',

TO_DATE('09-SEP-2007','DD-MON-YYYY'),

TO_DATE('21-APR-1983','DD-MON-YYYY'));

COMMIT;

QUERY 5

SELECT COUNT(SGRADE), SGRADE

FROM EMPLOYEE

GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S01
4	S02
3	S03

3 rows returned in 0.00 seconds CSV Export

QUERY 6

```
SELECT MIN(DOB), MAX(DOJ)
FROM EMPLOYEE;
```

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
10-MAR-78	10-DEC-11

1 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT SGRADE, SALARY + HRA
FROM SALGRADE
WHERE SGRADE = 'S02';
```

Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	60000

1 rows returned in 0.01 seconds CSV Export

QUESTION 12

```
CREATE TABLE SUPPLIERS
(
   Scode NUMBER(5) PRIMARY KEY,
   Sname VARCHAR2(30)
);


CREATE TABLE STORE
(
   ItemNo NUMBER(5) PRIMARY KEY,
```

```
    Item VARCHAR2(30),
    Scode NUMBER(5),
    Qty NUMBER(5),
    Rate NUMBER(10,2),
    LastBuy DATE,
    CONSTRAINT FK_STORE_SUPPLIER
    FOREIGN KEY (Scode) REFERENCES SUPPLIERS(Scode)
);
INSERT INTO SUPPLIERS VALUES (21, 'ABC Suppliers');
INSERT INTO SUPPLIERS VALUES (22, 'Global Traders');
INSERT INTO SUPPLIERS VALUES (23, 'Star Suppliers');
INSERT INTO SUPPLIERS VALUES (24, 'National Traders');

COMMIT;
INSERT INTO STORE VALUES
(2001, 'Pen', 21, 100, 10,
 TO_DATE('10-JAN-2024','DD-MON-YYYY'));

INSERT INTO STORE VALUES
(2002, 'Pencil', 22, 120, 8,
 TO_DATE('15-FEB-2024','DD-MON-YYYY'));

INSERT INTO STORE VALUES
(2003, 'Notebook', 23, 90, 25,
 TO_DATE('20-MAR-2024','DD-MON-YYYY'));
```

```
INSERT INTO STORE VALUES
(2004, 'Bag', 22, 50, 20,
 TO_DATE('12-APR-2024','DD-MON-YYYY'));

INSERT INTO STORE VALUES
(2005, 'Eraser', 24, 130, 5,
 TO_DATE('25-MAY-2024','DD-MON-YYYY'));

INSERT INTO STORE VALUES
(2006, 'Marker', 23, 80, 30,
 TO_DATE('10-JUN-2024','DD-MON-YYYY'));

COMMIT;
```

QUERY 1

```
SELECT *
FROM STORE
ORDER BY LastBuy ASC;
```

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Pen	21	100	10	10-JAN-24
2002	Pencil	22	120	8	15-FEB-24
2003	Notebook	23	90	25	20-MAR-24
2004	Bag	22	50	20	12-APR-24
2005	Eraser	24	130	5	25-MAY-24
2006	Marker	23	80	30	10-JUN-24

6 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT ItemNo, Item

FROM STORE

WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Notebook
2004	Bag
2006	Marker

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT *

FROM STORE

WHERE Scode = 22

OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2002	Pencil	22	120	8	15-FEB-24
2004	Bag	22	50	20	12-APR-24
2005	Eraser	24	130	5	25-MAY-24

3 rows returned in 0.02 seconds CSV Export

QUERY 4

SELECT Scode, MIN(Rate)

FROM STORE

GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	8
21	10
24	5
23	25

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(DISTINCT Scode)

FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
4

1 rows returned in 0.01 seconds CSV Export

QUERY 6

SELECT Rate * Qty

FROM STORE

WHERE ItemNo = 2004;

Results Explain Describe Saved SQL History

RATE*QTY
1000

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT MAX(LastBuy)

FROM STORE;

Results Explain Describe Saved SQL History

MAX(LASTBUY)
10-JUN-24

1 rows returned in 0.00 seconds CSV Export

QUESTION 14

```
CREATE TABLE FABRIC
(
   FCODE VARCHAR2(5) PRIMARY KEY,
   FTYPE VARCHAR2(30)
);
CREATE TABLE GARMENT
(
   GCODE VARCHAR2(5) PRIMARY KEY,
   DESCRIPTION VARCHAR2(40),
   TYPE VARCHAR2(20),
   FCODE VARCHAR2(5),
   PRICE NUMBER(10,2),
   READYDATE DATE,
   CONSTRAINT FK_GARMENT_FABRIC
   FOREIGN KEY (FCODE) REFERENCES FABRIC(FCODE)
);
INSERT INTO FABRIC VALUES ('F01','Cotton');
INSERT INTO FABRIC VALUES ('F02','Silk');
INSERT INTO FABRIC VALUES ('F03','Wool');
INSERT INTO FABRIC VALUES ('F04','Polyester');
```

```
INSERT INTO FABRIC VALUES ('F05','Nylon');

COMMIT;
INSERT INTO GARMENT VALUES
('G01','Cotton Shirt','Shirt','F01',950,
 TO_DATE('10-NOV-07','DD-MON-RR'));

INSERT INTO GARMENT VALUES
('G02','Silk Saree','Saree','F02',1500,
 TO_DATE('08-DEC-07','DD-MON-RR'));

INSERT INTO GARMENT VALUES
('G03','Wool Sweater','Sweater','F03',1250,
 TO_DATE('15-JAN-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES
('G04','Designer Suit','Suit','F02',1800,
 TO_DATE('16-JUN-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES
('G05','Wool Coat','Coat','F03',1400,
 TO_DATE('20-MAY-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES
('G06','Nylon Jacket','Jacket','F05',1100,
```

TO_DATE('25-JUL-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES

('G07','Cotton Pant','Pant','F01',1200,

TO_DATE('05-AUG-08','DD-MON-RR'));

COMMIT;

QUERY 1

SELECT GCODE, DESCRIPTION

FROM GARMENT

ORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
G07	Cotton Pant
G06	Nylon Jacket
G05	Wool Coat
G04	Designer Suit
G03	Wool Sweater
G02	Silk Saree
G01	Cotton Shirt

7 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT *

FROM GARMENT

WHERE READYDATE BETWEEN

TO_DATE('08-DEC-07','DD-MON-RR')

AND

TO_DATE('16-JUN-08','DD-MON-RR');

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	TYPE	FCODE	PRICE	READYDATE
G02	Silk Saree	Saree	F02	1500	08-DEC-07
G03	Wool Sweater	Sweater	F03	1250	15-JAN-08
G04	Designer Suit	Suit	F02	1800	16-JUN-08
G05	Wool Coat	Coat	F03	1400	20-MAY-08

4 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT AVG(PRICE)
FROM GARMENT
WHERE FCODE = 'F03';
```

Results Explain Describe Saved SQL History

AVG(PRICE)
1325

1 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT FCODE, MAX(PRICE) AS HIGHEST_PRICE,
    MIN(PRICE) AS LOWEST_PRICE
FROM GARMENT
GROUP BY FCODE;
```

Results Explain Describe Saved SQL History

FCODE	HIGHEST_PRICE	LOWEST_PRICE
F02	1800	1500
F03	1400	1250
F05	1100	1100
F01	1200	950

4 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT SUM(PRICE)
```

FROM GARMENT

WHERE FCODE = 'F01';

Results Explain Describe Saved SQL History

SUM(PRICE)
2150

1 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT DESCRIPTION, TYPE

FROM GARMENT, FABRIC

WHERE GARMENT.FCODE = FABRIC.FCODE

AND GARMENT.PRICE >= 1260;

Results Explain Describe Saved SQL History

SUM(PRICE)
2150

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT MAX(FCODE)

FROM FABRIC;

Results Explain Describe Saved SQL History

MAX(FCODE)
F05

1 rows returned in 0.02 seconds CSV Export

QUERY 8

SELECT COUNT(DISTINCT PRICE)

FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.00 seconds CSV Export

QUESTION 15

```
CREATE TABLE FURNITURES
(
   ITEMNO NUMBER(5) PRIMARY KEY,
   ITEMNAME VARCHAR2(40),
   TYPE VARCHAR2(20),
   DATEOFSTOCK DATE,
   PRICE NUMBER(10,2),
   DISCOUNT NUMBER(5,2)
);
CREATE TABLE ARRIVAL
(
   ITEMNO NUMBER(5) PRIMARY KEY,
   ITEMNAME VARCHAR2(40),
   TYPE VARCHAR2(20),
   DATEOFARRIVAL DATE,
   PRICE NUMBER(10,2),
   DISCOUNT NUMBER(5,2)
);
INSERT INTO FURNITURES VALUES
(1,'Baby Cot','Baby cot',
```

```
 TO_DATE('10-JAN-02','DD-MON-RR'),12000,20);

INSERT INTO FURNITURES VALUES
(2,'Wooden Sofa','Sofa',
 TO_DATE('15-FEB-02','DD-MON-RR'),18000,30);

INSERT INTO FURNITURES VALUES
(3,'Royal Sofa','Sofa',
 TO_DATE('05-JAN-02','DD-MON-RR'),22000,35);

INSERT INTO FURNITURES VALUES
(4,'Study Table','Table',
 TO_DATE('20-MAR-02','DD-MON-RR'),10000,15);

INSERT INTO FURNITURES VALUES
(5,'Baby Cradle','Baby cot',
 TO_DATE('12-DEC-01','DD-MON-RR'),14000,28);

INSERT INTO FURNITURES VALUES
(6,'Dining Table','Table',
 TO_DATE('08-JAN-02','DD-MON-RR'),16000,25);

COMMIT;S
INSERT INTO ARRIVALS VALUES
(10,'Classic Chair','Chair',
```

TO_DATE('10-JAN-03','DD-MON-RR'),8000,10);

INSERT INTO ARRIVALS VALUES

(11,'Modern Sofa','Sofa',

TO_DATE('15-FEB-03','DD-MON-RR'),20000,20);

INSERT INTO ARRIVALS VALUES

(12,'Wooden Cot','Baby cot',

TO_DATE('20-FEB-03','DD-MON-RR'),15000,25);

INSERT INTO ARRIVALS VALUES

(13,'King Bed','Double bed',

TO_DATE('10-MAR-03','DD-MON-RR'),30000,35);

COMMIT;

QUERY 1

SELECT *

FROM FURNITURES

WHERE TYPE = 'Baby cot';

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
1	Baby Cot	Baby cot	10-JAN-02	12000	20
5	Baby Cradle	Baby cot	12-DEC-01	14000	28

2 rows returned in 0.00 seconds CSV Export

QUIERY 2

SELECT ITEMNAME

FROM FURNITURES

WHERE PRICE > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
Wooden Sofa
Royal Sofa
Dining Table

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT ITEMNAME, TYPE

FROM FURNITURE

WHERE DATEOFSTOCK < TO_DATE('22-JAN-02','DD-MON-RR')

ORDER BY ITEMNAME DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Royal Sofa	Sofa
Dining Table	Table
Baby Cradle	Baby cot
Baby Cot	Baby cot

4 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT ITEMNAME, DATEOFSTOCK

FROM FURNITURE

WHERE DISCOUNT > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Wooden Sofa	15-FEB-02
Royal Sofa	05-JAN-02
Baby Cradle	12-DEC-01

3 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*)
FROM FURNITURE
WHERE TYPE = 'Sofa';
```

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
INSERT INTO ARRIVALS
VALUES
(
   14,
   'Velvet touch',
   'Double bed',
   TO_DATE('25-MAR-03','DD-MON-RR'),
   25000,
   30
);

COMMIT;
```

```
SELECT *
FROM ARRIVAL
WHERE ITEMNO = 14;
```

Results | Explain | Describe | Saved SQL | History

ITEMNO	ITEMNAME	TYPE	DATEOFARRIVAL	PRICE	DISCOUNT
14	Velvet touch	Double bed	25-MAR-03	25000	30

1 rows returned in 0.00 seconds CSV Export

QUERY 7

```
SELECT COUNT(DISTINCT TYPE)
FROM FURNITURE;
```

Results | Explain | Describe | Saved SQL | History

COUNT(DISTINCTTYPE)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT AVG(DISCOUNT)
FROM FURNITURE
WHERE TYPE = 'Baby cot';
```

Results | Explain | Describe | Saved SQL | History

AVG(DISCOUNT)
24

1 rows returned in 0.00 seconds CSV Export

QUERY 9

```
SELECT SUM(PRICE)
FROM FURNITURE
WHERE DATEOFSTOCK < TO_DATE('12-FEB-02','DD-MON-RR');
```

Results Explain Describe Saved SQL History

SUM(PRICE)
-

1 rows returned in 0.00 seconds CSV Export

QUESTION 16

```
CREATE TABLE COVID19
(
   PatientID NUMBER PRIMARY KEY,
   PatientName VARCHAR2(30),
   State VARCHAR2(30),
   Age NUMBER,
   Status VARCHAR2(20),
   Vaccinated VARCHAR2(5)
);

INSERT INTO COVID19 VALUES
(101, 'Arun', 'Tamil Nadu', 25, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES
(102, 'Meena', 'Kerala', 32, 'Active', 'Yes');

INSERT INTO COVID19 VALUES
(103, 'Kavin', 'Tamil Nadu', 45, 'Active', 'No');

INSERT INTO COVID19 VALUES
```

(104, 'Divya', 'Karnataka', 28, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES

(105, 'Ravi', 'Tamil Nadu', 52, 'Recovered', 'No');

INSERT INTO COVID19 VALUES

(106, 'Priya', 'Andhra Pradesh', 35, 'Active', 'Yes');

INSERT INTO COVID19 VALUES

(107, 'Suresh', 'Tamil Nadu', 60, 'Active', 'No');

COMMIT;

QUERY 1

```
SELECT *
FROM COVID19
WHERE State = 'Tamil Nadu';
```

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	25	Recovered	Yes
103	Kavin	Tamil Nadu	45	Active	No
105	Ravi	Tamil Nadu	52	Recovered	No
107	Suresh	Tamil Nadu	60	Active	No

4 rows returned in 0.00 seconds CSV Export

QUERY 2

```
SELECT PatientName
FROM COVID19
WHERE Status = 'Recovered';
```

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Divya
Ravi

3 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT PatientName, Age
FROM COVID19
ORDER BY Age ASC;
```

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Arun	25
Divya	28
Meena	32
Priya	35
Kavin	45
Ravi	52
Suresh	60

7 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT PatientName
FROM COVID19
WHERE Vaccinated = 'Yes';
```

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Meena
Divya
Priya

4 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*) AS Active_Patients
FROM COVID19
WHERE Status = 'Active';
```

Results Explain Describe Saved SQL History

ACTIVE_PATIENTS
4

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
INSERT INTO COVID19
VALUES
(108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');


COMMIT;
```

QUERY 7

```
SELECT COUNT(*)
FROM COVID19;
```

Results Explain Describe Saved SQL History

COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT MAX(Age)
FROM COVID19;
```

Results Explain Describe Saved SQL History

MAX(AGE)
60

1 rows returned in 0.00 seconds CSV Export

QUERY 9

```
SELECT COUNT(*)
FROM COVID19
WHERE Vaccinated = 'Yes';
```

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

QUERY 10

```
SELECT MIN(Age)
FROM COVID19
WHERE Status = 'Recovered';
```

Results Explain Describe Saved SQL History

MIN(AGE)
25

1 rows returned in 0.00 seconds CSV Export

QUESTION 17

```
CREATE TABLE HOUSEPRICE
(
   HouseID NUMBER PRIMARY KEY,
   Location VARCHAR2(30),
   Bedrooms NUMBER,
   AreaSqft NUMBER,
   Price NUMBER,
   HouseType VARCHAR2(20)
);

INSERT INTO HOUSEPRICE VALUES
(201, 'Chennai', 2, 1200, 5500000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES
(202, 'Chennai', 3, 1500, 7500000, 'Villa');

INSERT INTO HOUSEPRICE VALUES
(203, 'Coimbatore', 3, 1800, 6800000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES
(204, 'Madurai', 2, 1100, 4500000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES
(205, 'Chennai', 4, 2200, 9500000, 'Villa');
```

INSERT INTO HOUSEPRICE VALUES

(206, 'Salem', 3, 1600, 6200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES

(207, 'Trichy', 4, 2000, 8000000, 'Villa');

COMMIT;

QUERY 1

SELECT *

FROM HOUSEPRICE

WHERE Location = 'Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2200	9500000	Villa

3 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT Location, Price

FROM HOUSEPRICE

WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

LOCATION	PRICE
Chennai	7500000
Coimbatore	6800000
Salem	6200000

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT Location, AreaSqft, Price

FROM HOUSEPRICE

ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9500000
Trichy	2000	8000000
Chennai	1500	7500000
Coimbatore	1800	6800000
Salem	1600	6200000
Chennai	1200	5500000
Madurai	1100	4500000

7 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT *

FROM HOUSEPRICE

WHERE Price > 7000000;

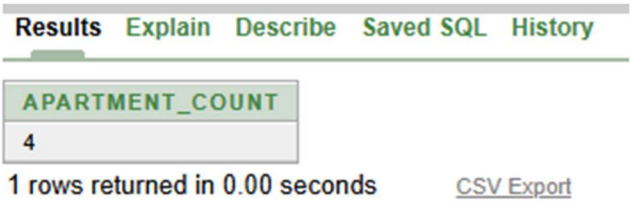
Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2200	9500000	Villa
207	Trichy	4	2000	8000000	Villa

3 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*) AS Apartment_Count
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';
```



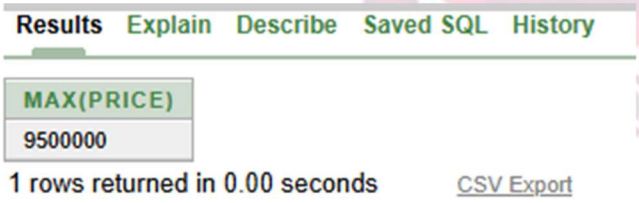
QUERY 6

```
INSERT INTO HOUSEPRICE
VALUES
(208, 'Vellore', 3, 1700, 7300000, 'Villa');
```

```
COMMIT;
```

QUERY 7

```
SELECT MAX(Price)
FROM HOUSEPRICE;
```



QUERY 8

```
SELECT MIN(AreaSqft)
FROM HOUSEPRICE;
```

Results Explain Describe Saved SQL History

MIN(AREASQFT)
1100

1 rows returned in 0.00 seconds CSV Export

QUERY 9

```
SELECT AVG(Price)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';
```

Results Explain Describe Saved SQL History

AVG(PRICE)
5750000

1 rows returned in 0.00 seconds CSV Export

QUERY 10

```
SELECT COUNT(*)
FROM HOUSEPRICE
WHERE Bedrooms = 3;
```

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

QUESTION 18

```
CREATE TABLE CANCERDATA
(
   PatientID NUMBER PRIMARY KEY,
   PatientName VARCHAR2(30),
   CancerType VARCHAR2(30),
   Age NUMBER,
   Stage VARCHAR2(5),
   SurvivalYears NUMBER
);

INSERT INTO CANCERDATA VALUES
(301, 'Arun', 'Breast', 45, 'II', 6);

INSERT INTO CANCERDATA VALUES
(302, 'Meena', 'Lung', 52, 'III', 4);

INSERT INTO CANCERDATA VALUES
(303, 'Kavin', 'Breast', 38, 'I', 8);

INSERT INTO CANCERDATA VALUES
(304, 'Divya', 'Lung', 60, 'II', 3);

INSERT INTO CANCERDATA VALUES
(305, 'Ravi', 'Colon', 55, 'III', 7);
```

INSERT INTO CANCERDATA VALUES

(306, 'Priya', 'Breast', 42, 'II', 5);

INSERT INTO CANCERDATA VALUES

(307, 'Suresh', 'Lung', 48, 'I', 9);

COMMIT;

QUERY 1

SELECT *

FROM CANCERDATA

WHERE CancerType = 'Breast';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Arun	Breast	45	II	6
303	Kavin	Breast	38	I	8
306	Priya	Breast	42	II	5

3 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT PatientName

FROM CANCERDATA

WHERE Stage = 'III';

Results Explain Describe Saved SQL History

PATIENTNAME
Meena
Ravi

2 rows returned in 0.02 seconds CSV Export

QUERY 3

```
SELECT PatientName, Age
FROM CANCERDATA
ORDER BY Age ASC;
```

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Kavin	38
Priya	42
Arun	45
Suresh	48
Meena	52
Ravi	55
Divya	60

7 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT PatientName
FROM CANCERDATA
WHERE SurvivalYears > 5;
```

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Kavin
Ravi
Suresh

4 rows returned in 0.00 seconds CSV Export

QUERY 5

```
SELECT COUNT(*) AS Lung_Cancer_Count
FROM CANCERDATA
WHERE CancerType = 'Lung';
```

Results Explain Describe Saved SQL History

LUNG_CANCER_COUNT
3

1 rows returned in 0.00 seconds CSV Export

QUERY 6

```
INSERT INTO CANCERDATA
VALUES
(308, 'Anand', 'Leukemia', 50, 'II', 7);


COMMIT;
```

QUERY 7

```
SELECT COUNT(DISTINCT CancerType)
FROM CANCERDATA;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT MAX(SurvivalYears)
FROM CANCERDATA;
```

Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
9

1 rows returned in 0.01 seconds CSV Export

QUERY 9

```
SELECT AVG(Age)
FROM CANCERDATA
WHERE CancerType = 'Breast';
```

Results Explain Describe Saved SQL History

AVG(AGE)
41.666666666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

QUERY 10

```
SELECT MIN(Age)
FROM CANCERDATA
WHERE Stage = 'II';
```

Results Explain Describe Saved SQL History

MIN(AGE)
42

1 rows returned in 0.00 seconds CSV Export

QUESTION 19

```
CREATE TABLE ITEMS
(
   CODE NUMBER PRIMARY KEY,
   INAME VARCHAR2(30),
   QTY NUMBER,
   PRICE NUMBER,
   COMPANY VARCHAR2(30),
   TCODE VARCHAR2(5)
```

```
);
INSERT INTO ITEMS VALUES
(1001, 'DIGITAL PAD 12i', 120, 11000, 'XENITA', 'T01');

INSERT INTO ITEMS VALUES
(1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T02');

INSERT INTO ITEMS VALUES
(1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T01');

INSERT INTO ITEMS VALUES
(1003, 'DIGITAL CAMERA', 160, 8000, 'DIGICLICK', 'T02');

INSERT INTO ITEMS VALUES
(1005, 'PEN DRIVE 32GB', 600, 1200, 'STOREHOME', 'T03');

COMMIT;
CREATE TABLE TRADERS
(
   TCODE VARCHAR2(5) PRIMARY KEY,
   TNAME VARCHAR2(30),
   CITY VARCHAR2(20)
);
INSERT INTO TRADERS VALUES
('T01', 'ELECTRONIC SALES', 'MUMBAI');
```

INSERT INTO TRADERS VALUES

('T03', 'BUSY STORE CORP', 'DELHI');

INSERT INTO TRADERS VALUES

('T02', 'DISP HOUSE INC', 'CHENNAI');

COMMIT;

QUERY 1

SELECT *

FROM ITEMS

ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA	160	8000	DIGICLICK	T02
1001	DIGITAL PAD 12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32GB	600	1200	STOREHOME	T03

5 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT INAME, PRICE

FROM ITEMS

WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD 12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds CSV Export

QUERY 3

```
SELECT TCODE, COUNT(*) AS ITEM_COUNT
FROM ITEMS
GROUP BY TCODE
ORDER BY TCODE;
```

Results Explain Describe Saved SQL History

TCODE	ITEM_COUNT
T01	2
T02	2
T03	1

3 rows returned in 0.00 seconds CSV Export

QUERY 4

```
SELECT PRICE, INAME, QTY
FROM ITEMS
WHERE QTY > 150;
```

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA	160
1200	PEN DRIVE 32GB	600

2 rows returned in 0.01 seconds CSV Export

QUERY 5

```
SELECT TNAME
```

FROM TRADERS

WHERE CITY IN ('DELHI', 'MUMBAI');

Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT COMPANY, INAME

FROM ITEMS

ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD 12i
STOREHOME	PEN DRIVE 32GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA

5 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT MAX(PRICE), MIN(PRICE)

FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

QUERY 8

```
SELECT PRICE * QTY AS AMOUNT
FROM ITEMS
WHERE CODE = 1004;
```

Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

QUERY 9

```
SELECT DISTINCT TCODE
FROM ITEMS
ORDER BY TCODE;
```

Results Explain Describe Saved SQL History

TCODE
T01
T02
T03

3 rows returned in 0.00 seconds CSV Export

QUERY 10

```
SELECT I.INAME, T.TNAME
FROM ITEMS I, TRADERS T
WHERE I.TCODE = T.TCODE
AND I.QTY < 100;
```

Results Explain Describe Saved SQL History

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC

2 rows returned in 0.00 seconds CSV Export